

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 2 — Servizi, Analisi e Database

Coaching Services, Knowledge & Retrieval, Analysis Engines, Database
Schema, Training Pipeline

Autore: Renan Augusto Macena

Agosto 2026

Ultimate CS2 Coach — Parte 2: Servizi, Analisi, Conoscenza, Controllo, Progresso e Database

Argomenti: Servizi di Coaching (pipeline 4-modi: COPER/Ibrido/RAG/Base, Ollama LLM), Motori di Coaching (11 motori di analisi game-theoretic e statistica), Conoscenza e Recupero (RAG + Experience Bank COPER), Motori di analisi (belief model bayesiano, expectiminimax, momentum, ruoli, deception index, entropia, engagement range, utility economy, win probability), Elaborazione e Feature Engineering (vettore canonico 25-dim, baselines professionali), Modulo di Controllo (lifecycle daemon, code ingestione, controllo ML), Progresso e Tendenze (tracking longitudinale), Database e Storage (schema SQLite WAL three-tier), Pipeline di Addestramento e Orchestrazione, Funzioni di Perdita.

Autore: Renan Augusto Macena

Questo documento è la continuazione della **Parte 1B** — *I Sensi e lo Specialista*, che copre il modello RAP Coach e tutte le sorgenti dati esterne. La Parte 2 documenta i servizi di coaching che consumano quegli output neurali e li trasformano in consigli azionabili, insieme ai motori di analisi, alla conoscenza tattica e all'infrastruttura di storage e addestramento.

Indice

Parte 2 — Servizi, Analisi e Database (questo documento)

5. **Sottosistema 3 — Servizi di Coaching** ([backend/services/](#))
 - CoachingService (pipeline 4-modi)
 - OllamaWriter (raffinamento LLM)
 - AnalysisOrchestrator
 - Server FastAPI (endpoint REST standalone)
 - Onboarding (flusso utente a 3 stadi) 5B. **Sottosistema 3B — Motori di Coaching** ([backend/coaching/](#))
6. **Sottosistema 4 — Conoscenza e Recupero** ([backend/knowledge/](#))
 - RAG Knowledge (retrieval per pattern di coaching)
 - Experience Bank COPER (storage e retrieval di esperienze)
7. **Sottosistema 5 — Motori di analisi** ([backend/analysis/](#))

- Belief Model bayesiano
- Game Tree (expectiminimax)
- Momentum, Role Classifier, Deception Index, Entropy Analysis
- Engagement Range, Utility Economy, Win Probability, Blind Spots

8. Sottosistema 6 — Elaborazione e Feature Engineering (`backend/processing/`)

- Vectorizer (contratto canonico METADATA_DIM=25)
- TensorFactory (costruzione tensor visione/mappa)
- Baselines professionali e meta drift detection
- Heatmap, validazione

9. Sottosistema 7 — Modulo di Controllo (`backend/control/`)

10. Sottosistema 8 — Progresso e Tendenze (`backend/progress/`)

11. Sottosistema 9 — Database e Storage (`backend/storage/`)

12. Pipeline di Addestramento e Orchestrazione

13. Funzioni di Perdita — Dettaglio Implementativo

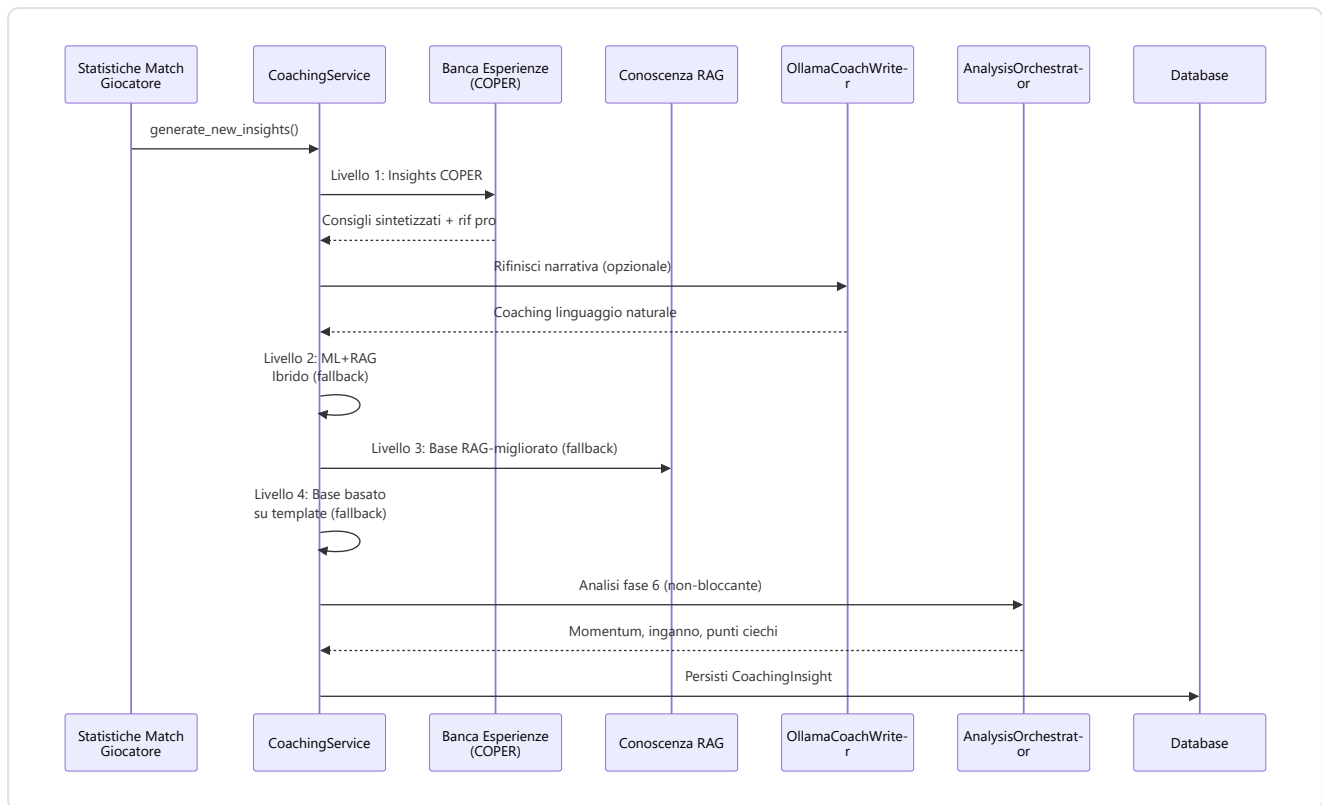
5. Sottosistema 3 — Servizi di Coaching

Cartella nella repo: `backend/services/` **File:** `coaching_service.py` , `ollama_writer.py` , `analysis_orchestrator.py`

Questo sottosistema è il **centro decisionale**: prende tutto ciò che le reti neurali e i sistemi di conoscenza hanno prodotto e lo sintetizza in veri e propri consigli di coaching per il giocatore.

-CoachingService (`coaching_service.py`)

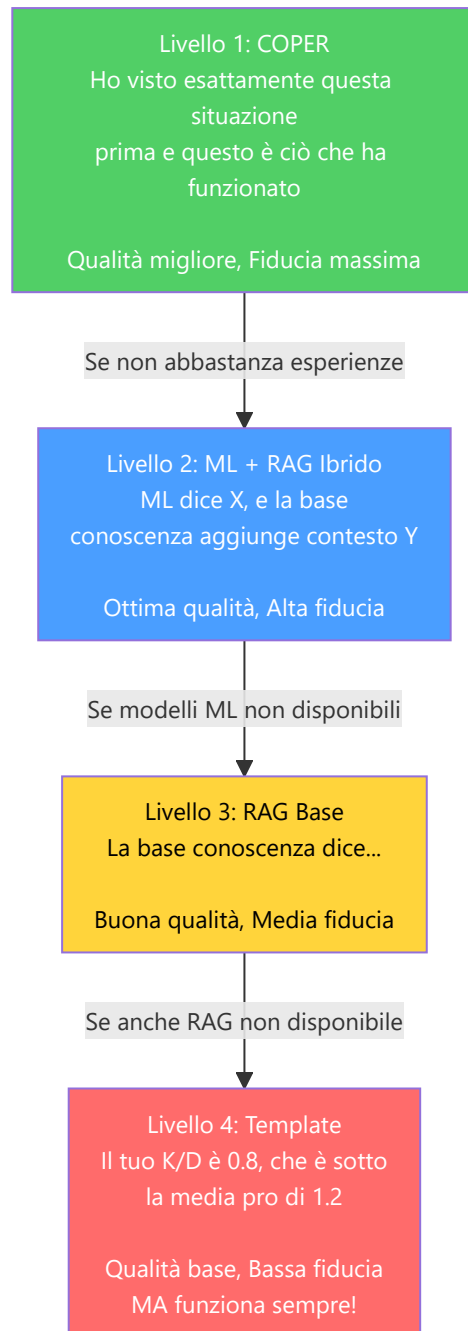
Il **motore di sintesi centrale** che implementa una **catena di fallback di coaching a 4 livelli** con degradazione graduale:



Spiegazione Diagramma: Segui le frecce dall'alto verso il basso: questo è il "processo di pensiero" dell'allenatore in ordine: (1) Arrivano i dati della partita. (2) L'allenatore chiede prima all'Experience Bank: "Abbiamo già visto questa situazione? Cosa ha funzionato?". (3) Se la risposta sembra troppo robotica, la invia facoltativamente a Ollama (un programmatore di intelligenza artificiale locale) per renderla più naturale. (4) Se l'Experience Bank non dispone di dati sufficienti, ricorre alla modalità ibrida (previsioni ML + conoscenza RAG combinate). (5) Se anche i modelli ML non sono disponibili, ricorre solo a RAG (cercando suggerimenti pertinenti). (6) Se anche RAG fallisce, utilizza semplici modelli statistici ("Il tuo rapporto K/D è 0,8, sotto la media"). (7) Nel frattempo, in background, 7 motori di analisi eseguono indagini speciali attraverso 5 pipeline (momentum, inganno, entropia, strategia+punti ciechi, distanza ingaggio). (8) Tutto viene salvato nel database per riferimento futuro.

Catena di fallback a 4 livelli:

Livello	Metodo	Fiducia	Quando utilizzato
1. COPER	<code>_generate_coper_insights()</code>	Massima	Se <code>USE_COPER_COACHING</code> (default True) + mappa + tick data disponibili; eseguito con timeout di 30s (<code>_COACHING_TIMEOUT</code>), allo scadere degrada al livello 2
2. Ibrido	<code>_generate_hybrid_insights()</code>	Alta	Se <code>USE_HYBRID_COACHING</code> (default False) + player_stats disponibili
3. RAG Base	<code>_enhance_with_rag()</code>	Media	Percorso tradizionale (<code>generate_corrections</code>) arricchito da RAG se <code>USE_RAG_COACHING</code> (default False) + mappa
4. Template	Modello statistico di base	Bassa	Ultima risorsa — restituisce sempre <i>qualcosa</i> (<code>_save_generic_insight</code> , guardia C-01)



Progettazione chiave: Non restituisce mai zero insight. L'analisi di Fase 6 è non bloccante (incapsulata in try-catch, registrata in modo non fatale).

Correzione G-08 (Coaching Fallback): In precedenza, il metodo `_generate_coper_insights()` non riceveva i parametri `deviations` e `rounds_played` dal chiamante, causando un fallback silenzioso ai template di base. Dopo la rimediazione, `generate_new_insights()` ora passa esplicitamente entrambi i parametri (`deviations=deviations, rounds_played=rounds_played`) al handler COPER, garantendo che il Livello 1 COPER possa generare insight contestuali basati sulle deviazioni statistiche reali del giocatore rispetto alla baseline pro. Senza questa correzione, il sistema avrebbe sempre degradato al Livello 4 (template) anche quando i dati COPER erano disponibili.

Arricchimento della baseline temporale (Proposta 11): Il servizio di coaching ora integra confronti pro ponderati nel tempo tramite due nuovi metodi:

- `_get_temporal_baseline(map_name)` — recupera una baseline pro ponderata in base a `TemporalBaselineDecay` (emivita = 90 giorni) invece di medie statiche
- `_baseline_context_note(deviations, map_name)` — genera una descrizione in linguaggio naturale ("In base ai dati pro recenti ponderati in base alla recenza, il tuo ADR è inferiore del 12% rispetto alla meta media attuale su de_mirage") per l'arricchimento COPER

Questo garantisce che gli insight di coaching riflettano la **meta media attuale** anziché medie storiche obsolete. Se i dati temporali non sono sufficienti (< 10 schede statistiche), il servizio ricorre alla funzione legacy `get_pro_baseline()` in modo trasparente.

Inferenza della fase del round (`_infer_round_phase`): Valore dell'attrezzatura → classificazione della fase del round:

Valore dell'attrezzatura	Fase del round
< \$1.500	pistol
\$1.500 – \$2.999	eco
\$ 3.000 – \$ 3.999	force
≥ \$ 4.000	full_buy

Classificazione dell'intervallo di salute (`_health_to_range`): Utilizzato per l'hashing del contesto COPER: "full" (≥80), "damaged" (40–79), "critical" (<40).

-OllamaCoachWriter (`ollama_writer.py`)

Trasforma le informazioni di coaching strutturate in linguaggio naturale tramite LLM locale (Ollama).

- **Singleton** tramite factory `get_ollama_writer()`
- **Funzionalità contrassegnata:** impostazione `USE_OLLAMA_COACHING` (predefinita: False)
- **Degradazione graduale:** restituisce il testo originale se Ollama non è disponibile
- **Prompt di sistema:** tono da esperto di coaching CS2, <100 parole, fattibile, incoraggiante

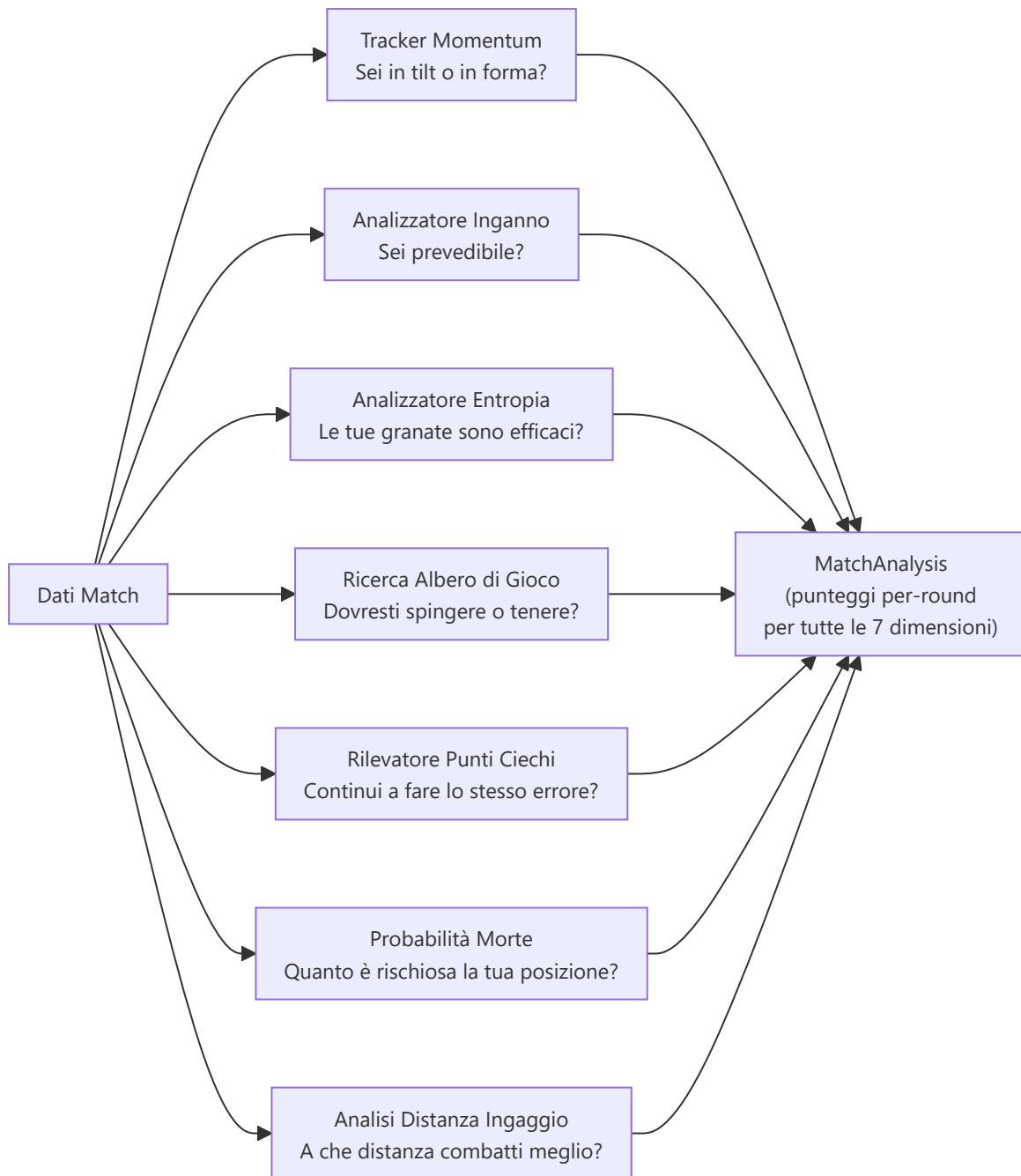
-AnalysisOrchestrator (`analysis_orchestrator.py`)

Sintetizza l'analisi avanzata di Fase 6 istanziando 7 motori e orchestrando 5 pipeline di analisi:

Input: Dati di tick della partita, eventi, statistiche dei giocatori **Output:** `MatchAnalysis` con oggetti `RoundAnalysis` per round contenenti:

- `momentum_score` (tilt/striscia vincente)
- `deception_score` (s sofisticatezza tattica)
- `utility_entropy` (misurazione dell'efficacia)
- `blind_spots` (lacune strategiche)
- `strategy_rec` (raccomandazione dell'albero di gioco)
- `engagement_range` (analisi distanza di ingaggio)

Motori istanziati: `belief_estimator`, `deception_analyzer`, `momentum_tracker`, `entropy_analyzer`, `game_tree`, `blind_spot_detector`, `engagement_analyzer` (7 motori). Il `belief_estimator` è istanziato ma attualmente non invocato direttamente come pipeline separata.



-Servizi Aggiuntivi (Non documentati in precedenza)

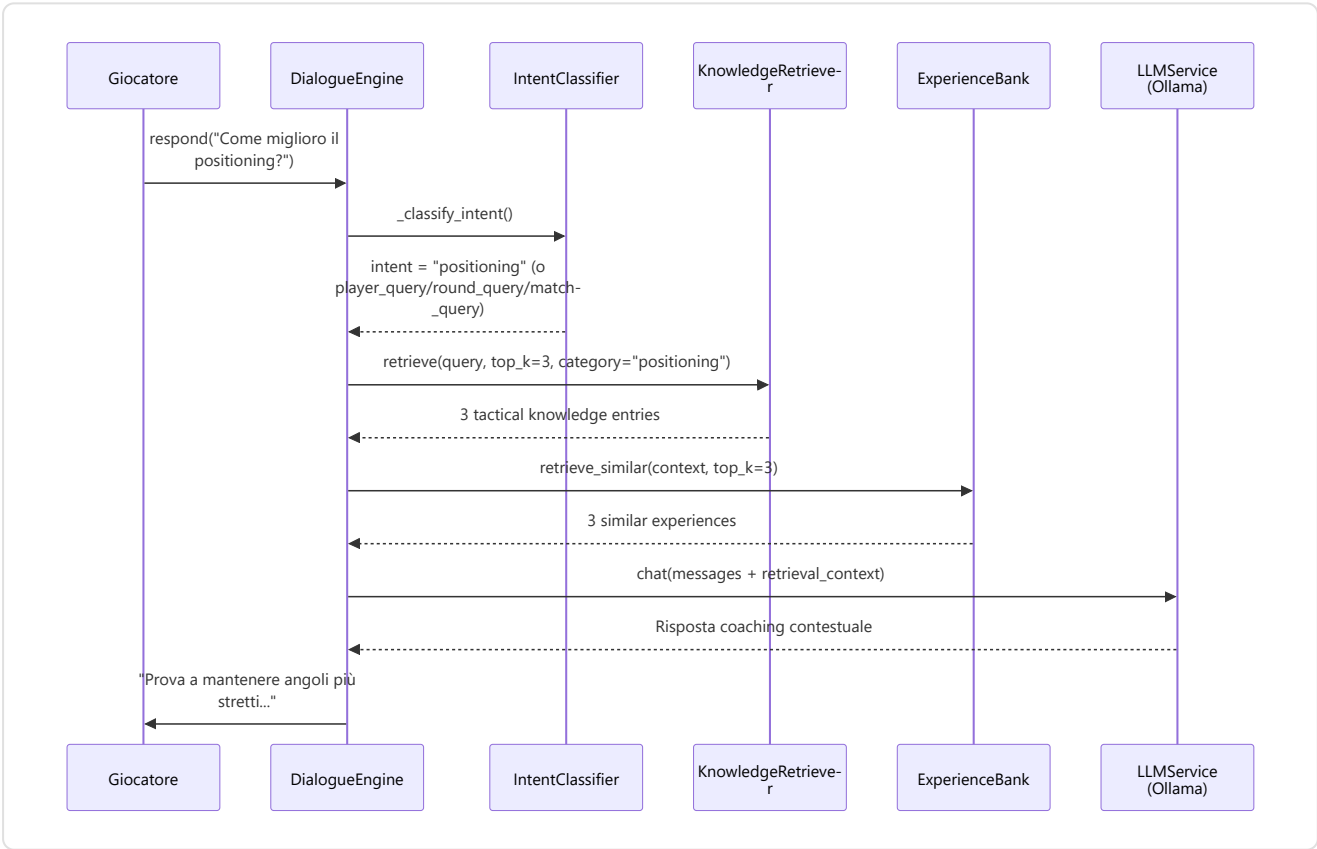
Oltre ai tre servizi principali (CoachingService, OllamaCoachWriter, AnalysisOrchestrator), la directory `backend/services/` contiene **8 servizi aggiuntivi** che completano l'ecosistema di coaching:

CoachingDialogueEngine (`coaching_dialogue.py`)

Motore di dialogo multi-turno con augmentazione RAG e Experience Bank. Evolve il single-shot OllamaCoachWriter in una **sessione interattiva** dove i giocatori possono fare domande follow-up sulle loro prestazioni.

Componente	Dettaglio
Classificazione intent	Keyword-based: 7 categorie (positioning, utility, economy, aim, player_query, round_query, match_query) + general fallback
Contesto sliding window	<code>MAX_CONTEXT_TURNS = 6</code> (12 messaggi: 6 user + 6 assistant)
Retrieval augmentation	RAG top-3 + Experience Bank top-3, iniettati nel messaggio utente
Player entity detection	Integrazione con <code>PlayerLookupService</code> per rilevare menzioni di giocatori professionisti nel messaggio utente e iniettare blocchi "VERIFIED PLAYER DATA" nel contesto LLM
Fallback offline	Template-based con RAG-only quando Ollama non è disponibile
Singleton	<code>get_dialogue_engine()</code> — factory module-level
Anti-hallucination (WR-78/79)	System prompt con regole: "use ONLY verified data", provenance markers ("pro data" vs "user data"), mai fabbricare narrative tattiche

Pipeline di risposta:



Gestione sessione: `start_session(player_name, demo_name)` → carica contesto dal DB (ultimi 5 insight, area focus primaria) → genera messaggio di apertura via LLM → `respond(user_message)` per turni successivi → `clear_session()` per reset.

Sicurezza dello storico (F5-06): Il messaggio utente viene aggiunto alla cronologia solo *dopo* aver ottenuto una risposta valida dal LLM, evitando stati inconsistenti in caso di eccezione.

La fase agantica: il coach interroga il database da solo

Fino all'agosto 2026 il coach riceveva un contesto pre-confezionato e, se la domanda richiedeva un dato che nessuno gli aveva messo davanti, rispondeva che non aveva accesso ai dati delle partite. Il contesto era deciso *prima* di sapere cosa servisse.

Oggi, quando il modello LLM supporta il tool-calling nativo, `respond()` prova prima un **ciclo agentico** in cui è il coach a chiedere ciò che gli manca. Quattro strumenti, deliberatamente pochi e a grana grossa:

Strumento	Cosa restituisce
<code>list_matches(team?, map_name?)</code>	L'elenco delle partite disponibili, filtrabile per squadra o per mappa
<code>get_match_overview(demo_name)</code>	Il quadro d'insieme di una partita: punteggio, tabellino, metriche per giocatore
<code>get_round_details(demo_name, rounds[])</code>	Il dettaglio di round specifici
<code>lookup_player(name)</code>	La scheda di un giocatore professionista

Il ciclo gira al massimo `_MAX_TOOL_ROUNDS = 4` volte più un giro finale in cui la risposta è obbligatoria, ed esegue al massimo quattro chiamate per giro. Fra un giro e l'altro controlla se l'utente ha annullato. Se il modello non supporta gli strumenti, il metodo restituisce `None` e si torna al percorso classico con il contesto pre-caricato: la fase agantica è un miglioramento, non un requisito.

Due decisioni di progetto meritano di essere raccontate.

L'inventario nel prompt di sistema. Il prompt include un blocco `MATCH DATABASE INVENTORY` con l'elenco delle partite realmente presenti (prima quelle con i dati per round, fino a `_INVENTORY_LIMIT = 60`). La ragione è semplice quanto vincolante: **un LLM non può scegliere una partita che non gli è mai stata mostrata**. Senza inventario il modello inventava nomi di demo plausibili e poi falliva la chiamata.

La fiducia zero sugli argomenti. Tutto ciò che l'LLM produce come argomento è trattato come input non fidato, perché lo è — è testo generato, non un valore validato da un form:

- ogni `demo_name` passa da `_canonical_demo()`, che lo risolve contro una **whitelist costruita interrogando il database**; se non corrisponde, lo strumento restituisce `ERROR: unknown demo_name. Call list_matches for exact names.` — un messaggio da cui il modello può riprendersi, non un'eccezione;
- i numeri di round devono essere interi, sono limitati all'intervallo 1–50, deduplicati e tagliati a `_TOOL_MAX_ROUNDS_PER_CALL = 5`;
- i frammenti di nome squadra sono ripuliti di tutto ciò che non sia `[a-z0-9_-]` e troncati a `_TOOL_ARG_MAX_LEN = 40`; un `map_name` che non appartiene alla lista nota viene semplicemente ignorato;
- il risultato di ogni strumento viene ripulito dai caratteri di controllo e troncato a `_TOOL_RESULT_MAX_LEN = 8000` caratteri **prima di rientrare nel prompt**, perché ciò che torna dal database diventa a sua volta input del modello.

Gli errori restituiti sono stringhe brevi e neutre: mai il testo di un'eccezione, che rivelerebbe struttura interna a un componente che non ha motivo di conoscerla.

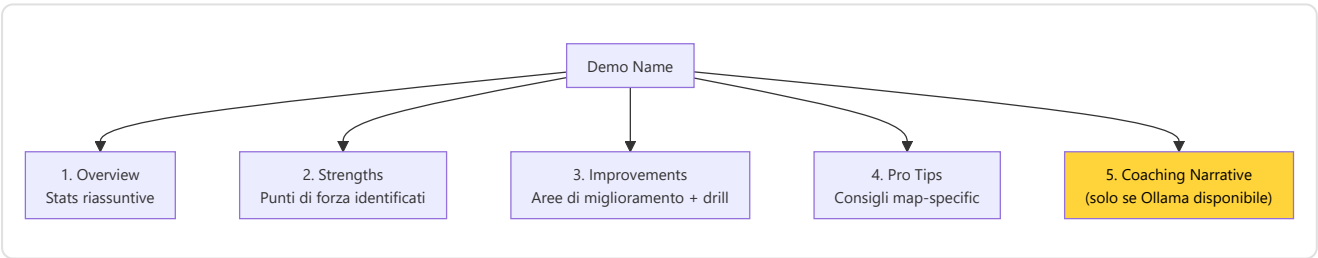
Durante i giri di strumenti la risposta **non è in streaming** — l'interfaccia riceve avvisi di avanzamento ("*sto consultando il database delle partite...*") e poi un'unica risposta completa.

LessonGenerator (`lesson_generator.py`)

Generatore di lezioni educative strutturate a partire dall'analisi delle demo:

Soglia	Costante	Valore	Uso
ADR forte	<code>_ADR_STRONG_THRESHOLD</code>	75.0	Identifica punti di forza
ADR debole	<code>_ADR_WEAK_THRESHOLD</code>	60.0	Identifica aree di miglioramento
HS% forte	<code>_HS_STRONG_THRESHOLD</code>	0.40	Precisione sopra la media
HS% debole	<code>_HS_WEAK_THRESHOLD</code>	0.35	Precisione sotto la media
Rating sopra media	<code>_RATING_ABOVE_AVG</code>	1.0	Prestazione positiva
KAST forte	<code>_KAST_STRONG_THRESHOLD</code>	0.70	Contributo costante
Rapporto morti	<code>_DEATH_RATIO_WARNING</code>	1.5×	deaths > kills × 1.5 = warning

Struttura lezione generata:



Pro Tips map-specific: Database interno con consigli per mirage, inferno, dust2, ancient, nuke. Fallback a consigli generali per mappe non coperte.

LLMService (`llm_service.py`)

Servizio di integrazione Ollama per inferenza LLM locale:

Parametro	Valore	Descrizione
<code>OLLAMA_URL</code>	<code>http://localhost:11434</code>	Endpoint Ollama (env: <code>OLLAMA_URL</code>)
<code>DEFAULT_MODEL</code>	<code>gemma4:e2b</code>	Modello di default; risoluzione in ordine: env <code>OLLAMA_MODEL</code> → setting <code>LLM_COACH_MODEL</code> → default hard-coded (<code>llama3.1:8b</code> compare solo come baseline di confronto nel CS2 Coach Bench)
<code>_AVAILABILITY_TTL</code>	60s	Cache di disponibilità (health check con timeout 3s)
<code>temperature</code>	0.7	Creatività delle risposte
<code>top_p</code>	0.9	Nucleus sampling
<code>num_predict</code>	-1	Nessun limite di lunghezza risposta
<code>num_ctx</code>	32768	Finestra di contesto
Timeout generate/chat	600s	POST verso Ollama; stream: connect 10s, stall 30s

API supportate:

Metodo	Endpoint Ollama	Uso
<code>generate(prompt)</code>	<code>/api/generate</code>	Generazione single-shot
<code>chat(messages)</code>	<code>/api/chat</code>	Conversazione multi-turno
<code>generate_lesson(insights)</code>	<code>/api/generate</code>	Lezioni da insight RAP
<code>explain_round_decision(round_data)</code>	<code>/api/generate</code>	Spiegazione round singolo
<code>generate_pro_tip(context)</code>	<code>/api/generate</code>	Tip contestuale

Auto-discovery modello: Se il modello configurato non è disponibile, utilizza automaticamente il primo modello installato. Singleton via `get_llm_service()` .

VisualizationService (`visualization_service.py`)

Genera grafici radar Matplotlib per confronto User vs Pro:

- `generate_performance_radar(user_stats, pro_stats, output_path)` → file PNG con radar overlay
- `plot_comparison_v2(p1_name, p2_name, p1_stats, p2_stats)` → `io.BytesIO` buffer PNG
- Feature confrontate: avg_kills, avg_adr, avg_hs, avg_kast, accuracy
- Rendering wrappato in try/except (F5-19) per gestire stats vuote o backend matplotlib assente

ProfileService (`profile_service.py`)

Orchestratore di sincronizzazione profili esterni:

- `fetch_steam_stats(steam_id)` → nickname, avatar, playtime CS2 (ore)
- `fetch_faceit_stats(nickname)` → faceit_elo, faceit_level, faceit_id
- `sync_all_external_data(steam_id, faceit_name)` → upsert su `PlayerProfile` in DB

- Chiavi API caricate da environment/keyring (F5-22), mai hardcoded

AnalysisService (analysis_service.py)

Servizio di coordinamento analisi con drift detection:

- analyze_latest_performance(player_name) → ultime stats dal DB
- get_pro_comparison(player_name, pro_name) → confronto side-by-side
- check_for_drift(player_name) → detect_feature_drift() sulle ultime 100 partite

TelemetryClient (telemetry_client.py)

Client async per invio metriche a server centrale:

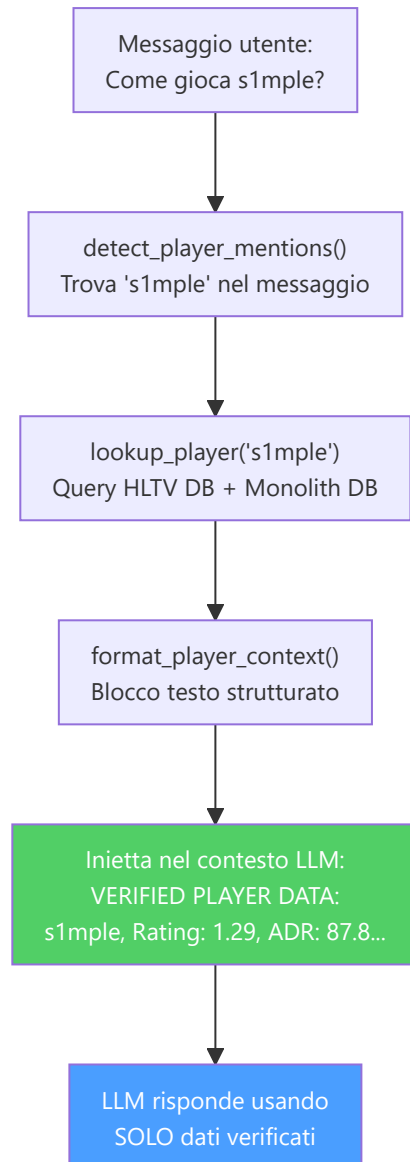
- Protocollo: httpx.Client → POST /api/ingest/telemetry
- URL configurabile: CS2_TELEMETRY_URL (default: http://127.0.0.1:8000)
- Fallback graceful se httpx non installato (feature opzionale)
- **Anti-fabrication compliant:** nessun dato sintetico nel self-test

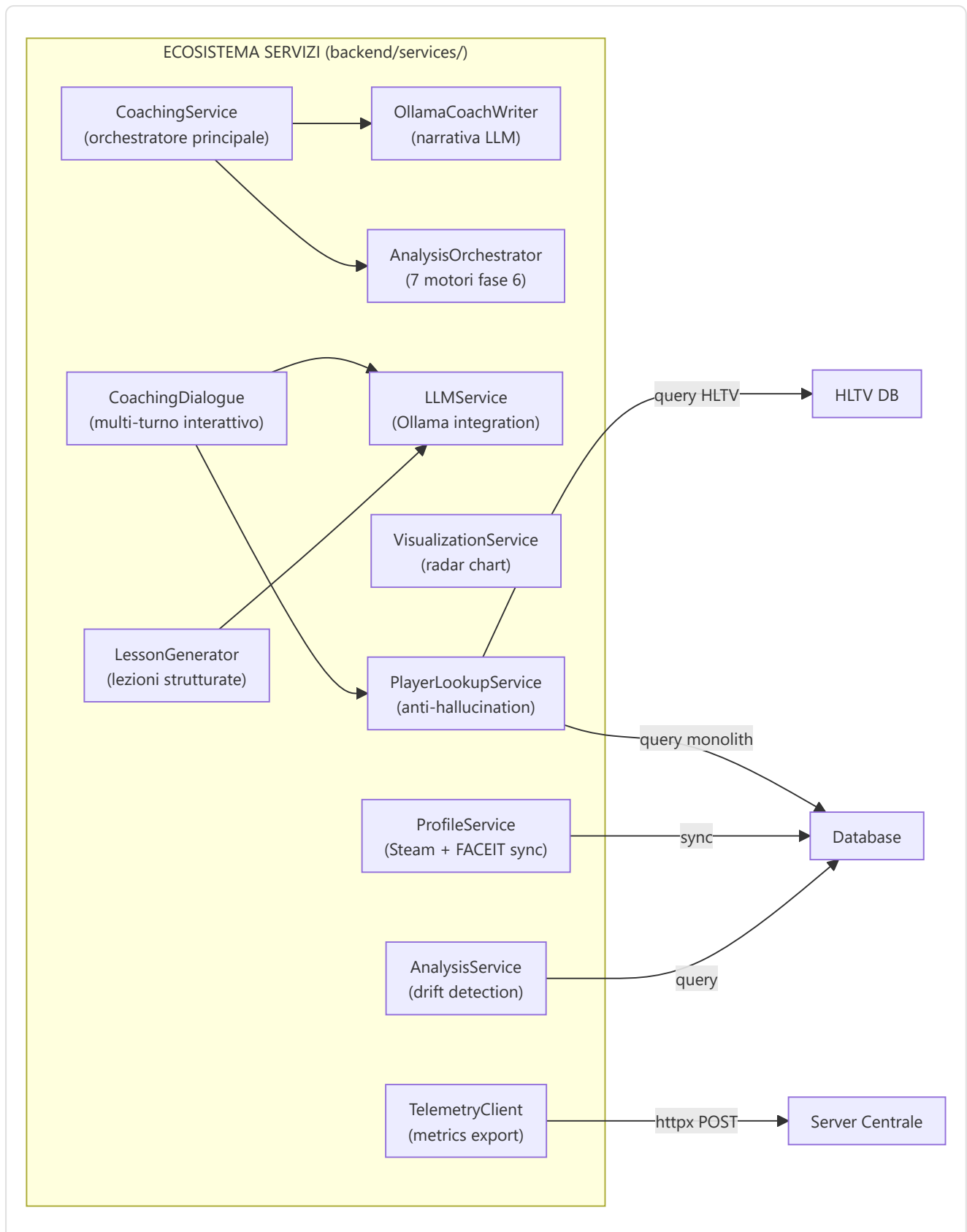
PlayerLookupService (player_lookup.py)

Servizio di ricerca giocatori professionisti che previene l'allucinazione LLM iniettando dati verificati nel contesto del dialogo di coaching:

Componente	Dettaglio
Matching a 3 livelli	Esatto (case-insensitive) → Fuzzy (SequenceMatcher ≥ 0.75) → Pattern (regex nickname)
Cache nickname	TTL 60s, carica tutti i ProPlayer.nickname dal DB HLTV all'avvio
Stop-word filter	89 parole comuni inglesi filtrate per evitare falsi positivi
Output	ProPlayerProfile dataclass: nickname, hltv_id, real_name, country, team, statistiche HLTV (rating, KPR, ADR, KAST), performance da demo locali
Integrazione	CoachingDialogueEngine → detect_player_mentions() → lookup_player() → format_player_context() → blocco "VERIFIED PLAYER DATA" iniettato nel contesto LLM
Singleton	get_player_lookup_service()

Pipeline anti-allucinazione:





-Server FastAPI (`server.py`)

Server REST standalone che espone endpoint per telemetria, insight, health e controllo remoto dei servizi.

Non è integrato nel flusso principale dell'applicazione — è un servizio di utilità separato, avviabile indipendentemente.

Rate Limiting: Classe `RateLimiter` con finestra scorrevole — massimo 10 richieste per minuto per indirizzo IP. La struttura interna `_requests` mappa ogni IP a una lista di timestamp `float`, purgando i timestamp scaduti ad ogni controllo.

Lifecycle: Il server utilizza il lifespan handler asincrono di FastAPI per inizializzare il database all'avvio tramite `init_database()`.

Modelli Pydantic:

Modello	Tipo	Campi principali
<code>InsightRead</code>	Response	<code>id</code> , <code>player_name</code> , <code>title</code> , <code>message</code> , <code>focus_area</code> , <code>severity</code> , <code>created_at</code>
<code>MatchTelemetry</code>	Request	<code>player_id</code> , <code>match_id</code> , <code>stats</code> (dict), <code>timestamp</code> (float)

Tabella Endpoint:

Metodo	Path	Rate Limited	Descrizione
POST	<code>/api/ingest/telemetry</code>	Sì	Ingestione dati telemetria partita. Scrittura background su disco via <code>_write_telemetry_to_disk()</code>
GET	<code>/api/insights</code>	No	Restituisce insight di coaching per un giocatore (query param: <code>player_name</code>)
GET	<code>/api/status</code>	No	Stato applicazione: versione, uptime, stato database
GET	<code>/api/console/health</code>	No	Health check per monitoraggio servizi
POST	<code>/api/training/start</code>	No	Avvia training ML via Console backend
POST	<code>/api/training/stop</code>	No	Ferma training ML
POST	<code>/api/training/pause</code>	No	Mette in pausa training ML
POST	<code>/api/training/resume</code>	No	Riprende training ML da pausa
POST	<code>/api/services/hunter/start</code>	No	Avvia servizio HLTV Hunter
POST	<code>/api/services/hunter/stop</code>	No	Ferma servizio HLTV Hunter
GET	<code>/api/services/status</code>	No	Stato di tutti i servizi gestiti

Sicurezza telemetria: `_write_telemetry_to_disk(data)` sanitizza i nomi dei file derivati da `player_id` e `match_id` per prevenire attacchi di path traversal — i caratteri pericolosi vengono rimossi prima della costruzione del path di destinazione.

-Onboarding (`new_user_flow.py`)

Gestisce l'esperienza dei nuovi utenti attraverso un processo di onboarding a **3 stadi progressivi** che guida il giocatore dall'installazione al coaching completo.

Stadi di Onboarding:

Costante	Valore	Significato	Condizione
<code>AWAITING_FIRST_DEMO</code>	<code>"awaiting_first_demo"</code>	Nessuna demo caricata	0 demo
<code>BUILDING_BASELINE</code>	<code>"building_baseline"</code>	Demo caricate ma insufficienti per baseline stabile	1-2 demo
<code>COACH_READY</code>	<code>"coach_ready"</code>	Coaching completo attivo	3+ demo

Classe `UserOnboardingManager` :

- `MIN_INITIAL_DEMOS = 1` — minimo per avviare il coaching base
- `RECOMMENDED_DEMOS = 3` — minimo per baseline statistica stabile
- `_CACHE_TTL_SECONDS = 60` — cache TTL per conteggio demo (evita query ripetute durante refresh UI rapidi)

Metodi:

- `get_status(user_id) → OnboardingStatus` — restituisce lo stato completo di onboarding con conteggio demo, stadio, messaggi e flag `coach_ready` / `baseline_stable`
- `_count_user_demos(user_id)` — conta demo processate (non-pro) con cache TTL-based. Filtro: `player_name == user_id AND is_pro == False`
- `invalidate_cache(user_id=None)` — invalida la cache dopo caricamento nuova demo (per utente specifico o intera cache)
- `_determine_stage(count)` — mappa conteggio demo → stadio
- `_get_stage_message(stage, count)` — messaggio human-readable per ogni stadio

Dataclass `OnboardingStatus` : `stage` , `demos_uploaded` , `demos_required` (1), `demos_recommended` (3), `coach_ready` , `baseline_stable` , `message` .

Singleton: `get_onboarding_manager()` — factory module-level (non thread-safe con double-checked locking — accettabile per questo caso d'uso a bassa contesa).

5B. Sottosistema 3B — Motori di Coaching

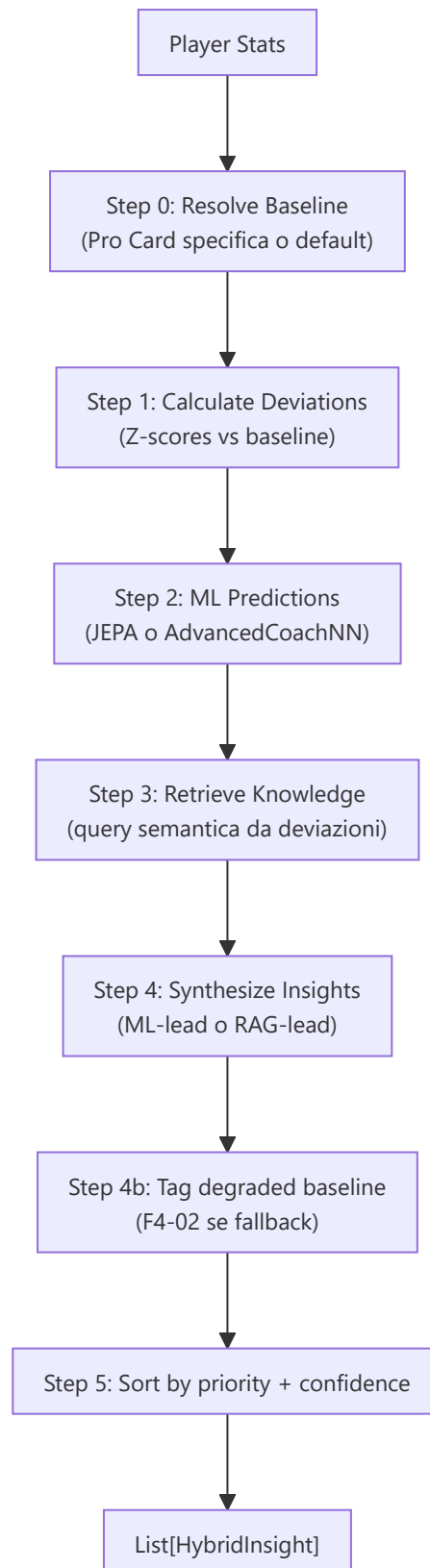
Cartella nella repo: `backend/coaching/` **File:** 8 moduli

Questo sottosistema contiene i **motori decisionali di coaching** che trasformano deviazioni statistiche grezze in consigli prioritizzati e contestualizzati. A differenza dei Servizi (Sezione 5) che orchestrano e presentano, i Motori di Coaching contengono la **logica di ragionamento** dell'allenatore.

-HybridCoachingEngine (`hybrid_engine.py`)

Il **cuore decisionale** del coaching: sintetizza predizioni ML con conoscenza RAG per generare insight unificati e deduplicati.

Pipeline a 5 stadi:



Classi e dataclass:

Tipo	Nome	Descrizione
Enum	InsightPriority	CRITICAL (Z >2.5, conf>0.8), HIGH (Z >2.0, conf>0.6), MEDIUM (Z >1.0, conf>0.4), LOW
dataclass	HybridInsight	title, message, priority, confidence, feature, ml_z_score, knowledge_refs, pro_examples, tick_range, demo_name

Formula di confidenza:

```
base_confidence = |z_score|/3.0 × 0.6 + knowledge_effectiveness × 0.4
confidence = base_confidence × MetaDriftEngine.get_meta_confidence_adjustment()
```

Dove `knowledge_effectiveness = min(1.0, mean(usage_count) / 100)`.

Strategia di sintesi:

Condizione	Strategia
Z > 2 (alta confidenza ML)	Lead con ML, supporto con RAG
Z < 1 (bassa confidenza ML)	Lead con RAG
1 ≤ Z ≤ 2	Approccio bilanciato
Nessuna deviazione significativa	Insight knowledge-only (priorità LOW)

TASK 2.7.1 — Reference Clip: Ogni `HybridInsight` può includere `tick_range: (start, end)` e `demo_name` per permettere alla UI di saltare direttamente all'evidenza nel file demo.

Fallback baseline (F4-02): Se `get_pro_baseline()` fallisce, usa valori statici hardcoded e marca tutti gli insight con `baseline_quality=degraded` nel messaggio.

-CorrectionEngine (`correction_engine.py`)

Genera le **top-3 correzioni pesate** da deviazioni Z-score:

Costante	Valore	Uso
<code>CONFIDENCE_ROUNDS_CEILING</code>	300	Scaling confidenza: <code>min(1.0, rounds / 300)</code>

Pesi di importanza (DEFAULT_IMPORTANCE):

Feature	Peso
avg_kast	1.5
avg_adr	1.5
accuracy	1.4
impact_rounds	1.3
avg_hs	1.2
econ_rating	1.1
positional_aggression_score	1.0

Pipeline: `deviations` → applica confidence scaling × rounds_played → opzionale `apply_nn_refinement()` → sort per `|weighted_z| × importance` → restituisci top 3.

Override utente: I pesi sono sovrascrivibili via `get_setting("COACH_WEIGHT_OVERRIDES")`.

-ExplanationGenerator (`explainability.py`)

Traduce segnali latenti RL in **narrative comprensibili** organizzate per i 5 assi di competenza (`SkillAxes`):

Asse	Template Negativo	Template Azione
MECHANICS	"Your {feature} is {delta}% below professional standards..."	"Focus on crosshair height when clearing corners..."
POSITIONING	"Positioning at {location} was suboptimal..."	"Try holding a tighter angle at {location}..."
UTILITY	"Utility timing with {weapon} was suboptimal..."	"Wait for a clear sound cue before deploying..."
TIMING	"Engagement timing is lagging behind..."	"Coordinate with teammates to trade-frag..."
DECISION	"Decision efficiency is {delta}% lower..."	"In clutch scenarios, prioritize the round objective..."

Principio "Silence is a Valid Action":

Soglia	Costante	Valore	Comportamento
Silenzio	<code>SILENCE_THRESHOLD</code>	0.2	$ \text{delta} < 0.2 \rightarrow$ nessun feedback (silenzio)
Severità alta	<code>SEVERITY_HIGH_BOUNDARY</code>	1.5	$ \text{delta} > 1.5 \rightarrow$ "High"
Severità media	<code>SEVERITY_MEDIUM_BOUNDARY</code>	0.8	$ \text{delta} > 0.8 \rightarrow$ "Medium"

Filtro di complessità: Per `skill_level < 3` (principianti), le narrative negative sono semplificate alla sola azione suggerita, evitando sovraccarico cognitivo.

-NNRefinement (`nn_refinement.py`)

Applica aggiustamenti di peso dalla rete neurale alle correzioni pre-calcolate:

```
refined_z = weighted_z × (1 + nn_adjustments["{feature}_weight"])
```

Modulo leggero che scala le correzioni dal **CorrectionEngine** usando pesi appresi dal modello ML, permettendo al sistema neurale di influenzare la prioritizzazione dei consigli.

-ProBridge (`pro_bridge.py`)

Layer di traduzione che assimila Player Card HLTV nel modello cognitivo del coach:

Classe	Responsabilità
<code>PlayerCardAssimilator</code>	Converte <code>ProPlayerStatCard</code> → baseline coach-compatible
<code>get_pro_baseline_for_coach()</code>	Factory function diretta

Costante legacy: `ESTIMATED_ROUNDS_PER_MATCH = 24.0` — presente nel codice ma **non più utilizzata** per la conversione KPR/DPR dopo la correzione P3-02.

Mappatura metriche:

Metrica HLTV	Metrica Coach	Trasformazione
<code>card.kpr</code>	<code>avg_kills</code>	diretto (P3-02: NON moltiplicato per 24)
<code>card.dpr</code>	<code>avg_deaths</code>	diretto (P3-02: NON moltiplicato per 24)
<code>card.adr</code>	<code>avg_adr</code>	diretto
<code>card.kast</code>	<code>avg_kast</code>	V-2: normalizzazione difensiva (<code>/100</code> se <code>kast > 1.0</code> , altrimenti diretto)
<code>card.impact</code>	<code>impact_rounds</code>	diretto
<code>card.rating_2_0</code>	<code>rating</code>	diretto
<code>detailed_stats.headshot_pct</code>	<code>avg_hs</code>	V-2: normalizzazione difensiva (<code>/100</code> se <code>> 1.0</code> , default 0.45)
<code>detailed_stats.total_opening_kills</code>	<code>entry_rate</code>	<code>/100</code> (euristica)
<code>detailed_stats.utility_damage_per_round</code>	<code>utility_damage</code>	diretto (default 45.0)

Correzione P3-02 — Scala KPR/DPR: Il codice precedente moltiplicava `kpr × 24` e `dpr × 24` , producendo uccisioni/morti totali per partita (valori 15-20) anziché valori per-round (0.6-0.8). Questo rendeva tutti gli z-score di confronto invalidi poiché le statistiche utente estratte da `base_features.py` e `pro_baseline.py` sono già in scala per-round. Dopo la rimediazione, `get_coach_baseline()` usa direttamente i tassi per-round.

Correzione V-2 — Normalizzazione difensiva legacy: I record HLTV più vecchi nel database potrebbero contenere valori percentuali (es. `kast=72.0` anziché `kast=0.72`). La normalizzazione condizionale (`/100 se > 1.0`) gestisce entrambi i formati in modo trasparente.

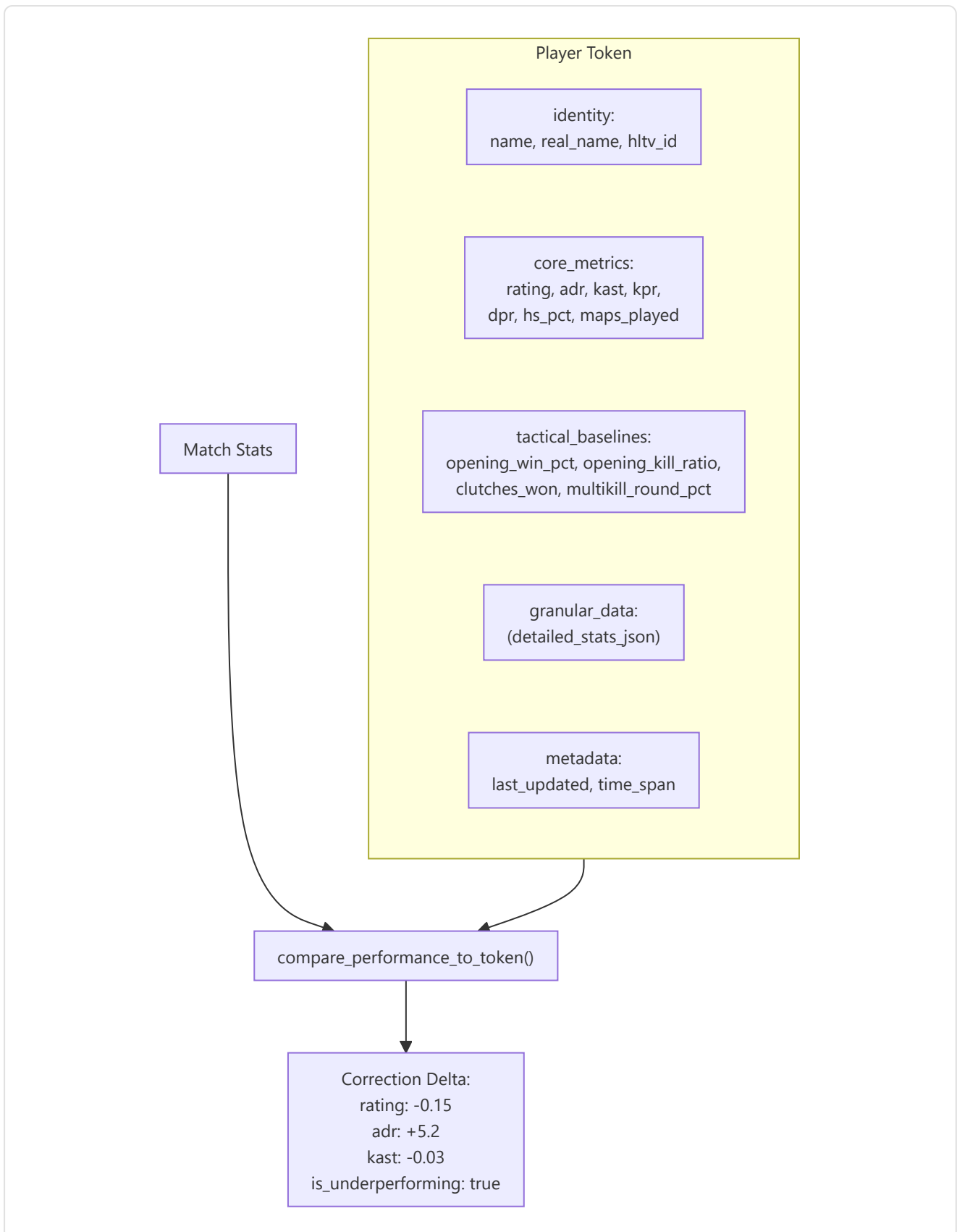
Classificazione archetipo: `get_player_archetype()` → Star Fragger (impact>1.3), Support Anchor (kast>0.75), Sniper Specialist (AWP kills>40%), All-Rounder (default).

-PlayerTokenResolver (`token_resolver.py`)

Risolve **token statici** (Card) per il confronto dinamico:

- `get_player_token(player_name)` → token dict con identity, core_metrics, tactical_baselines, granular_data, metadata
- `compare_performance_to_token(match_stats, token)` → **Correction Delta** con deltas per rating, adr, kast, accuracy_vs_hs e flag `is_underperforming` (rating < 85% del pro)

Struttura Token:

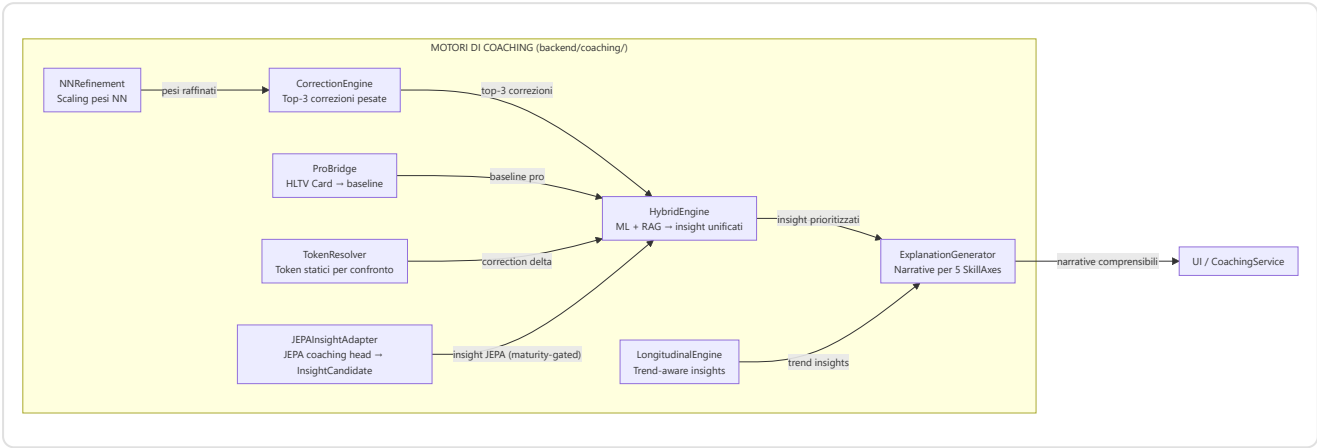


-LongitudinalEngine (`longitudinal_engine.py`)

Genera insight trend-aware comparando traiettorie di performance nel tempo con segnali di stabilità NN:

- **Input:** `List[FeatureTrend]` (dal modulo progress) + `nn_signals` (dal training pipeline)

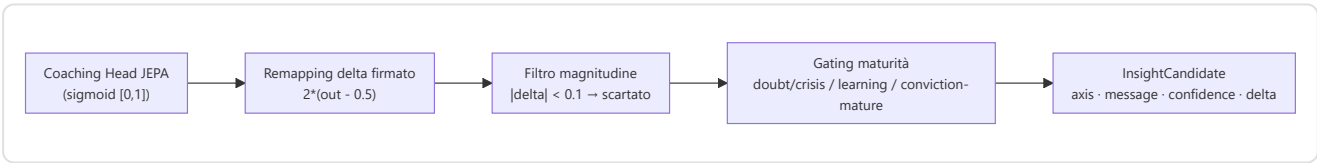
- **Filtro confidenza:** Solo trend con `confidence ≥ 0.6`
- **Output:** Max 3 insight, categorizzati come:
 - **Regression:** slope < 0 → severità "Medium" (o "High" se `nn_signals.stability_warning` attivo)
 - **Improvement:** slope > 0 → severità "Positive", focus "Reinforcement"



-JEPAINsightAdapter (`jepa_insight_adapter.py`)

Converte i vettori grezzi in uscita dalla **Coaching Head JEPA** in oggetti `InsightCandidate` pronti per la pipeline di coaching (F1.1 / W5.1).

Flusso:



Contratto dimensionale: Opera sui primi 10 elementi del contratto 25-dim (target features: health, armor, has_helmet, has_defuser, equipment_value, is_crouching, is_scoped, is_blinded, enemies_visible, pos_x) mappati su cinque SkillAxes (`survival` , `economy` , `movement` , `utility` , `positioning`).

Scala maturità:

Stato	Moltiplicatore confidenza	Max candidati	Modalità
<code>doubt</code> / <code>crisis</code>	0.5×	1	Hedged: "Observation: ..."
<code>learning</code>	0.8×	2	Diretto
<code>conviction</code> / <code>mature</code>	1.0×	3	Diretto

Flag di attivazione: `USE_JEPA_MODEL` (default `False`). Quando disattivata, `generate_jepa_insights()` restituisce lista vuota — nessuna modifica al comportamento esistente (F1.2 byte-identical). Rispetta il vincolo **NO-WALLHACK** (F1.3): consuma esclusivamente la finestra tick del giocatore osservato (contratto POV 25-dim, P-X-01).

6. Sottosistema 4 — Conoscenza e Recupero

cartella nella repo: `backend/knowledge/` **File:** `rag_knowledge.py` , `experience_bank.py`

Questo sottosistema gestisce due tipi di dati complementari: le **conoscenze tattiche** (strategie, posizionamenti, uso delle utility documentati e indicizzati per ricerca semantica) e le **esperienze di coaching** (record di ogni sessione di coaching con esito, efficacia e contesto della partita, utilizzati per apprendimento basato su casi).

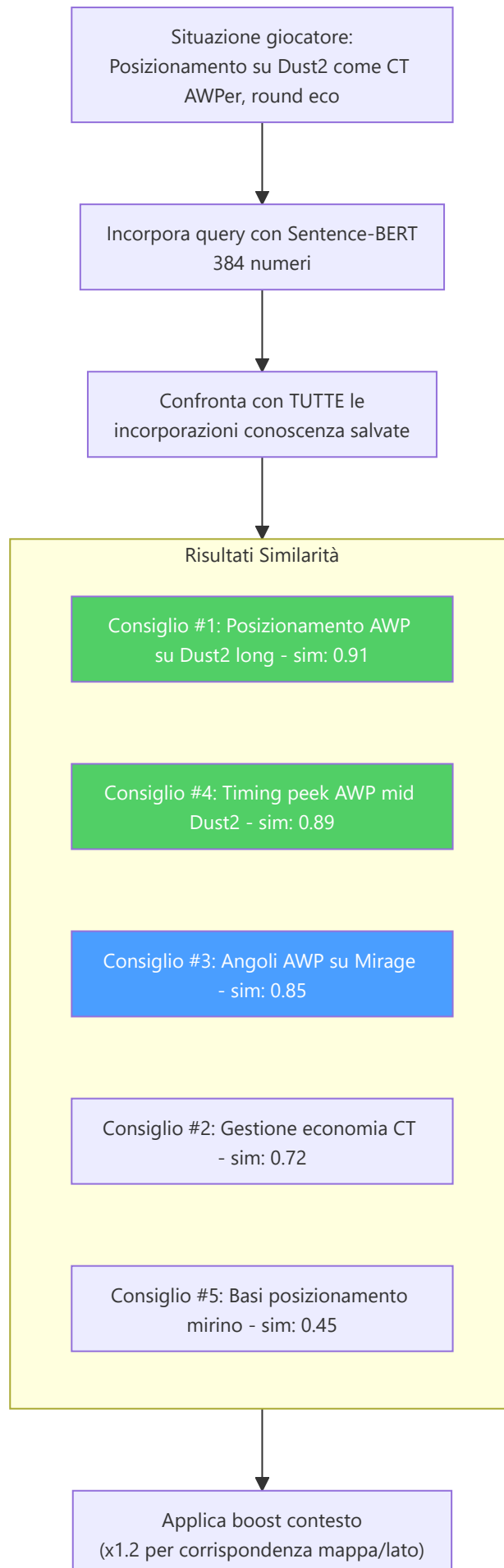
-Knowledge Base RAG (`rag_knowledge.py`)

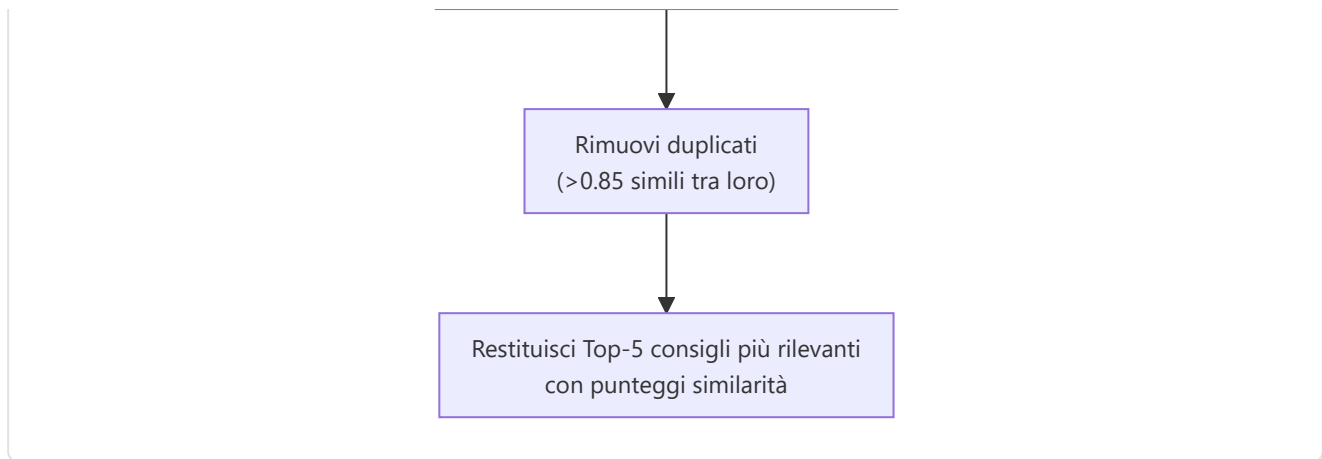
Implementa una pipeline di **generazione aumentata dal recupero** utilizzando la ricerca per similarità vettoriale densa:

Componente	Dettaglio
Modello di incorporamento	<code>sentence-transformers/all-MiniLM-L6-v2</code> (vettori a 384 dimensioni)
Fallback	Incorporamenti hash bag-of-words a 100 dimensioni se Sentence-BERT non è disponibile
Archiviazione	Tabella SQLite <code>TacticalKnowledge</code> (embedding memorizzato come array float codificato in JSON)
Recupero	Similarità del coseno via numpy: <code>np.dot(a,b)/(a · b +1e-8)</code> ; fast-path FAISS (<code>VectorIndexManager</code> , over-fetch ×10) con fallback brute-force
Top-k	Configurabile, predefinito k=3 nel retriever
Versioning	<code>CURRENT_VERSION = "v3"</code> ; incorporamenti obsoleti ricalcolati automaticamente
Sorgente primaria	Coach Book in <code>backend/knowledge/book/</code> : 8 file JSON di contenuto (7 mappe Active Duty + general) + <code>index.json</code> , 508 voci totali
Categorie	13 (dal Coach Book): <code>aim_and_duels</code> , <code>positioning</code> , <code>economy</code> , <code>retakes_post_plant</code> , <code>clutch_play</code> , <code>mid_round</code> , <code>utility</code> , <code>anti_strat</code> , <code>role_play_entry</code> , <code>role_play_awp</code> , <code>role_play_support</code> , <code>role_play_lurk</code> , <code>role_play_igl</code>

Correzione M-07 — Rifiuto vettore norma-zero: `VectorIndex.search()` valida la norma del vettore query prima della ricerca. Se la norma è zero (tipicamente dovuto a un embedding fallback vuoto o a un input corrotto), il metodo ritorna `None` con un warning nel log anziché propagare un errore di divisione per zero nella similarità del coseno. Questo protegge il pipeline RAG da query degenerate senza interrompere il flusso di coaching.



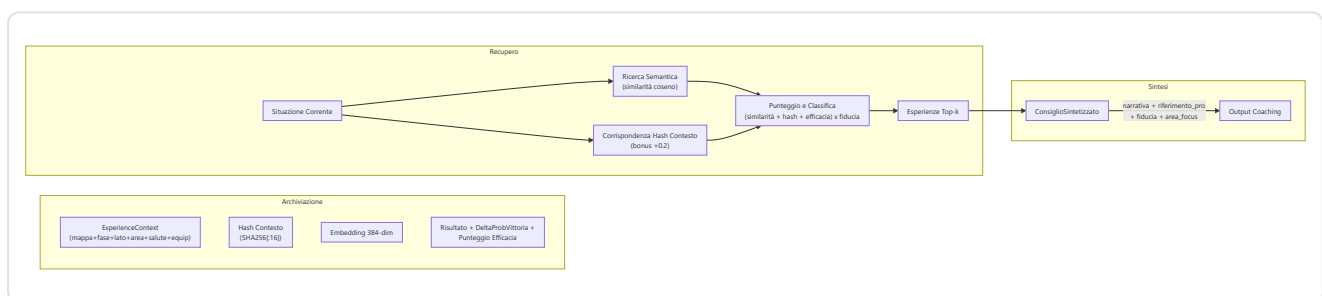




Costruzione query: query dinamiche in linguaggio naturale da statistiche giocatore, mappa, lato, ruolo. Gli elementi che corrispondono al contesto ottengono un moltiplicatore di rilevanza 1,2x. La deduplicazione filtra gli elementi con una similarità >0,85 rispetto ai risultati già selezionati.

-Banca Esperienza (`experience_bank.py`) — Framework COPER (KT-01 Enhanced)

Implementa il framework **COPER** — "**Context Optimized with Prompt, Experience, and Replay**" con semantica CRUD, replay prioritizzato e integrazione TrueSkill:

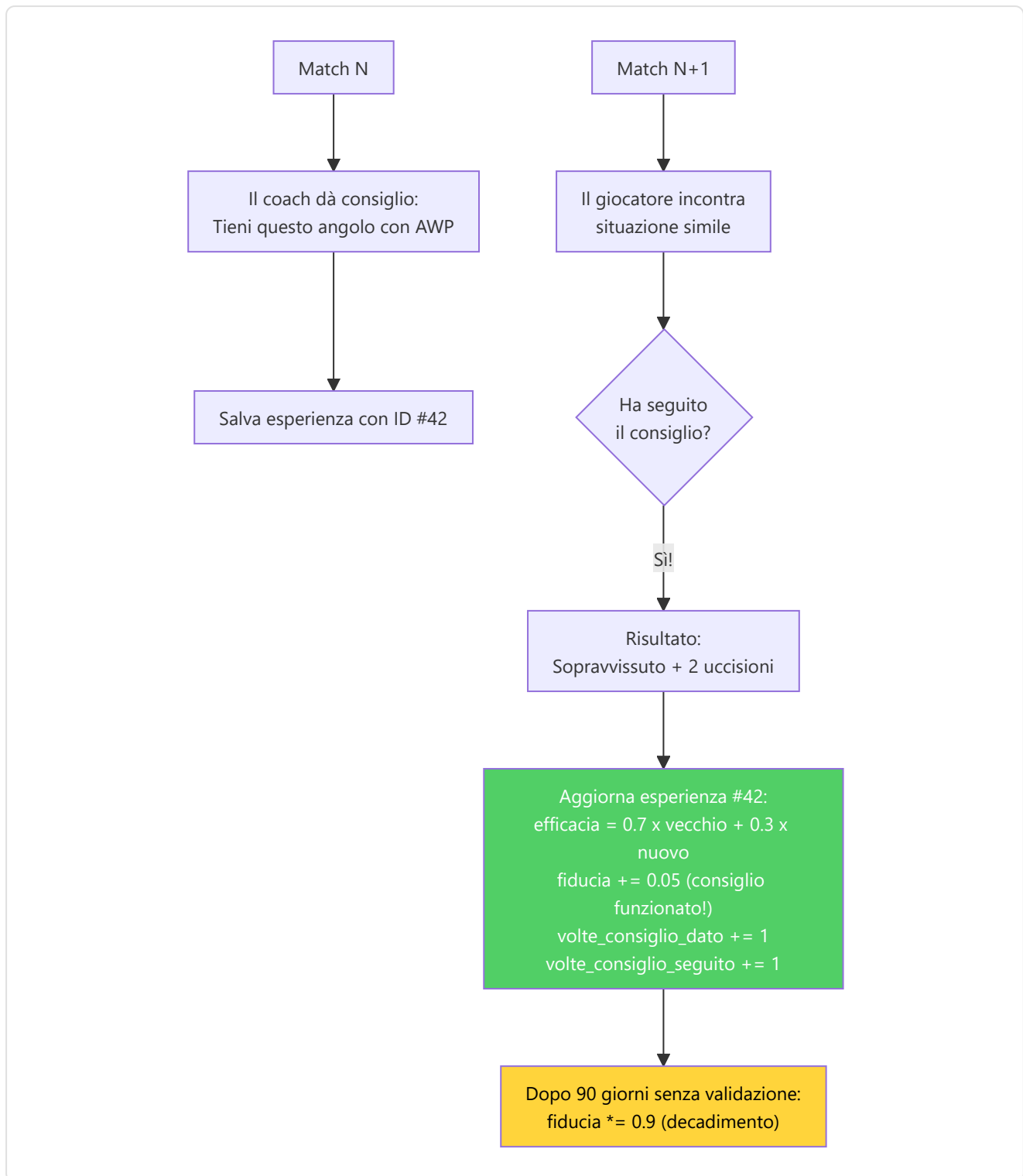


Strategia di doppio recupero:

1. **Esperienze utente:** Situazioni passate tratte dalla cronologia di gioco dell'utente.
2. **Esperienze professionali:** Come i professionisti hanno gestito situazioni analoghe.
3. **Analisi dei pattern:** Identifica debolezze ricorrenti, tendenze di miglioramento, correlazioni contestuali.

Ciclo di feedback (basato su EMA):

- Ogni esperienza tiene traccia di `outcome_validated` , `effectiveness_score` , `times_advice_given` , `times_advice_followed`
- Le corrispondenze di follow-up aggiornano l'efficacia: $\text{new_score} = 0,7 \times \text{old_score} + 0,3 \times \text{outcome_value}$
- Esperienze obsolete (>90 giorni senza convalida): la fiducia diminuisce del 10%
- Il monitoraggio dell'utilizzo incrementa `usage_count` a ogni recupero

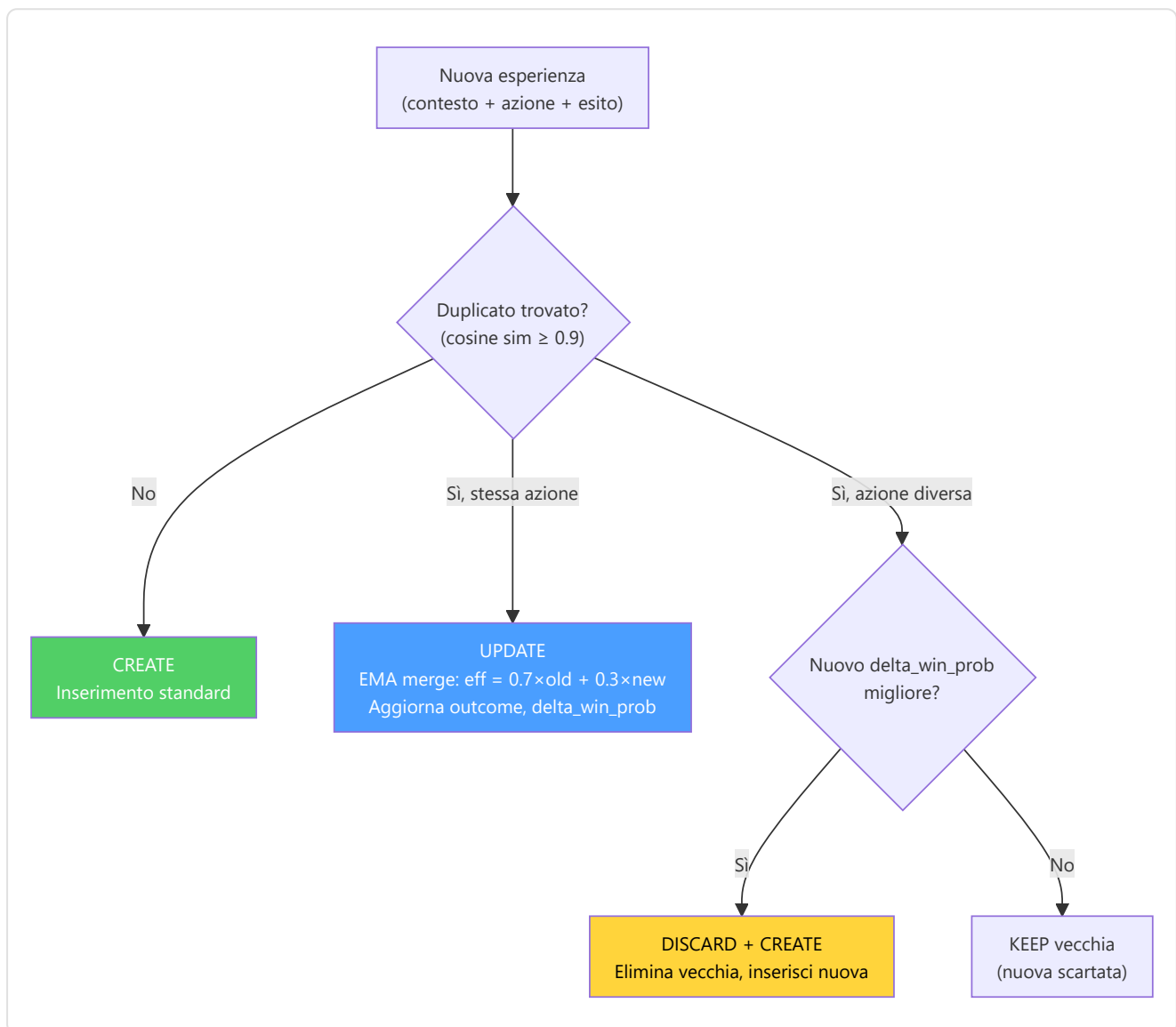


Estrazione dell'esperienza dalle demo: Raggruppa gli eventi per tick, identifica le uccisioni/morti dei giocatori, crea il contesto a partire da un'istantanea del tick, deduce l'azione (scoped_hold, crouch_peek, pushed, held_angle), determina l'esito.

Miglioramenti KT-01 — Semantica CRUD e Replay Prioritizzato:

Costante	Valore	Scopo
<code>DUPLICATE_SIMILARITY_THRESHOLD</code>	0.9	Similarità coseno per rilevamento duplicati
<code>CRUD_EMA_FACTOR</code>	0.3	Peso EMA per merge effectiveness su UPDATE
<code>REPLAY_ALPHA</code>	0.6	Esponente priorità replay (più basso = più uniforme)
<code>REPLAY_GATE</code>	0.4	Confidenza minima per essere eleggibile al replay
<code>_MIN_EFFECTIVENESS_TRIALS</code>	5	Trial minimi prima che effectiveness influenzi il retrieval

Decisione CRUD al momento dell'inserimento:



Replay Prioritizzato (KT-01): `sample_for_replay()` campiona le esperienze con priorità **inversa alla frequenza di recupero**: $p_i = (1 / \max(\text{times_retrieved}, 1))^{\text{REPLAY_ALPHA}}$ normalizzata (RNG seedato con `GLOBAL_SEED=42`). Solo esperienze con `confidence_score(κ=1.0) ≥ REPLAY_GATE` (0.4) sono eleggibili. Questo favorisce l'exploration delle esperienze meno rivisitate mantenendo un gate minimo di qualità.

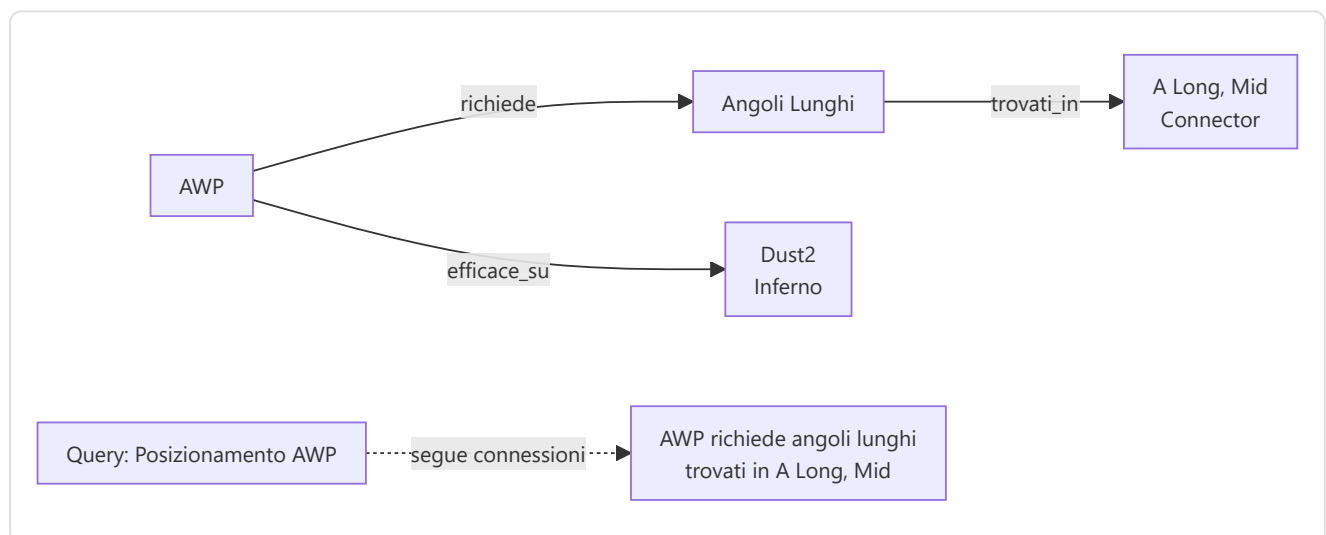
Integrazione TrueSkill (KT-01): Campi `mu_skill` e `sigma_skill` per tracking bayesiano della qualità dell'esperienza. Su UPDATE la `sigma` si restringe ($\sigma \times 0.95$) e la `mu` si aggiorna con peso 0.8/0.2; il gate di replay usa `confidence_score =  $\mu - \kappa \cdot \sigma$` ($\kappa=1.0$) — esperienze con alta incertezza sono penalizzate. Il DISCARD è un **soft-retire**: azzerla la confidence senza cancellare la riga.

Compressione Embedding: Gli embedding 384-dim sono ora codificati come `base64(float32)` anziché JSON array, ottenendo una compressione 4× sullo spazio di archiviazione nel database senza perdita di precisione.

Linking Riferimento Pro: Ogni esperienza può includere `pro_player_name`, `pro_match_id`, `source_demo` per collegare direttamente a come un professionista specifico ha gestito una situazione analoga.

-Knowledge Graph (`graph.py`)

Un **grafo entità-relazione** leggero gestito da `KnowledgeGraphManager` in un database SQLite dedicato (`knowledge_graph.db`, tabelle `entities` e `relations`). Supporta query di sottografo con esplorazione BFS per il ragionamento multi-salto, a integrazione della similarità semantica. Nota: il grafo è consultato da strumenti diagnostici e test (`tools/Goliath_Hospital.py`), non dal percorso di coaching principale.



-Inizializzazione Knowledge Base (`init_knowledge_base.py`)

Script di orchestrazione che popola il database RAG con conoscenza tattica da due fonti:

1. **Coach Book (preferito):** Caricamento da `backend/knowledge/book/index.json` — il `KnowledgePopulator` riconosce il formato indice (`"files"` presente e `"knowledge"` assente) e carica gli 8 file di contenuto; le voci passano un **allow-list** di chiavi (`title`, `description`, `category`, `situation`, `map_name`, `pro_example`) con strip delle chiavi sconosciute. Fallback legacy: `tactical_knowledge.json`
2. **Mining automatico:** `auto_populate_from_pro_demos(limit=10)` per estrarre pattern tattici dai dati professionali

Dopo il caricamento, genera un report per categoria e mappa (query COUNT aggregate) e ricostruisce gli indici FAISS: `rebuild_from_db("knowledge")` e `rebuild_from_db("experience")`.

-ProDemoMiner (`pro_demo_miner.py`)

Estrae conoscenza tattica dalle demo professionali utilizzando pattern detection:

Classe	Righe	Descrizione
<code>ProDemoMiner</code>	~180	Estrazione pattern da metadati match (map, team, success rate)
<code>AdvancedProDemoMiner(ABC)</code>	~60	Base astratta (F5-05) per parsing demo con demoparser2

Soglie di qualità:

Costante	Valore	Significato
<code>MIN_SUCCESS_RATE</code>	0.70	Pattern accettato solo se successo $\geq 70\%$
<code>MIN_SAMPLE_SIZE</code>	5	Minimo 5 occorrenze per pattern valido

Mappe supportate: mirage, dust2, inferno, nuke, overpass, vertigo, ancient, anubis.

Tipologie di conoscenza generata:

- Conoscenza specifica per mappa (posizionamento, utilità, rotazioni)
- Conoscenza specifica per team (strategie, tendenze, stili)
- Conoscenza per esecuzioni di successo (pattern con tasso $\geq 70\%$)

-Round Utils (`round_utils.py`)

Utility condivisa per classificazione fase economica del round, estratta per eliminare duplicazione tra i layer knowledge e services:

Valore Equipaggiamento	Fase	Costante
< \$1.500	<code>pistol</code>	<code>_PISTOL_MAX_EQUIP</code>
\$1.500 – \$2.999	<code>eco</code>	<code>_ECO_MAX_EQUIP</code>
\$3.000 – \$3.999	<code>force</code>	<code>_FORCE_MAX_EQUIP</code>
$\geq$ \$4.000	<code>full_buy</code>	—

7. Sottosistema 5 — Motori di analisi

cartella nella repo: `backend/analysis/` 11 file, ~2.600 righe di codice di produzione

Questo sottosistema contiene **11 motori di analisi specializzati**, ognuno progettato per indagare una diversa dimensione del gameplay. Funzionano come analisi di Fase 6, fornendo approfondimenti che vanno oltre ciò che le sole reti neurali possono offrire.



I 11 MOTORI DI ANALISI - IL TUO TEAM DI SPECIALISTI

1. Classificatore Ruoli - Che posizione giochi?

2. Probabilità Vittoria - Quali sono le probabilità ora?

3. Albero di Gioco - Qual è la mossa migliore?

4. Morte Bayesiana - Quanto è probabile che tu muoia?

5. Indice Inganno - Quanto sei imprevedibile?

6. Tracker Momentum - Sei in forma o in tilt?

7. Analizzatore Entropia - Le tue granate sono efficaci?

8. Rilevatore Punti Ciechi - Che errore ripeti?

9. Utilità ed Economia - Ottimizzatore acquisti

10. Analizzatore Ingaggio - A che distanza combatti meglio?

11. Qualità Movimento - Fai errori di posizionamento?

-Classificatore di Ruoli (`role_classifier.py`)

Assegna uno dei 6 ruoli utilizzando **soglie statistiche apprese**:

Ruolo	Segnale Primario	Segnale Secondario
AWPer	Rapporto uccisioni AWP vs soglia	—
Entry Fragger	Tasso di ingresso + bonus alla prima morte (0,3×)	—
Supporto	Tasso di assistenza + bonus danni da utilità (max 0,3×)	—
IGL	Tasso di sopravvivenza + bonus bilanciamento KD	—
Lurker	Rapporto uccisioni in solitaria vs soglia	—
Flex	Ripiegamento quando la fiducia è bassa	—

Protezione per avvio "da zero": `RoleThresholdStore` richiede ≥ 10 campioni e ≥ 3 soglie valide per uscire dall'avvio "da zero". Restituisce `(FLEX, 0.0)` se in avvio da zero. Le soglie vengono **mantenute nel database** tramite `persist_to_db()` e `load_from_db()` — completamente implementate, non come stub.

Audit del bilanciamento del team (`audit_team_balance()`): rileva più AWPer (ALTA), Entry mancante (ALTA), Supporto mancante (MEDIA), nessuna diversità (CRITICA), più Lurker (MEDIA).

La guardia di vocabolario (F-0030). Questo classificatore ha prodotto per mesi una frase che sembrava una diagnosi e non lo era: "*Ruolo: IGL rilevato, confidenza 100%*", su giocatori che non erano IGL.

Il meccanismo del difetto vale più della correzione. I cinque punteggi di affinità leggono chiavi statistiche precise — `awp_kills`, `total_kills`, `entry_frgs`, `rounds_played`, `assists`, `rounds_survived`, `solo_kills`. Il percorso vivo dell'orchestratore però passava dizionari di forma diversa, aggregati a livello di partita, che contenevano `kd_ratio` ma **nessuna** di quelle sette chiavi. Ogni affinità valeva quindi zero, tranne una: il bonus di equilibrio dell'IGL, che scatta quando il rapporto uccisioni/morti sta fra 0,9 e 1,2. La normalizzazione finale divide ciascun punteggio per la somma — e la somma era quell'unico bonus. Un solo segnale debole diventava così una certezza del 100%.

Il difetto non era nella formula: era nel fatto che nessuno verificasse che l'input avesse la forma attesa.

Oggi `classify()` controlla in apertura che il dizionario contenga almeno una chiave del vocabolario dei ruoli e, se non la contiene, restituisce `(FLEX, 0.0)` con un avviso esplicito nel log. La confidenza a zero fa scattare la guardia già presente a valle nell'orchestratore, che salta l'insight invece di pubblicarlo.

È lo stesso principio del resto del sistema: **meglio un sentinella documentato di uno zero plausibile** — e meglio ancora un silenzio dichiarato di una certezza fabbricata. Il test `test_role_vocabulary_guard.py` fissa il comportamento.

-Predittore di Probabilità di Vittoria (`win_probability.py`)

Rete neurale a 12 funzioni che stima $P(\text{round_win} \mid \text{game_state})$:

Architettura: `Lineare(12, 64) → ReLU → Dropout(0,2) → Lineare(64, 32) → ReLU → Dropout(0,1) → Lineare(32, 1) → Sigmoid`, con calibrazione opzionale delle probabilità via `PlattScaler`.

12 Caratteristiche:

#	Caratteristica	Normalizzazione
1	economia_team	/16000
2	economia_nemico	/16000
3	differenziale economia	(squadra-nemico)/16000
4	giocatori_vivi	/5
5	nemici_vivi	/5
6	differenziale conteggio giocatori	(vivi-nemico)/5
7	utilità_rimanente	/5
8	percentuale_controllo_mappa	[0, 1]
9	tempo_rimanente	/115
10	bomba_piantata	binario
11	is_ct	binario
12	rapporto valore equipaggiamento	$\min(\text{squadra}/\text{nemico}, 2)/2$

Override euristici: 3+ vantaggio → limite minimo all'85%, 3+ svantaggio → limite massimo al 15%, 0 vivi → 0%, aggiustamenti bomba piazzata (T: +0,10, CT: -0,10) — additivi sulla probabilità base, limiti economici di $\pm 8000\$$.

Nota A-12 — Guard cross-load: Questo predittore a 12 feature (`WinProbabilityNN`) è un modello *separato e incompatibile* rispetto al `WinProbabilityTrainerNN` a 9 feature descritto nella Sezione 12. I checkpoint non sono intercambiabili: al caricamento viene validata la dimensionalità del `state_dict` e, in caso di mismatch, il modello viene reinizializzato da zero con un warning nel log.

Predittore Aumentato con Elo (KT-07):

L' `EloAugmentedPredictor` avvolge il `WinProbabilityNN` base con un sistema Elo opzionale per sfruttare lo storico delle partite:

Costante	Valore	Descrizione
<code>_ELO_INITIAL</code>	1500.0	Elo iniziale per giocatori senza storico
<code>_ELO_K_FACTOR</code>	32.0	Fattore K base per la magnitudine degli aggiornamenti
<code>_ELO_RECENCY_HALF_LIFE</code>	20 partite	Il peso di una partita dimezza ogni 20 partite

Formula di aggiornamento Elo con peso di recenza:

```
new_elo = old_elo + K × w × (S - E)
```

dove:

S = punteggio effettivo (1 per vittoria, 0 per sconfitta)

E = punteggio atteso = $1 / (1 + 10^{((opp_elo - elo) / 400)})$

w = peso recenza = $2^{((match_index - N + 1) / half_life)}$

Il peso di recenza (adattato da Glickman, 1999) garantisce che le partite recenti contribuiscano di più al rating finale. Una partita di 20 match fa contribuisce la metà del K-factor rispetto all'ultima partita.

Blending NN + Elo:

```
final_prob = (1 - α) × nn_prob + α × elo_prob    (α = 0.15 default)
```

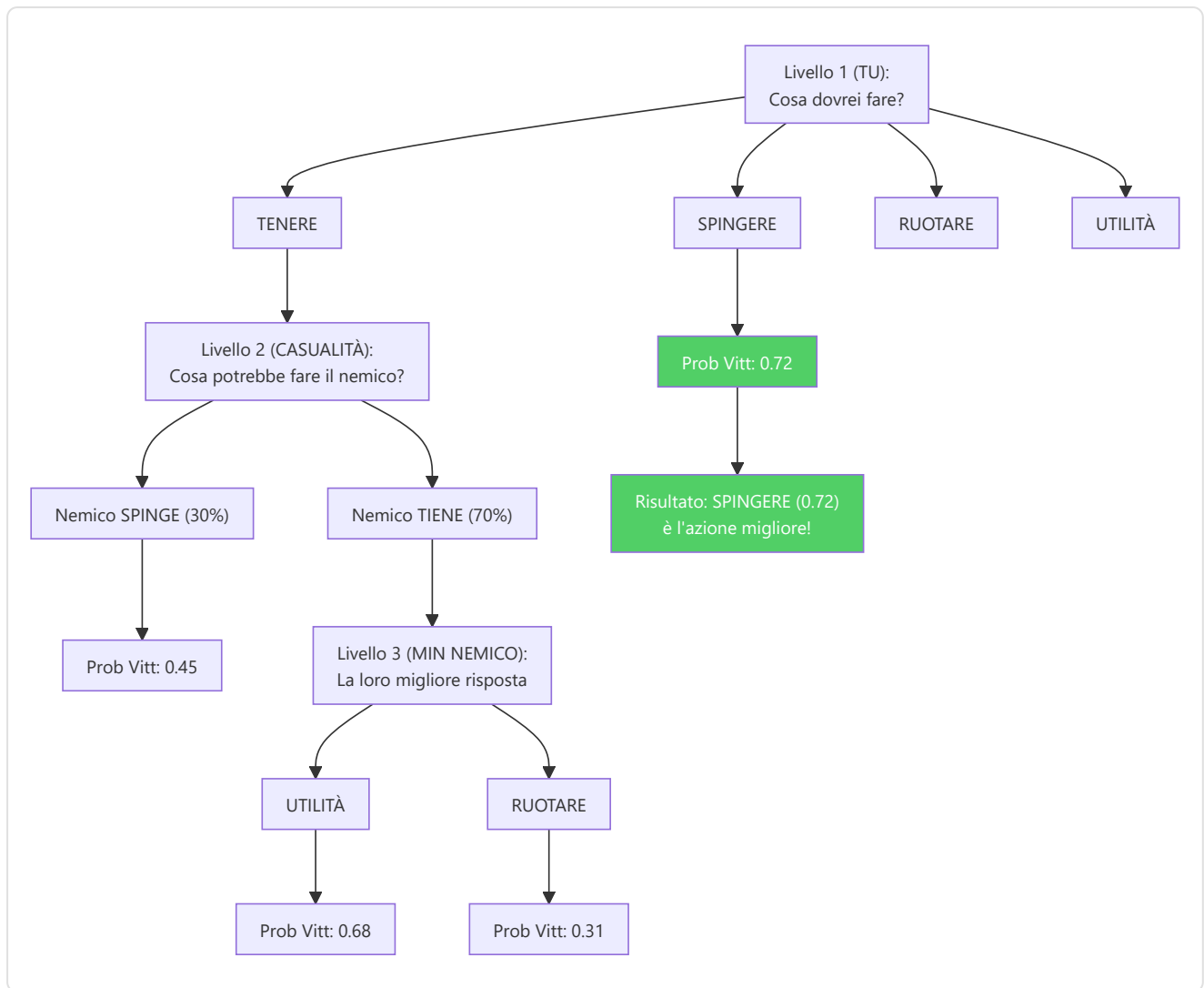
Il `compute_elo_differential(team_histories, enemy_histories)` calcola il differenziale Elo medio tra le due squadre, normalizzato per 400 (una "classe" Elo), e lo converte in probabilità tramite la formula logistica standard. Il blending è conservativo ($\alpha = 0.15$) perché l'Elo cattura solo informazione storica, mentre la NN vede lo stato corrente del round.

Nota: L'Elo è un'**augmentazione opzionale** — l'architettura base a 12 feature del `WinProbabilityNN` rimane invariata. Se lo storico non è disponibile, il predittore ricade sulla probabilità NN pura.

-Albero di gioco Expectiminimax (`game_tree.py`)

Implementa la **ricerca expectiminimax** con modellazione adattiva dell'avversario:

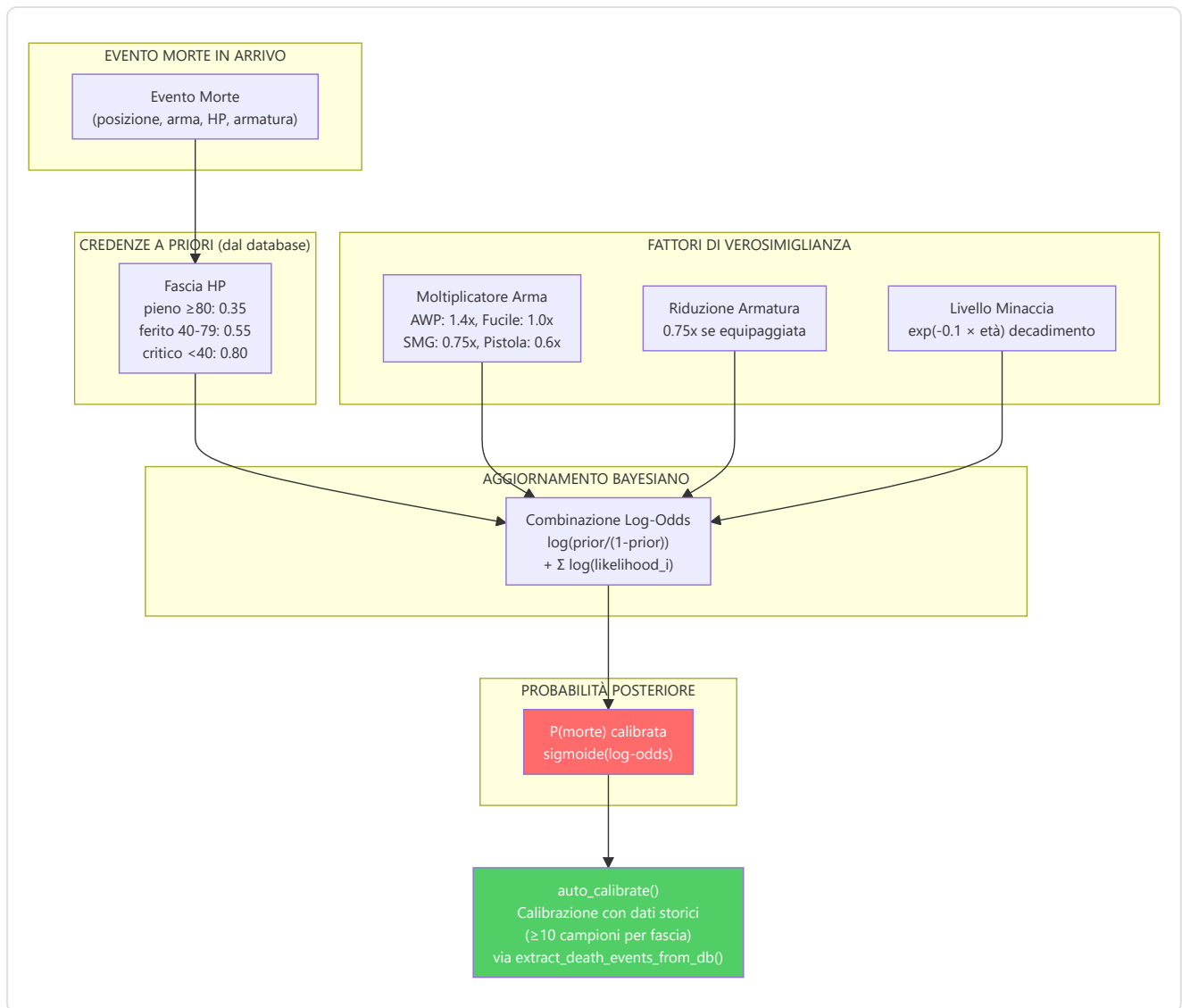
- **Azioni:** spingi, tieni premuto, ruota, usa_utilità
- **Modello avversario:** Priorità economiche (eco/forza/acquisto completo), aggiustamenti laterali, aggiustamenti del vantaggio, pressione temporale; blending EMA attivo da ≥ 10 round osservati
- **Profondità:** 3 livelli di default (max → probabilità → min); il `BlindSpotDetector` invoca l'albero con `depth=2`
- **Budget nodo:** `DEFAULT_NODE_BUDGET = 1000` (impedisce l'esplosione) + transposition table
`_TT_MAX_SIZE = 10.000`
- **Valutazione foglia:** `WinProbabilityPredictor` (caricamento lazy)



-Stimatore Bayesiano di Morte (`belief_model.py`)

Modelli $P(\text{morte} \mid \text{credenza}, \text{HP}, \text{armatura}, \text{classe_arma})$:

- **Antecedente:** Tassi di mortalità per fascia HP (pieno ≥ 80 : 0,35, danneggiato 40-79: 0,55, critico < 40 : 0,80)
- **Fattori di probabilità:** Livello di minaccia (con decadimento esponenziale, `THREAT_DECAY_LAMBDA = 0.1`), riduzione dell'armatura, moltiplicatori delle armi
- **Posteriore:** Combinazione logistica nello spazio log-odds
- **Calibrazione:** `AdaptiveBeliefCalibrator.auto_calibrate()` apprende prior empirici, letalità armi e λ di decadimento minaccia (fit `np.polyfit`), con `MIN_SAMPLES = 100` e bound di sicurezza (prior $[0.05, 0.95]$, letalità $[0.1, 3.0]$, decadimento $[0.01, 1.0]$); ogni calibrazione persiste un `CalibrationSnapshot`



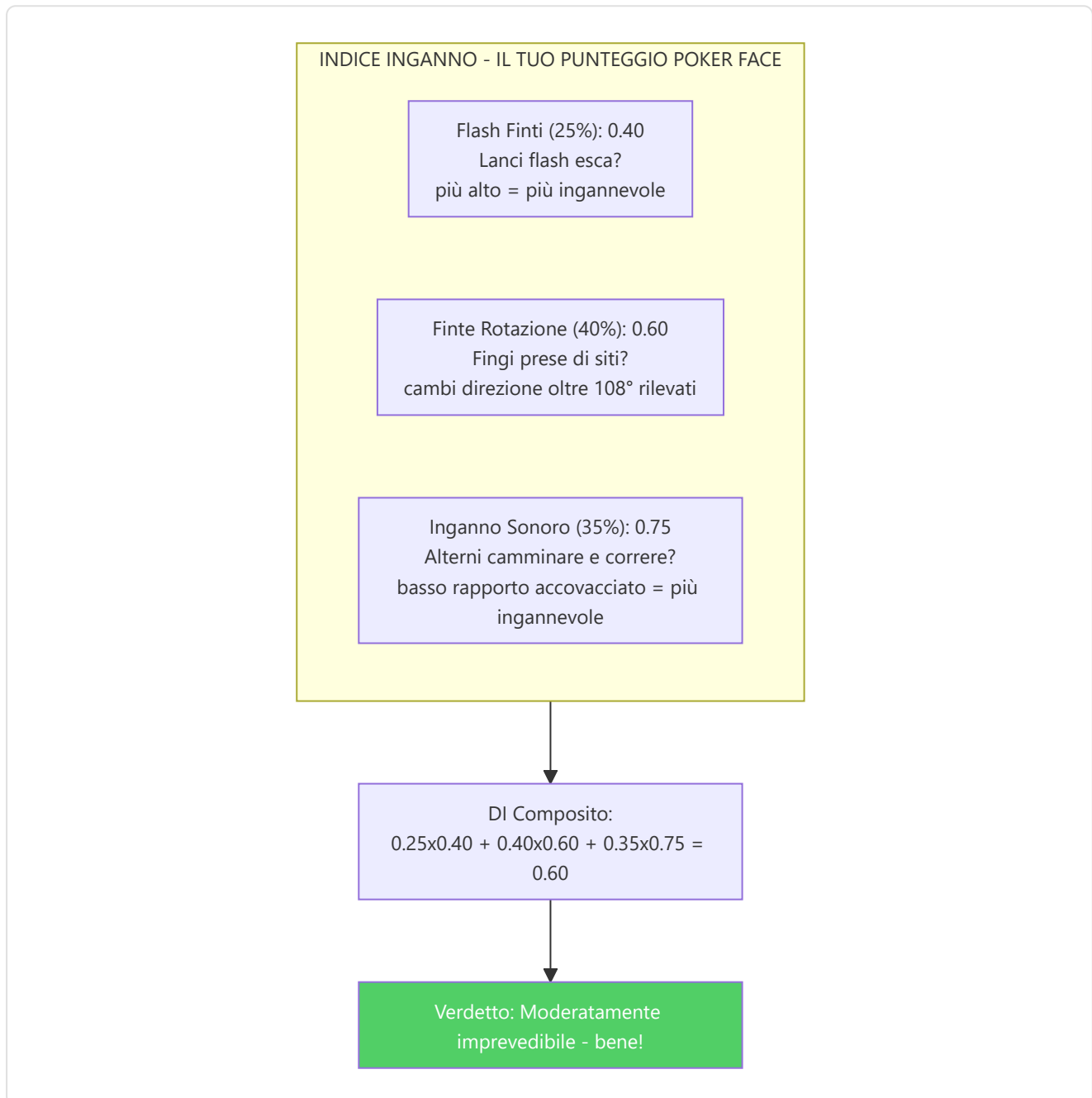
Nota sulla calibrazione (G-07): Lo Stimatore Bayesiano di Morte è ora un **componente live** grazie al cablaggio completato durante la rimediazione. La funzione `extract_death_events_from_db()` nel Session Engine estrae automaticamente gli eventi di morte dal database e li passa ad `auto_calibrate()`, permettendo al modello di affinare le sue probabilità a priori sulla base dei dati reali accumulati. Questo ciclo di feedback trasforma lo stimatore da un modello statico a un sistema che si auto-calibra con l'esperienza.

-Indice di Inganno (`deception_index.py`)

Quantifica l'inganno tattico tramite tre sottometriche:

Sottometrica	Peso	Metodo di rilevamento
Frequenza di falsi flash	0,25	Flash che non accecano i nemici — $\text{bait_rate} = 1 - \text{effettivo}/\text{totale}$
Frequenza di finta rotazione	0,40	Cambiamenti di direzione $> 108^\circ$ rilevati tramite campionamento della velocità angolare (20 intervalli di posizione)
Punteggio di inganno sonoro	0,35	Inverso del rapporto di accovacciamento — $1,0 - \text{rapporto_accovacciamento} \times 2,0$

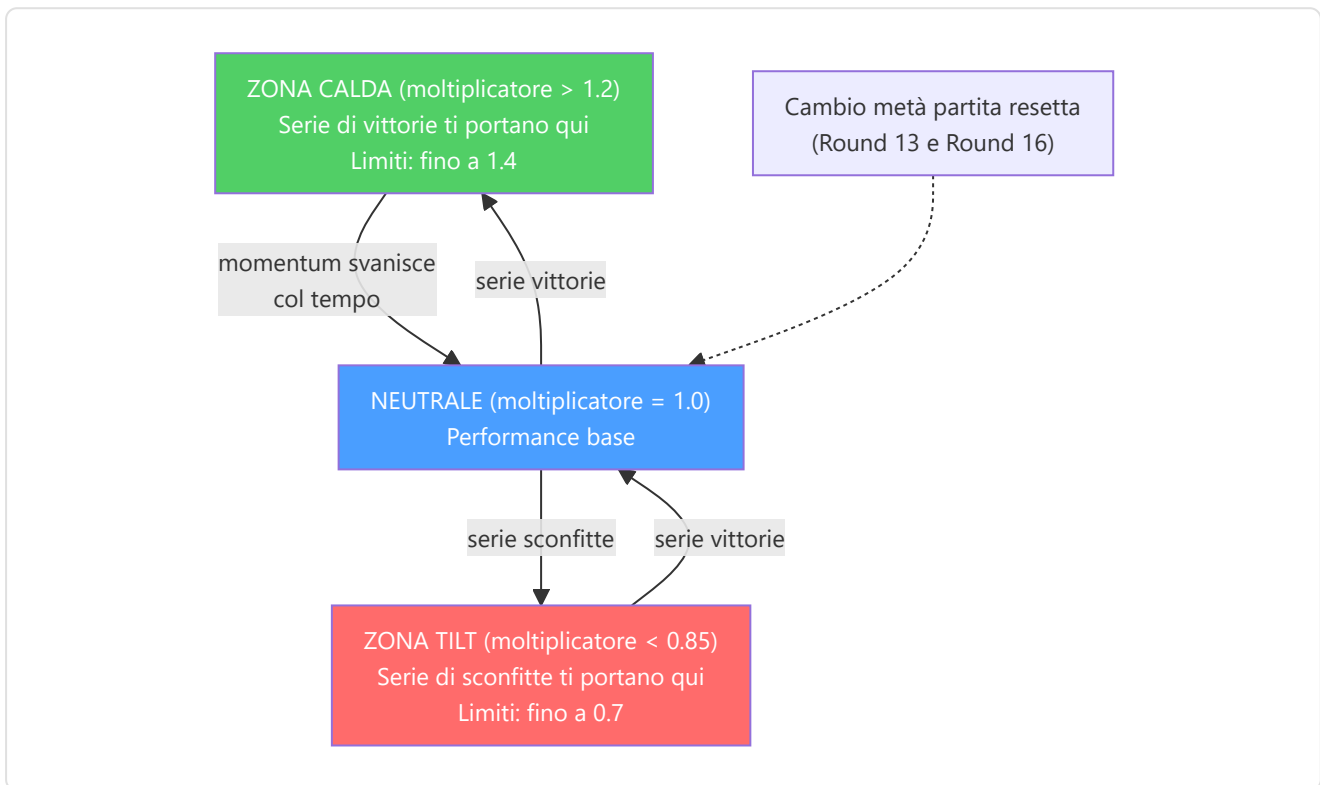
Composito: $\text{DI} = 0,25 \cdot \text{fake_flash} + 0,40 \cdot \text{rotation_feint} + 0,35 \cdot \text{sound_deception}$, fissato a [0, 1].



- Momentum Tracker (`momentum.py`)

Modella il momentum psicologico come un moltiplicatore di prestazioni che decresce nel tempo:

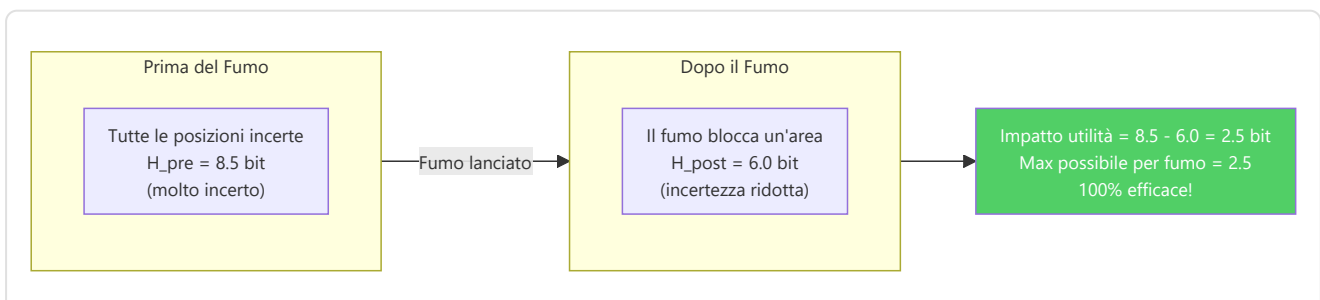
- Serie di vittorie: moltiplicatore = $1,0 + 0,05 \times \text{lunghezza della serie} \times \text{decadimento}$
- Serie di sconfitte: moltiplicatore = $1,0 - 0,04 \times \text{lunghezza della serie} \times \text{decadimento}$
- Decadimento: $\exp(-0,15 \times \text{gap_rounds})$
- Limiti: [0,7, 1,4]
- Rilevamento inclinazione: moltiplicatore < 0,85
- Rilevamento hot: moltiplicatore > 1,2
- Reset di mezzo switch: Round 13 (MR12) e 16 (MR13)



-Analizzatore di Entropia (`entropy_analysis.py`)

Misura l'efficacia dell'utilità tramite la **riduzione di entropia di Shannon** delle posizioni nemiche:

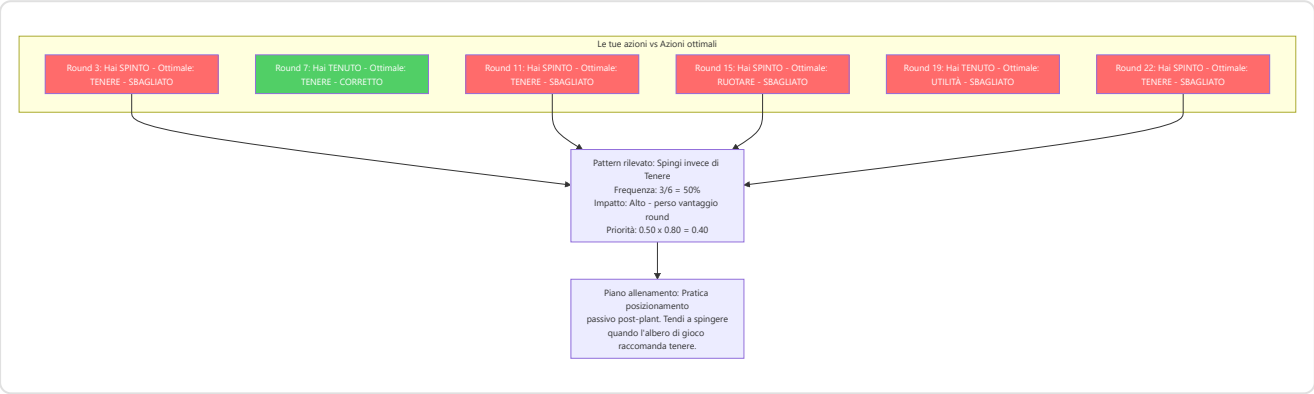
- Discretizza le posizioni in una griglia per-mappa (risoluzione 32–40 celle a seconda della mappa)
- Calcola $H = -\sum p(\text{cell}) \times \log_2(p(\text{cell}))$
- Impatto di utilità = $H_{\text{pre}} - H_{\text{post}}$ (positivo = informazione ottenuta)
- Riduzioni massime di entropia: Smoke 2,5 bit, Molotov 2,0, Flash 1,8, HE 1,5



-Rilevatore di Punti Ciechi (blind_spots.py)

Identifica decisioni ricorrenti non ottimali rispetto alle raccomandazioni dell'albero di gioco:

- Confronta le azioni reali dei giocatori con le azioni ottimali di ExpectiminimaxSearch
- Classifica le situazioni (post-impianto, frizione, eco, round avanzato, vantaggio numerico)
- Priorità = frequenza × impact_rating
- Genera piani di allenamento in linguaggio naturale per i punti ciechi principali



-Analizzatore distanza di ingaggio (engagement_range.py)

Analizza le distanze di uccisione per costruire **profili di ingaggio** specifici per ruolo e posizione:

Componenti principali:

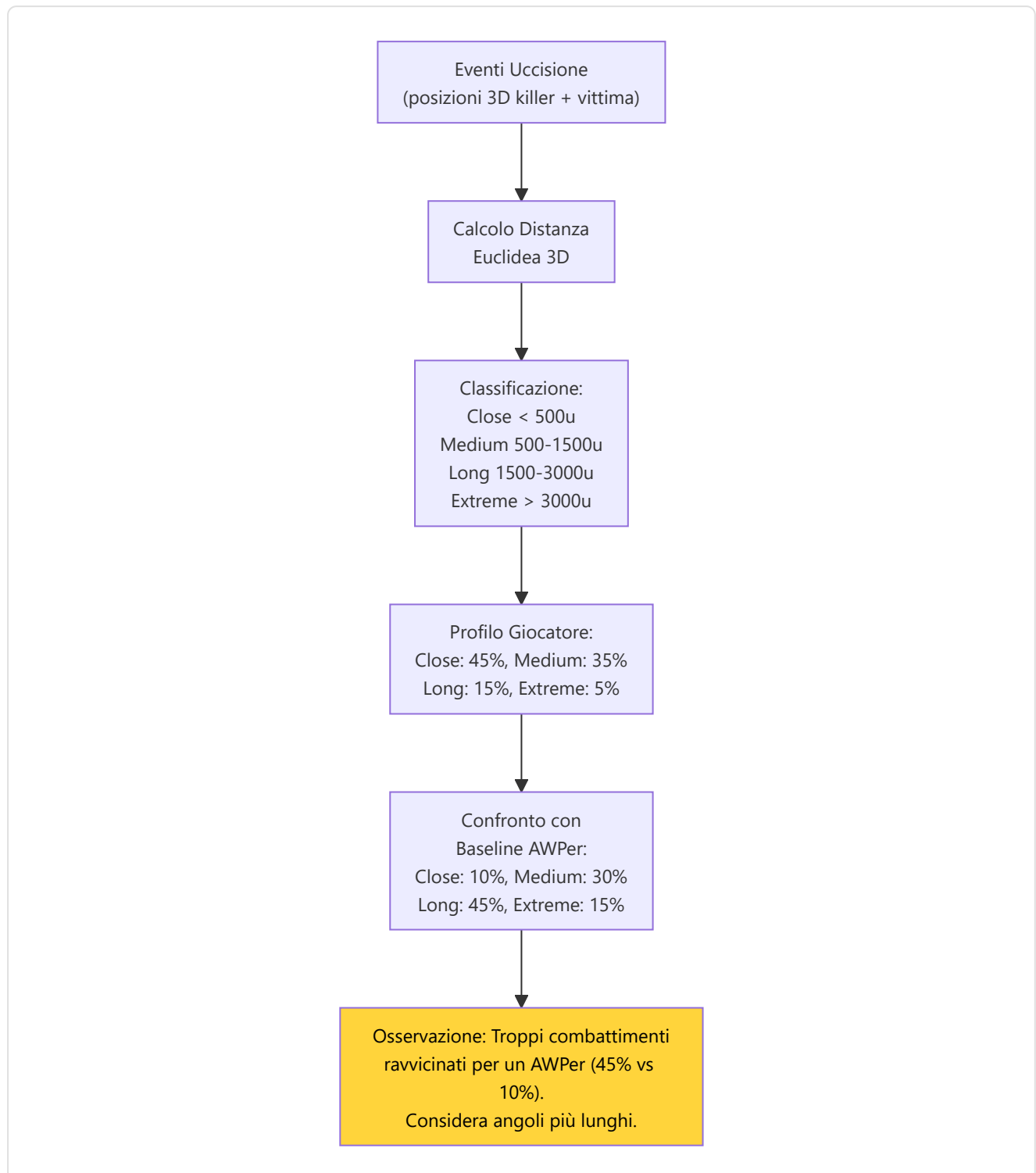
Componente	Scopo
NamedPositionRegistry	Registro di callout (in core/map_callouts.py): 160 posizioni nominate su 9 mappe competitive , con coordinate 3D e raggio
EngagementRangeAnalyzer	Calcolo distanza euclidea killer-vittima, classificazione e confronto con baseline pro
EngagementProfile	Distribuzione % per fascia: close (<500u), medium (500-1500u), long (1500-3000u), extreme (>3000u)

Baseline pro per ruolo:

Ruolo	Close	Medium	Long	Extreme
AWPer	10%	30%	45%	15%
Entry Fragger	40%	40%	15%	5%
Supporto	25%	45%	25%	5%
Lurker	35%	35%	20%	10%
IGL/Flex	25%	40%	25%	10%

Soglia di deviazione: Una differenza >15% rispetto alla baseline del ruolo genera un'osservazione di coaching (es. "Più uccisioni ravvicinate del tipico AWPer — considera angoli più lunghi").

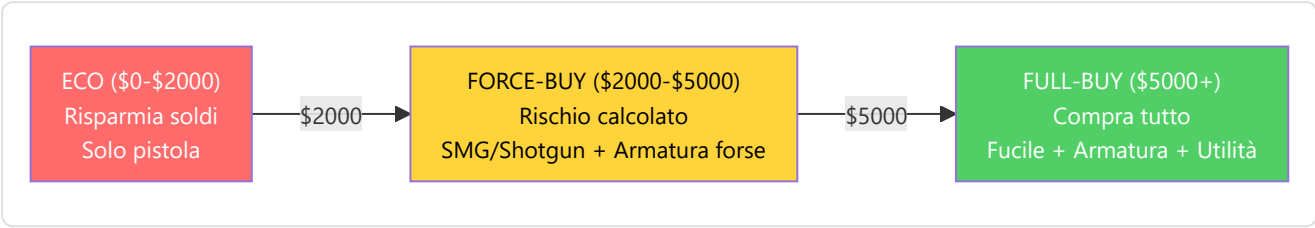
Mappe supportate (9): de_mirage, de_inferno, de_dust2, de_anubis, de_nuke, de_ancient, de_overpass, de_vertigo, de_train (le ultime due fuori dall'Active Duty pool attuale, supportate per demo storiche/workshop).



-Analizzatore di utilità ed economia (`utility_economy.py`)

Analizzatore di utilità: Punteggio di efficacia per tipo rispetto alle basi dei professionisti (Molotov: 35 danni/lancio, Flash: 1,2 nemici/flash, ecc.)

Ottimizzatore di economia: Consigli di acquisto basati su soglie economiche (\$5000 acquisto completo, \$2000 forza, <\$2000 economia), contesto del round, differenziale di punteggio e bonus di sconfitta.



-Analizzatore Qualità Movimento (`movement_quality.py`)

Rileva 4 errori comuni di posizionamento basandosi sul paper MLMove (SIGGRAPH 2024, Stanford/Activision/NVIDIA):

4 Tipologie di errore rilevate:

#	Tipo	Condizione di rilevamento	Descrizione
1	<code>high_ground_abandoned</code>	Discesa ≥ 100 unità senza contesto di combattimento	Abbandono high ground senza necessità
2	<code>position_abandoned</code>	Posizione tenuta $\geq 3s$ lasciata senza nuove info nemiche	Abbandono posizione consolidata
3	<code>over_aggressive_trade</code>	Push solitario dopo morte teammate con < 2 compagni rimasti	Trading troppo aggressivo
4	<code>over_passive_support</code>	Immobile quando teammate crea apertura + vantaggio numerico	Supporto troppo passivo

Soglie chiave:

Costante	Valore	Significato
<code>_ESTABLISHED_HOLD_TICKS</code>	384 (3s)	Tempo minimo per "posizione consolidata"
<code>_HIGH_GROUND_DROP</code>	100.0 unità	Discesa minima per flag high ground
<code>_TRADE_WINDOW_TICKS</code>	640 (5s)	Finestra temporale per analisi trade
<code>_AUDIO_RANGE_DISTANCE</code>	1500.0 unità	Distanza massima per "entro portata audio"
<code>_MOVEMENT_THRESHOLD</code>	300.0 unità	Spostamento minimo per contare come "mosso"
<code>_COMBAT_PROXIMITY_TICKS</code>	64 (~0.5s)	Tick intorno a un evento kill/death per contesto "in combattimento"

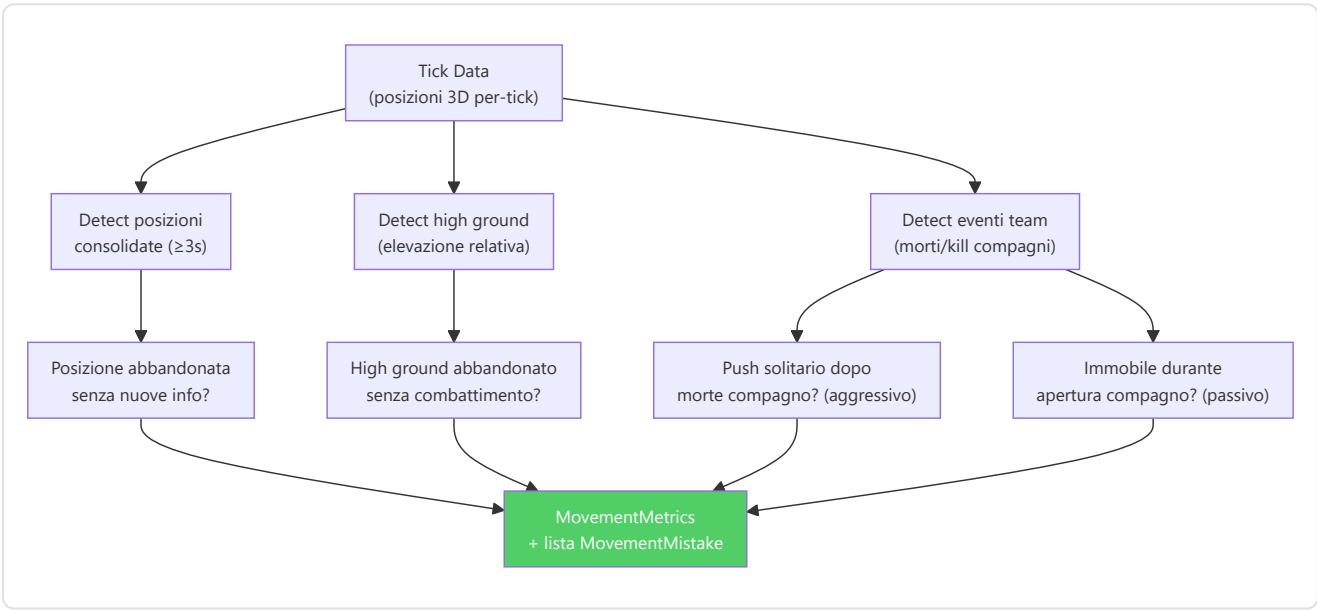
Dataclass di output:

- `MovementMistake` : tipo, round, tick, tempo nel round, descrizione, callout (posizione mappa), severità [0-1]
- `MovementMetrics` : `map_coverage_score`, `high_ground_utilization`, `position_stability`, `total_rounds_analyzed`, lista mistakes + proprietà `mistakes_per_round`

API pubblica:

Metodo	Input	Output
<code>analyze_round_ticks(ticks, map_name, player, round)</code>	Tick di un round	<code>List[MovementMistake]</code>
<code>analyze_match_ticks(all_ticks, map_name, player)</code>	Tutti i tick della partita	<code>MovementMetrics</code> (aggregato)
<code>get_movement_quality_analyzer()</code>	—	Singleton <code>MovementQualityAnalyzer</code>

ADDITIVE: NON modifica METADATA_DIM=25. Le metriche di movimento sono calcolate come feature derivate dai dati tick esistenti, memorizzate nei risultati di analisi.



Riepilogo degli 11 Motori di Analisi

#	Motore	File	Input	Output	Complessità
1	Classificatore Ruoli	<code>role_classifier.py</code>	PlayerMatchStats	Ruolo (6 classi) + confidenza	$O(n)$ features
2	Probabilità Vittoria	<code>win_probability.py</code>	12 feature stato round	$P(\text{CT win}) \in [0,1]$	$O(1)$ forward pass
3	Albero di Gioco	<code>game_tree.py</code>	Stato round + azioni	Nodo ottimale (minimax)	$O(b^d)$ branching
4	Morte Bayesiana	<code>belief_model.py</code>	Posizione + tempo + round	$P(\text{morte})$ + fattori rischio	$O(n)$ prior update
5	Indice Inganno	<code>deception_index.py</code>	Storico round + posizioni	Score imprevedibilità $[0,1]$	$O(n \times m)$ pattern match
6	Tracker Momentum	<code>momentum.py</code>	Sequenza round	Stato hot/cold/neutral	$O(n)$ sliding window
7	Analizzatore Entropia	<code>entropy_analysis.py</code>	Danni utility per tipo	Score efficacia vs pro	$O(k)$ per tipo utility
8	Rilevatore Punti Ciechi	<code>blind_spots.py</code>	Posizioni morte + angoli	Pattern ripetuti	$O(n^2)$ clustering
9	Utilità ed Economia	<code>utility_economy.py</code>	Economia round + utility	Consiglio acquisto + rating	$O(1)$ threshold check
10	Analizzatore Ingaggio	<code>engagement_range.py</code>	Kill events 3D	Profilo distanza (4 fasce)	$O(n)$ euclidean dist
11	Qualità Movimento	<code>movement_quality.py</code>	Tick data 3D per-round	MovementMetrics (4 tipi errore)	$O(n)$ per-tick scan

8. Sottosistema 6 — Elaborazione e Feature Engineering

Directory: `backend/processing/`

Questo sottosistema gestisce tutta la **preparazione dei dati**, trasformando le registrazioni grezze del gioco nei formati numerici precisi di cui le reti neurali hanno bisogno per l'addestramento e l'inferenza.

-Estrattore di Feature Unificato (`vectorizer.py`)

L'**unica fonte di verità** per i vettori di feature a livello di tick. Sia l'addestramento (`RAPStateReconstructor`) che l'inferenza (`GhostEngine`) DEVONO utilizzare questa classe.

Contratto vettoriale di feature a 25 dimensioni:

Indice	Feature	Normalizzazione	Intervallo
0	health	/100	[0, 1]
1	armor	/100	[0, 1]
2	has_helmet	binario	{0, 1}
3	has_defuser	binario	{0, 1}
4	equipment_value	/10000	[0, 1]
5	is_crouching	binario	{0, 1}
6	is_scoped	binario	{0, 1}
7	is_blinded	binario	{0, 1}
8	enemies_visible	/5 (bloccato)	[0, 1]
9	pos_x	/4096	[-1, 1]
10	pos_y	/4096	[-1, 1]
11	pos_z	/1024	[-1, 1]
12	view_yaw_sin	sin(yaw_rad)	[-1, 1]
13	view_yaw_cos	cos(yaw_rad)	[-1, 1]
14	view_pitch	/90	[-1, 1]
15	z_penalty	compute_z_penalty()	[0, 1]
16	kast_estimate	KAST esplicito o da statistiche; default 0,0	[0, 1]
17	map_id	Codifica deterministica basata su hash	[0, 1]
18	round_phase	0=pistola, 0,33=eco, 0,66=forza, 1=pieno	[0, 1]
19	weapon_class	Mappatura classi arma (0-1)	[0, 1]
20	time_in_round	/115 (secondi nel round)	[0, 1]
21	bomb_planted	binario	{0, 1}
22	teammates_alive	/4 (compagni vivi)	[0, 1]
23	enemies_alive	/5 (nemici vivi)	[0, 1]
24	team_economy	/16000 (media soldi team)	[0, 1]

Decisioni progettuali:

- **Codifica ciclica dell'imbardata** (sin/cos agli indici 12-13) elimina la discontinuità di $\pm 180^\circ$
- **Penalità Z** (indice 15) quantifica il rischio di livello errato per mappe multilivello
- **Integrazione del contesto tattico** (indici 19-24) fornisce al modello consapevolezza della situazione di gioco

- **Catena di fallback della stima KAST:** valore esplicito → calcolo dalle statistiche (`estimate_kast_from_stats`) → default 0,0
- **Codifica dell'identità della mappa:** hash deterministico (`md5(map_name) % 10000`) / 10000 consente l'apprendimento specifico della mappa
- **HeuristicConfig** (`base_features.py`) consente di sovrascrivere tutti i limiti di normalizzazione tramite JSON

Decisioni progettuali: La **codifica ciclica dell'imbardata** (sin/cos) risolve la discontinuità angolare: un angolo -179° e +179° differiscono di soli 2° nella realtà ma di 358° in rappresentazione lineare. La coppia (sin θ , cos θ) mappa l'angolo su un cerchio unitario dove la distanza euclidea riflette la distanza angolare reale. La **penalità Z** (`z_penalty` nel vettore 25-dim) segnala esplicitamente il rischio di errore di piano verticale — critico su mappe multilivello (Nuke, Vertigo) dove confondere upper/lower porta a decisioni tattiche completamente errate. L'**integrazione tattica** (economia, vivi, tempo) fornisce al modello il contesto necessario per valutare se una giocata aggressiva è corretta dato il vantaggio numerico, il budget economico e il tempo residuo nel round.

-Valutazione HLTV 2.0 (`rating.py`)

Il **modulo di valutazione unificato** che previene l'inferenza-distorsione nell'addestramento:

```
R = (R_kill + R_survival + R_kast + R_impact + R_damage) / 5

dove:
R_kill = KPR / 0,679
R_survival = (1 - DPR) / 0,317
R_kast = KAST / 0,70
R_impact = (2,13·KPR + 0,42·(ADR/100) - 0,41·(1-DPR)) / 1,0
R_damage = ADR / 73,3
```

Utilizzato da: `demo_parser.py` (analisi), `base_features.py` (aggregazione), `coaching_service.py` (insight).

-Metriche PlusMinus e Rating Role-Adjusted (`rating.py`) — KT-06

Modulo complementare alla valutazione HLTV 2.0 che fornisce due metriche aggiuntive progettate per catturare aspetti che il Rating 2.0 trascura:

PlusMinus:

```
PlusMinus = (kills - deaths) / max(rounds_played, 1) + team_contribution_bonus
```

Componente	Formula	Range tipico
Net frag differential	$(kills - deaths) / rounds$	[-1.0, +1.0]
Team contribution bonus	$_TEAM_CONTRIBUTION_SCALE \times (team_win_rate - 0.5)$	[-0.05, +0.05]
$_TEAM_CONTRIBUTION_SCALE$	0.10	—

Il bonus di contribuzione squadra premia i giocatori nelle squadre vincenti e penalizza quelli nelle squadre perdenti, analogamente al +/- nel basket/hockey.

Rating Role-Adjusted (Bayesian):

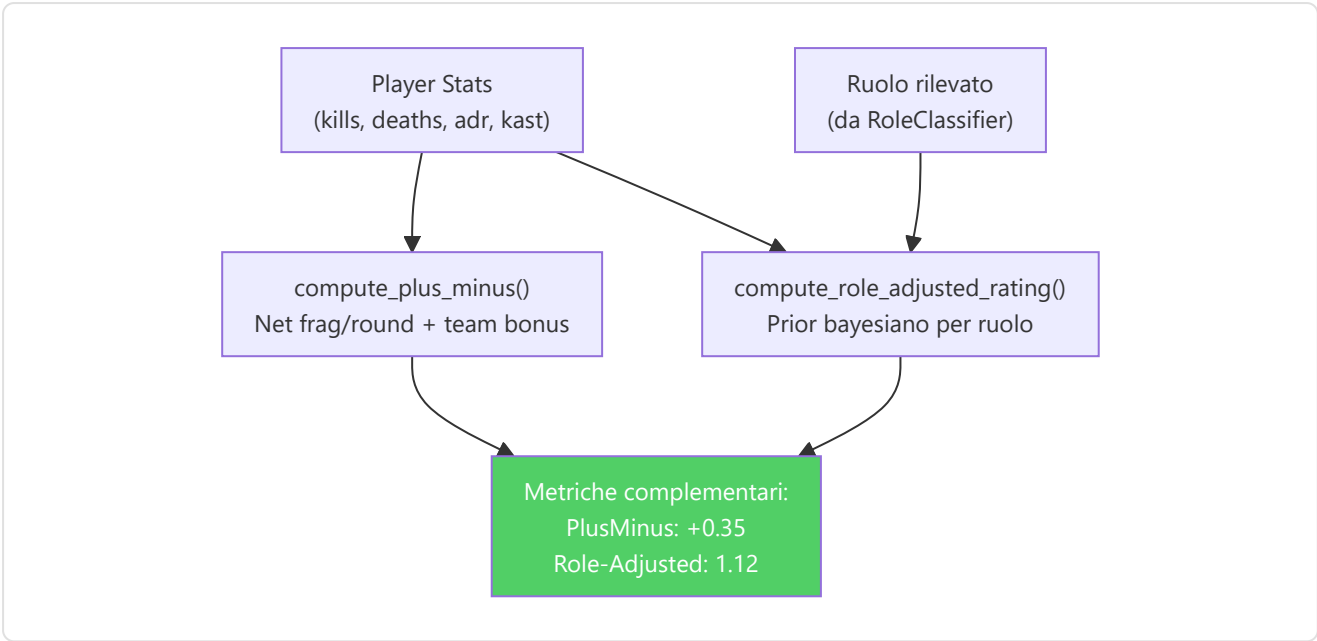
Applica prior bayesiani specifici per ruolo in modo che AWPer non siano penalizzati per KAST inferiore e support non siano penalizzati per K/D inferiore:

Ruolo	K/D Prior	KAST Prior	ADR Prior	Weight
AWPer	1.15	0.68	75.0	5.0
Entry	0.95	0.72	80.0	5.0
Support	0.90	0.78	65.0	5.0
Lurker	1.05	0.70	72.0	5.0
IGL	0.88	0.74	68.0	5.0

Prior calibrati dai dati medi dei team top-30 HLTV (stagione 2024-2025). Formula composita:

```
adj_metric = (n × observed + weight × prior) / (n + weight)
role_adjusted_rating = 0.40 × adj_kd + 0.35 × adj_kast + 0.25 × adj_adr_norm
```

Dove $adj_adr_norm = adj_adr / 120$ normalizza l'ADR in [0, 1]. Il framework è ispirato a TrueSkill (Herbrich et al., NeurIPS 2006) — quando il campione è piccolo (n basso), il prior domina; quando il campione è grande, i dati osservati dominano.

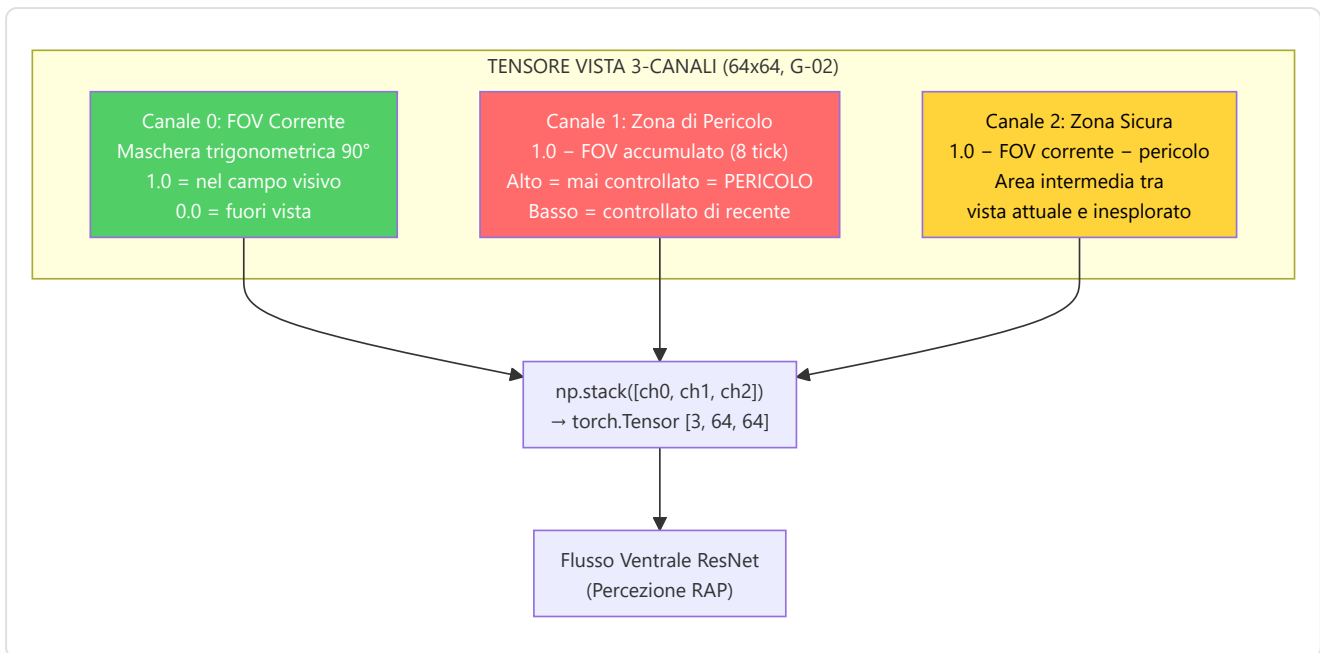


-Tensor Factory (`tensor_factory.py`)

Converte i dati tick grezzi in tensori di immagini 64x64 per il livello di percezione RAP:

Tensor	Canali	Contenuto
Mappa	3 (R/G/B)	R: posizione del giocatore, G: compagni di squadra (α -blended), B: nemici (α -blended)
Vista	3	Ch0 : maschera FOV corrente (90° predefinito, mascheramento trigonometrico); Ch1 : zona di pericolo (= 1 – FOV accumulato sugli ultimi 8 tick), le aree mai controllate sono potenziali posizioni nemiche; Ch2 : zona sicura (= 1 – FOV corrente – zona di pericolo), l'area tra la vista attuale e le zone inesplorate
Movimento	2 o 3	Mappe di calore di velocità/accelerazione (gocce 2D sfocate gaussiane)

Correzione G-02 (Zona di Pericolo): In precedenza, il tensore vista aveva 3 canali identici (placeholder). Dopo la rimediazione, i 3 canali codificano informazioni tattiche distinte: il **FOV corrente** mostra ciò che il giocatore vede adesso, la **zona di pericolo** accumula la storia FOV degli ultimi 8 tick (~125ms a 64 Hz) e inverte il risultato per identificare le aree mai controllate — dove i nemici potrebbero nascondersi — e la **zona sicura** rappresenta l'area intermedia tra vista attuale e zone inesplorate. Il calcolo della zona di pericolo usa `np.maximum(accumulated_fov, tick_fov)` per ogni tick storico, poi `danger_zone = 1.0 - accumulated_fov`. Questo fornisce al modello RAP una consapevolezza spaziale della copertura visiva, trasformando il tensore vista da un semplice input statico a un **indicatore tattico temporale**.



-Heatmap Engine (`heatmap_engine.py`)

Mappe di occupazione gaussiane ad alte prestazioni per la visualizzazione tattica:

- Generazione di dati thread-safe (`generate_heatmap_data()`)
- Creazione di texture solo nel thread principale (OpenGL)
- Heatmap differenziali con rilevamento di hotspot per l'allenamento posizionale

-Data Pipeline (`data_pipeline.py`)

`ProDataPipeline` gestisce la preparazione dei dati ML-ready:

1. **Recupera** tutti i `PlayerMatchStats` dal database
2. **Pulisce** i valori anomali (`avg_adr < 400` , `avg_kills < 3.0`)
3. **Ridimensiona** tramite `StandardScaler`
4. **Suddivide** temporalmente (70/15/15) con ordinamento cronologico per gruppo (pro/utente)
5. **Mantiene** la colonna `dataset_split` sul posto

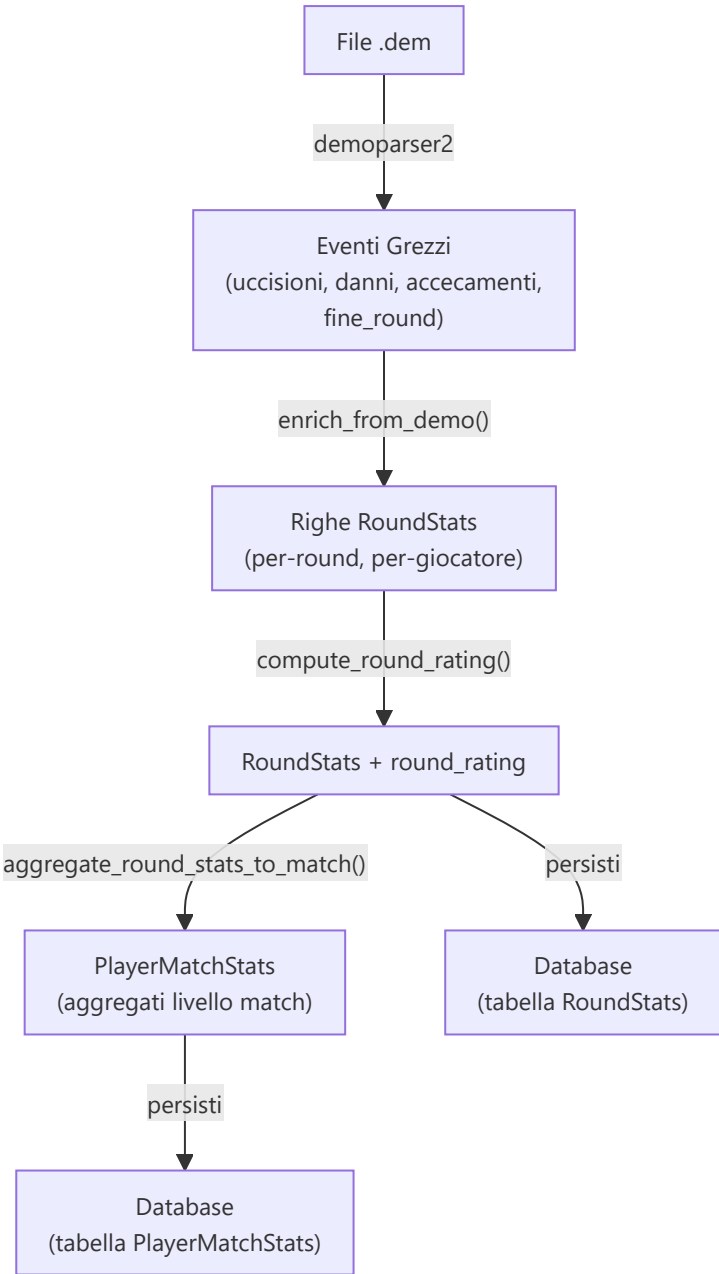
-Generatore di statistiche per round (`round_stats_builder.py`)

Collega gli eventi demo grezzi al **livello di isolamento per round** (`RoundStats` in `db_models.py`), impedendo la contaminazione statistica tra round e consentendo analisi dettagliate del coaching per round.

Funzioni chiave:

Funzione	Scopo
<code>build_round_stats(parser, demo_name)</code>	Analizza gli eventi <code>round_end</code> , <code>player_death</code> , <code>player_hurt</code> , <code>player_blind</code> e costruisce le statistiche per round
<code>enrich_from_demo(demo_path, demo_name)</code>	Arricchimento completo: uccisioni noscope/cieche, conteggio flash/fumo, assist flash, uccisioni con scambio, analisi delle utilità
<code>aggregate_round_stats_to_match(round_stats_list)</code>	Elabora le statistiche a livello di round → <code>PlayerMatchStats</code> a livello di partita
<code>compute_round_rating(round_stats)</code>	Valutazione HLTV 2.0 per round utilizzando il modulo di valutazione unificato

Pipeline di arricchimento:



Campi di arricchimento uccisioni aggiunti da `enrich_from_demo()` :

Campo	Metodo di rilevamento
<code>noscope_kills</code>	Uccidi dove AWP/Scout/SSG è equipaggiato ma <code>is_scoped=False</code>
<code>blind_kills</code>	Uccidi dove <code>attacker_blinded=True</code> al momento della morte
<code>flash_assists</code>	Uccidi entro 128 tick (2 s) da un nemico colpito da un flash dalla stessa squadra
<code>thrusmoke_kills</code>	Uccidi attraverso granata fumogena (dai flag dell'evento demo)
<code>wallbang_kills</code>	Uccidi attraverso la penetrazione del muro (dai flag dell'evento demo)
<code>trade_kills</code>	Uccidi entro 5 s dalla morte di un compagno di squadra per mano dello stesso nemico

Integrazione con le pipeline di ingestione: Sia `user_ingest.py` che `pro_ingest.py` ora chiamano `enrich_from_demo()` dopo l'analisi della demo e rendono persistenti gli oggetti `RoundStats` risultanti nel database. Questo garantisce che ogni demo ingerita, utente o professionista, produca dati granulari a livello di round.

-Decadimento della baseline temporale (`pro_baseline.py`)

La classe `TemporalBaselineDecay` integra (non sostituisce) la funzione `get_pro_baseline()` esistente con una media ponderata nel tempo, garantendo che i confronti tra i coach riflettano l'**attuale meta CS2** piuttosto che le obsolete medie storiche.

Formula di decadimento:

```
weight(age_days) = max(MIN_WEIGHT, exp(-ln(2) × age_days / HALF_LIFE))
```

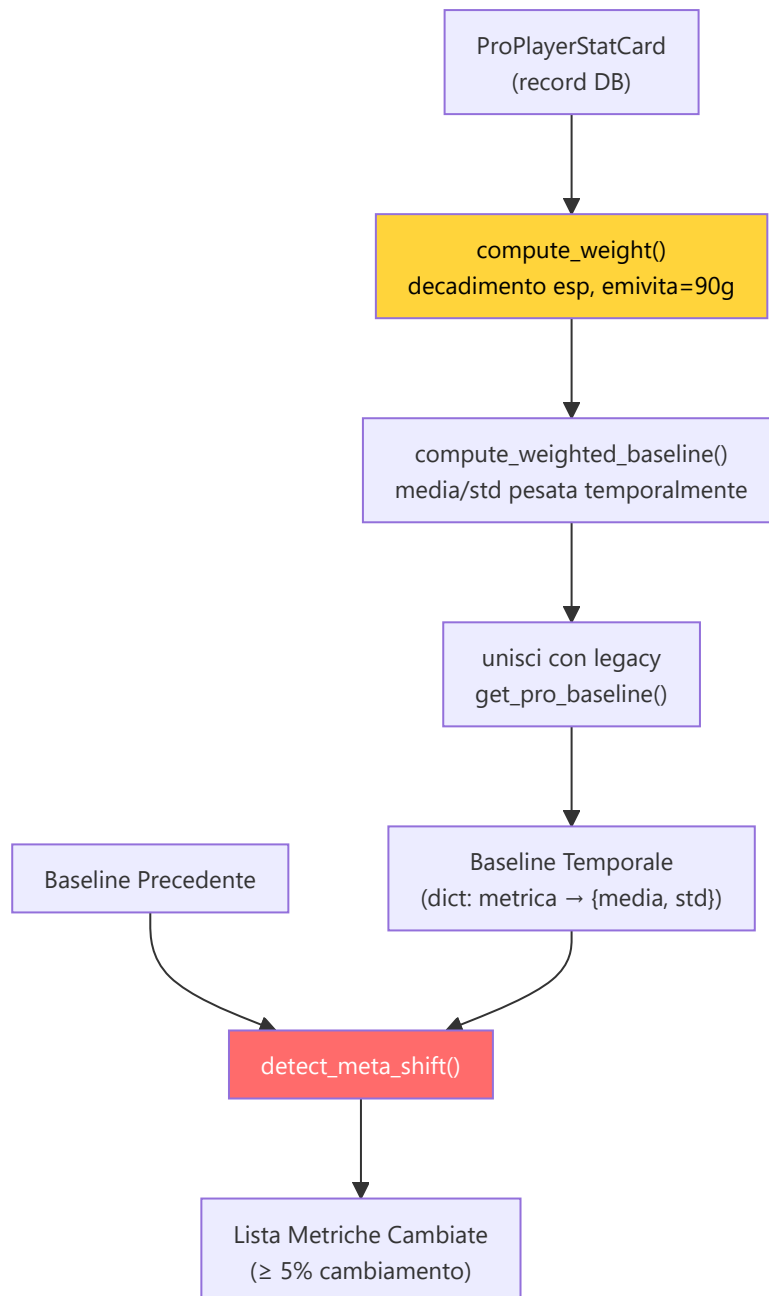
Parametro	Valore	Significato
<code>HALF_LIFE_DAYS</code>	90	Il peso si dimezza ogni 90 giorni
<code>MIN_WEIGHT</code>	0,1	Soglia minima: dati molto vecchi contribuiscono ancora per il 10%
<code>META_SHIFT_THRESHOLD</code>	0,05	Una variazione del 5% tra le epoche segnala un cambiamento del meta

Metodi chiave:

Metodo	Scopo
<code>compute_weight(stat_date)</code>	Peso di decadimento esponenziale per una singola scheda statistica
<code>compute_weighted_baseline(stat_cards)</code>	Media/std ponderata nel tempo su tutti i record <code>ProPlayerStatCard</code>
<code>get_temporal_baseline(map_name)</code>	Pipeline completa: recupero schede → peso → unione con i valori predefiniti legacy
<code>detect_meta_shift(old, new)</code>	Segnala le metriche che hanno subito uno scostamento $\geq 5\%$ tra le epoche di riferimento

Punti di integrazione:

- **CoachingService:** `_get_temporal_baseline()` lo utilizza per l'arricchimento COPER
- **Teacher Daemon:** `_check_meta_shift()` viene eseguito dopo ogni ciclo di riaddestramento, registrando le metriche che hanno subito uno scostamento



-Validazione (**validation/**)

- **drift.py**: Rilevamento della deriva nella distribuzione dei dati con oggetti DriftReport
- **schema.py**: Validazione dello schema per i record del database
- **sanity.py**** / **dem_validator.py**** Controlli di integrità dei dati e dei file demo

Copertura quantitativa: Il progetto comprende **167 file di test** (**test_*.py** in **Programma_CS2_RENAN/tests/**) più i test e gli script **verify_*.py** nella **tests/** root, e un **headless validator** con **39 fasi tematiche di controllo** (da Ambiente/Import fino a Security, GPU, Design-Tokens e

Quality-Adv). Questa copertura spazia dall'integrità dello schema DB alla coerenza dei vettori di embedding, dalla correttezza delle pipeline di addestramento alla validazione end-to-end dei flussi di coaching.

-PlayerKnowledge — Sistema Percettivo NO-WALLHACK (`player_knowledge.py`)

Modello di percezione **Player-POV** che ricostruisce ciò che un giocatore legittimamente sa in ogni tick, senza informazioni da wallhack. Questo è il fondamento per coaching eticamente corretto: il coach vede solo ciò che il giocatore vedeva.

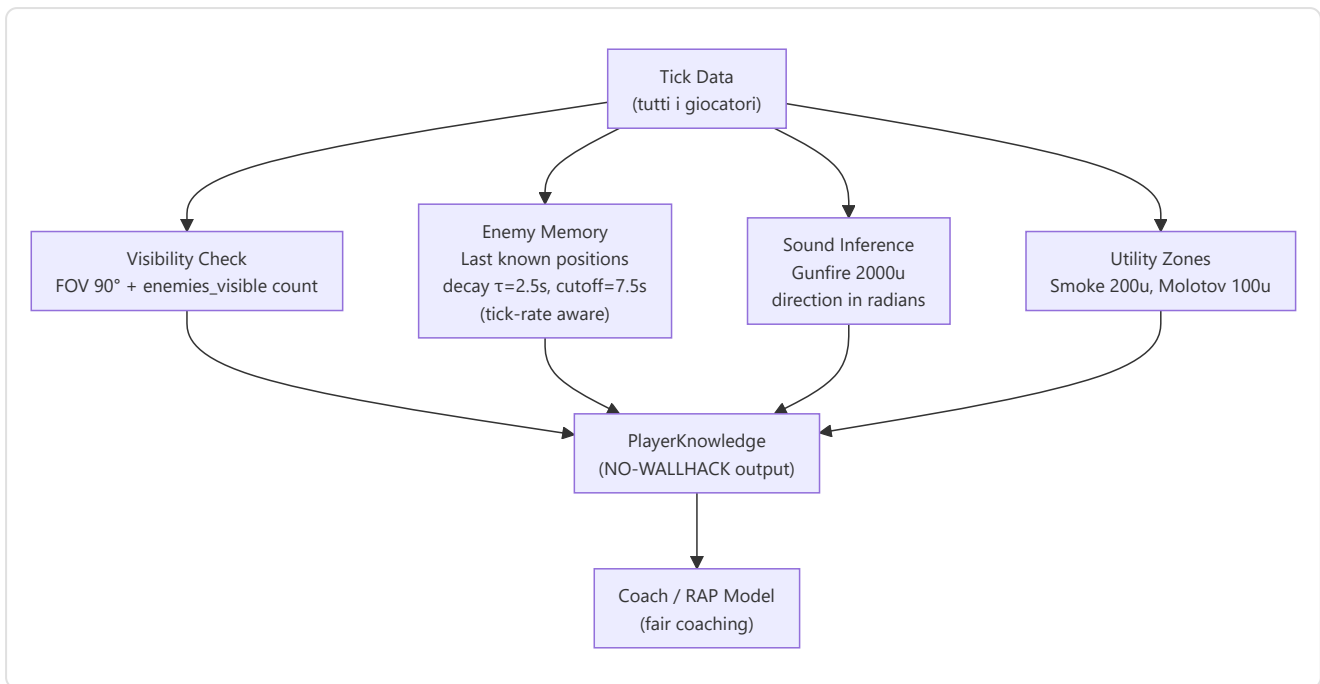
Dataclass di percezione:

Dataclass	Campi	Descrizione
<code>VisibleEntity</code>	position, distance, is_teammate, health, weapon	Entità visibile nel FOV
<code>LastKnownEnemy</code>	position, tick_seen, confidence	Posizione nemica dalla memoria (decadimento)
<code>HeardEvent</code>	position, distance, direction_rad, event_type	Evento sonoro percepito
<code>UtilityZone</code>	position, radius, utility_type, team	Zona utilità attiva
<code>PlayerKnowledge</code>	own_state, visible_entities, last_known, heard_events, utility_zones	Output completo

Costanti sensoriali (le costanti temporali vivono in `core/constants.py` **in secondi** e sono convertite in tick moltiplicando per il tick-rate della demo — tick-rate aware):

Costante	Valore	Significato CS2
<code>FOV_DEGREES</code>	90.0	Campo visivo orizzontale CS2
<code>HEARING_RANGE_GUNFIRE</code>	2000.0	Distanza massima percezione spari (world units)
<code>MEMORY_DECAY_TAU_S</code>	2.5s	Emivita memoria nemici ($\times \text{tick_rate} \rightarrow$ decadimento $\exp(-\text{ticks}/\tau)$)
<code>MEMORY_CUTOFF_S</code>	7.5s	Cutoff memoria ($3 \times \tau$)
<code>FLASH_DURATION_S</code> / <code>SMOKE_DURATION_S</code> / <code>MOLOTOV_DURATION_S</code>	2.0s / 18.0s / 7.0s	Durate delle utility
<code>SMOKE_RADIUS</code> / <code>FLASH_RADIUS</code> / <code>MOLOTOV_RADIUS</code>	200.0 / 400.0 / 100.0	Raggi delle zone utilità
<code>MAX_TRACKED_ENEMIES</code> / <code>MAX_HISTORY_TICKS</code>	10 / 512	Limiti di tracking

Pipeline percettiva:



Verifica FOV: `_is_in_fov()` usa trigonometria (`atan2` + `angle_diff`) per determinare se un target rientra nel campo visivo orizzontale del giocatore. Solo i nemici confermati visibili (dal contatore `enemies_visible` della demo) vengono inclusi.

-RAPStateReconstructor (`state_reconstructor.py`)

Vision Bridge Phase 1: Converte sequenze di `PlayerTickState` dal DB in tensori di percezione per il modello RAP-Coach:

- **Metadati:** Vettore METADATA_DIM (25) dal `FeatureExtractor` unificato
- **Percezione:** Tensori view/map/motion dalla `TensorFactory`
- **Player-POV:** Modalità opzionale via parametro `knowledge` (PlayerKnowledge)
- **Windowing:** Finestre temporali di `sequence_length=32` tick con overlap 50%

-ConnectMapContext (`connect_map_context.py`)

Feature spaziali Z-aware per mappe multilivello (Task 2.17.1):

Costante	Valore	Uso
<code>Z_LEVEL_THRESHOLD</code>	200	Separazione tra livelli (world units)
<code>Z_PENALTY_FACTOR</code>	2.0×	Moltiplicatore distanza cross-level

Output: Vettore di 6 feature spaziali normalizzate [0,1]: distanza da bombsite A/B, distanza da spawn T/CT, distanza da mid, penalità Z. Distanze calcolate con `distance_with_z_penalty()` per penalizzare percorsi cross-level su Nuke/Vertigo.

-EliteAnalytics (`external_analytics.py`)

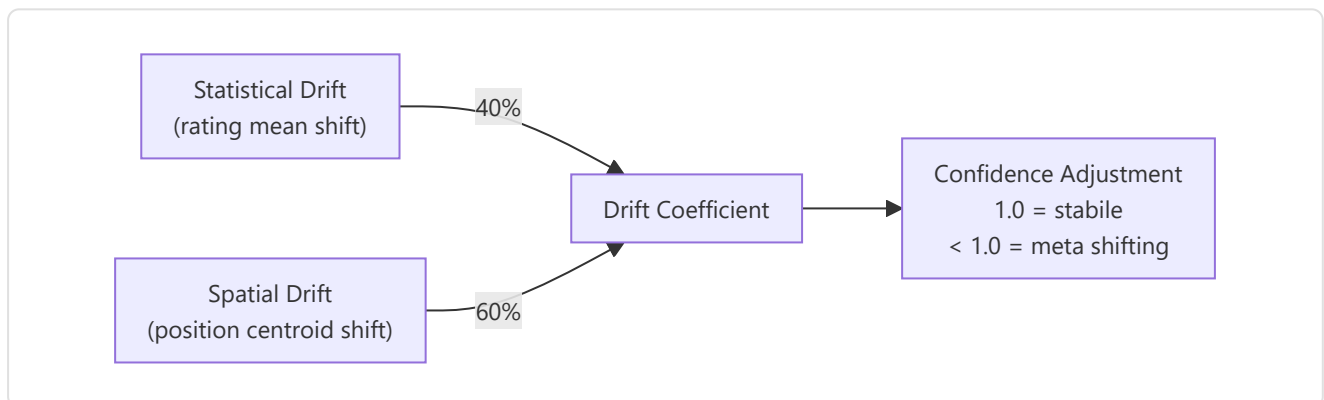
Carica e analizza dataset CSV di riferimento (top 100 giocatori, statistiche match, dati tornei):

- **7 dataset:** top100, top100_core, match_stats, player_stats, kills_processed, historical_stats, tournament_stats
- **Calcolo Z-score:** `_calc_z_scores()` confronta utente vs media storica per ADR, deaths, kills, rating, HS%
- **Z-score tornei:** `_calc_tournament_z()` confronta vs baseline tournament per accuracy, econ_rating, utility_value
- **Graceful degradation:** `is_healthy()` = True se ≥ 1 dataset caricato con successo

-MetaDriftEngine (`meta_drift.py`)

Pillar 2 Phase 3 — Meta-Drift Surveillance:

Confronta posizioni pro degli ultimi 30 giorni vs storico per rilevare cambiamenti nella meta:



Integrazione: `HybridCoachingEngine._calculate_confidence()` chiama `MetaDriftEngine.get_meta_confidence_adjustment()` per penalizzare la confidenza degli insight quando la meta è instabile.

-NicknameResolver (`nickname_resolver.py`)

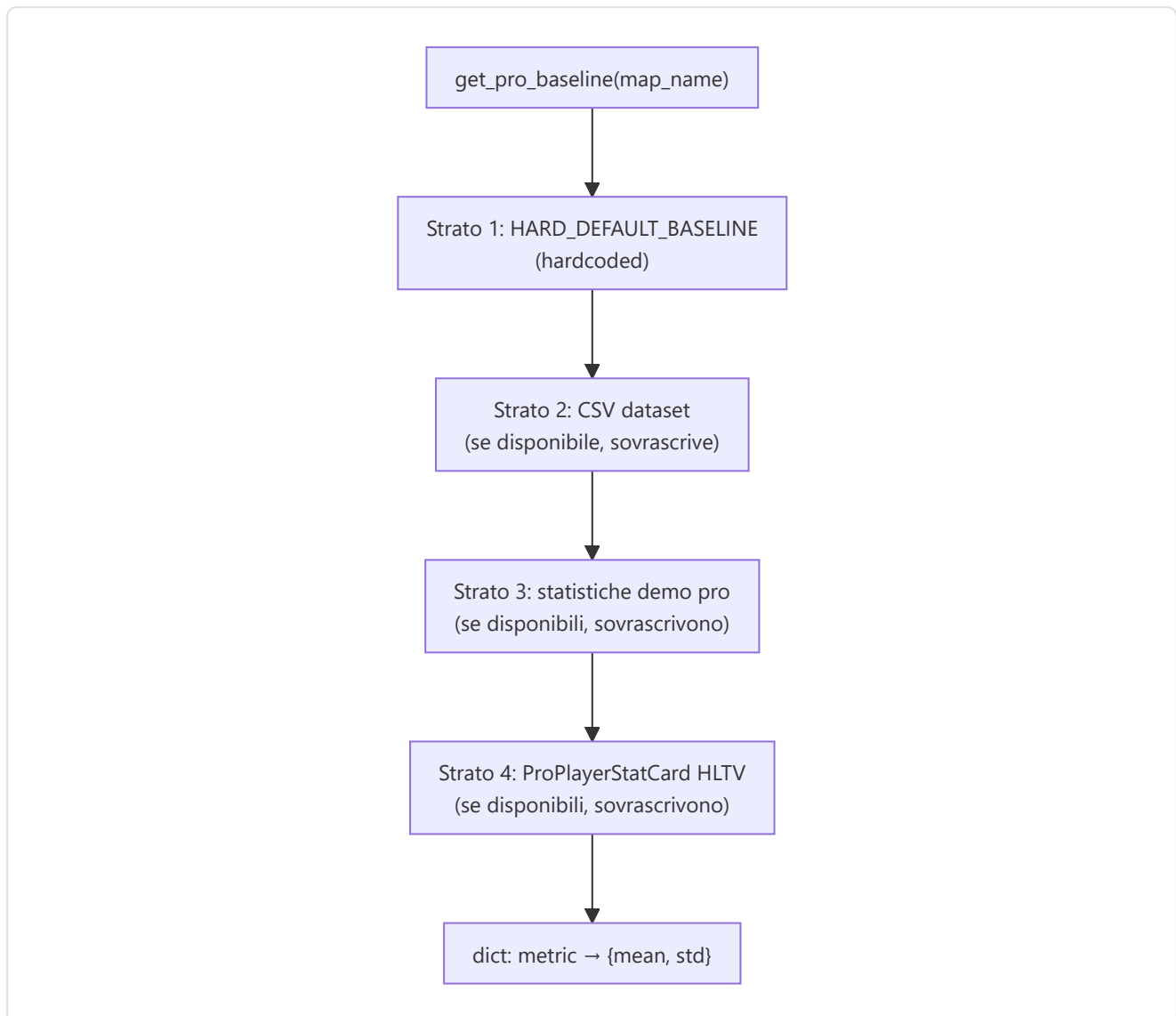
Task 2.18.2: Risolve nickname in-game delle demo in HLTV ProPlayer ID con matching a 3 livelli:

Livello	Metodo	Esempio
1. Esatto	Case-insensitive match	"s1mple" → ProPlayer.hltv_id
2. Sottosstringa	Nickname contenuto nel nome demo	"FaZe_ropz" contiene "ropz"
3. Fuzzy	SequenceMatcher ≥ 0.8 threshold	"s1mpl3" $\approx$ "s1mple"

Gestisce tag di team e prefissi clan. Complessità $O(n)$ per query, accettabile per < 1.000 pro nel DB.

-ProBaseline (`pro_baseline.py`)

Task 2.18.1: Sistema di baseline pro con supporto map-specific e catena di fallback a 3 livelli:



HARD_DEFAULT_BASELINE — dizionario di metriche hardcoded con mean + std, ultimo livello della catena di fallback. Copre: `rating`, `kd_ratio`, `avg_kills`, `avg_deaths`, `avg_adr`, `avg_hs`, `avg_kast`, `accuracy`, aggression, utility, opening/clutch, `rating_impact`, `rating_survival`, `rating_kast`. La baseline reale è la **fusione a 4 strati**: hardcoded → CSV → statistiche demo → HLTV (`get_pro_baseline()`).

`calculate_deviations(player_stats, baseline)` — Calcola Z-score per ogni feature: $z = \frac{\text{player_value} - \text{mean}}{\text{std}}$. Gestisce sia baseline flat (legacy) che strutturate (dict con mean/std).

-RoleThresholdStore (`role_thresholds.py`)

Principio anti-mock: Tutte le soglie di ruolo apprendono esclusivamente da dati pro reali (HLTV, demo, ML):

Statistica	Descrizione	Cold-Start Default
<code>awp_kill_ratio</code>	% uccisioni con AWP	—
<code>entry_rate</code>	Tasso di entry frag	—
<code>assist_rate</code>	Tasso di assistenze	—
<code>survival_rate</code>	Tasso di sopravvivenza	—
<code>solo_kill_rate</code>	Tasso uccisioni solitarie	—
<code>first_death_rate</code>	Tasso prima morte	—
<code>utility_damage_rate</code>	Danno utilità per round	—
<code>clutch_rate</code>	Tasso di clutch vinti	—
<code>trade_rate</code>	Tasso di trade kill	—

Cold-start protection:

Requisito	Valore	Significato
<code>MIN_SAMPLES_FOR_VALIDITY</code>	30	Minimo campioni per soglia valida
Soglie valide minime	3	Per uscire dal cold-start
Cold-start output	<code>(FLEX, 0.0)</code>	Ruolo generico, 0% confidenza
Apprendimento soglie	75° percentile	Le soglie sono apprese dal 75° percentile dei dati pro reali

Persistenza: `persist_to_db()` / `load_from_db()` — le soglie apprese sopravvivono ai riavvii.
`validate_consistency()` verifica la coerenza interna (es. `entry_rate` non può essere negativo).

9. Sottosistema 7 — Modulo di Controllo

Cartella nella repo: `backend/control/` **File:** 4 moduli

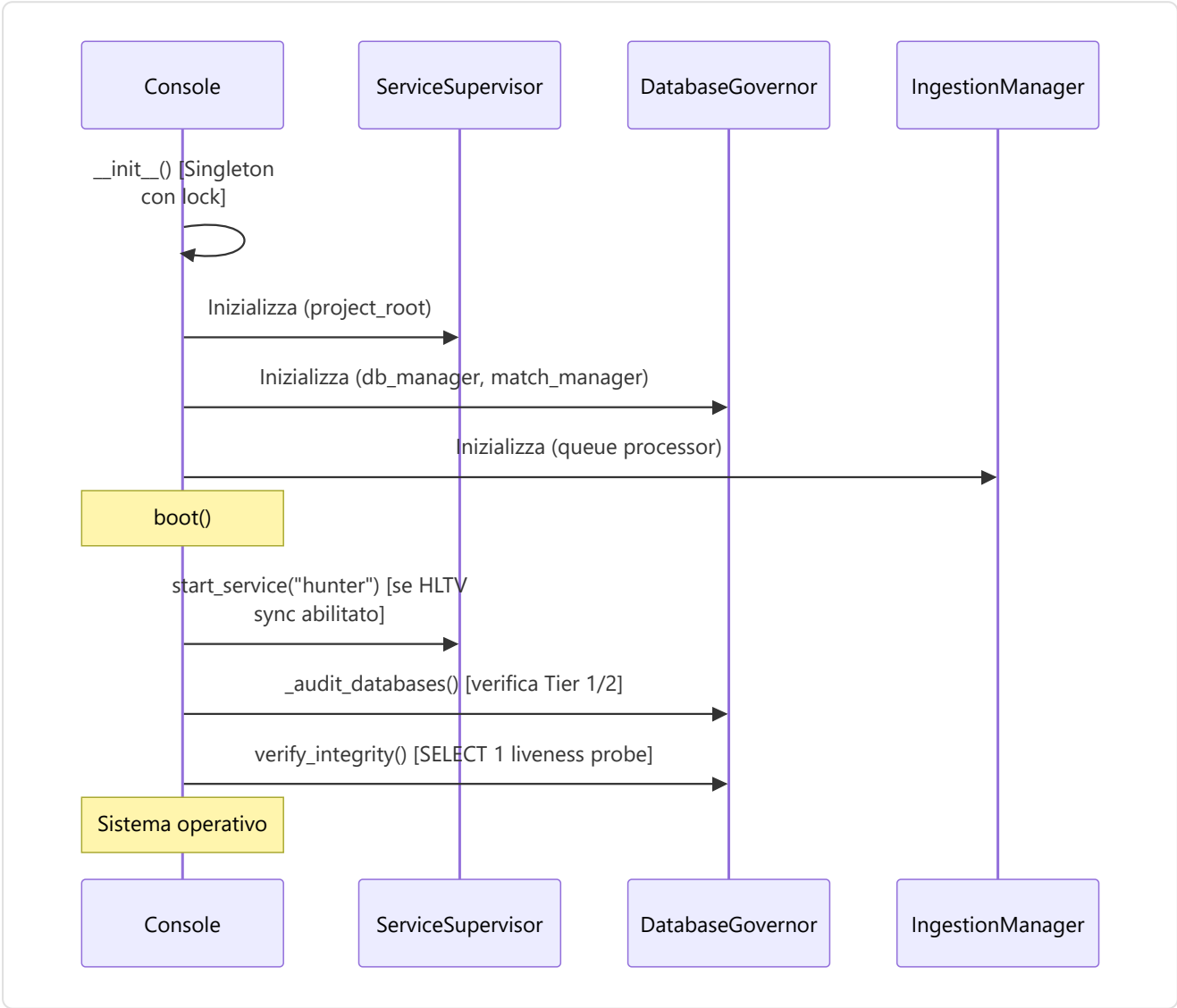
Questo sottosistema è la **torre di controllo** del sistema: supervisiona servizi background, governa i database, gestisce l'ingestione delle demo e controlla il ciclo di vita del training ML.

-Console (`console.py`) — Singleton

La **Console di Controllo Unificata**: autorità centrale per ML, Ingestion e System State.

Componente	Ruolo
ServiceSupervisor	Supervisore background daemons (PID, liveness, auto-restart)
IngestionManager	Controller ingestione demo governato dall'operatore
DatabaseGovernor	Controller integrità database Tier 1/2/3
MLController	Supervisore ciclo di vita ML training

Sequenza di boot:



Cache di stato:

Cache	TTL	Contenuto
_baseline_cache	60s	Stato baseline temporale (stat_cards count, mode)
_training_data_cache	120s	Progresso demo (pro/user processed, on disk, trained_on)

Shutdown graceful: `shutdown()` → ferma hunter → ferma ingestion → ferma ML training → attende max 5s per conferma → log warning se timeout.

Correzione M-08 — Validazione file impostazioni: `load_user_settings()` valida la struttura JSON al caricamento: verifica che il payload sia un `dict` (non una lista o un tipo primitivo), logga un errore dettagliato se la struttura è invalida, e applica un fallback graceful ai valori predefiniti. Questo impedisce che un file `settings.json` corrotto o manomesso provochi crash a cascata nel modulo di configurazione.

Correzione C-09 — Bootstrap anticipato: Le fasi di bootstrap precedenti all'inizializzazione del logger utilizzano `print()` per i messaggi diagnostici. Questo evita il `NameError` che si verificava quando il codice tentava di chiamare `logger.info()` prima che il logger fosse stato configurato.

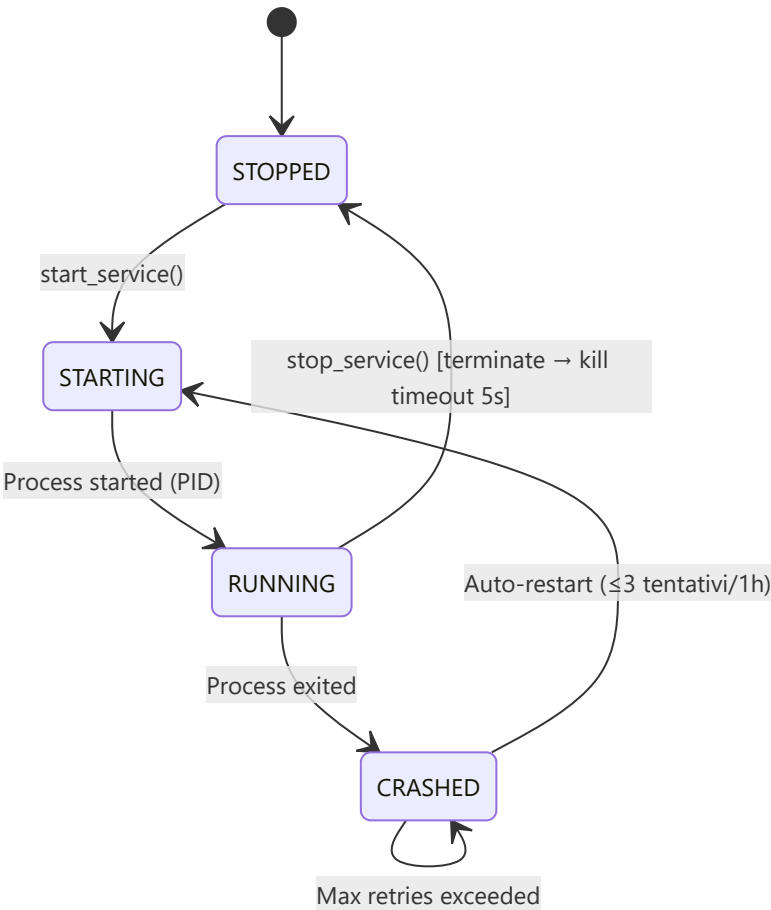
Aggregazione health report: `get_system_status()` → timestamp, state, services, teacher, ml_controller, ingestion, storage, baseline, training_data. Ogni sottosistema è isolato con `_safe_call()` — un errore in un sottosistema non impedisce il report degli altri.

-ServiceSupervisor (`console.py` , classe interna)

Gestore autorevole per daemon di background con auto-restart:

Costante	Valore	Descrizione
<code>_MAX_RETRIES</code>	3	Max auto-restart prima di arrendersi
<code>_RETRY_RESET_WINDOW_S</code>	3600 (1h)	Finestra reset contatore retry
<code>_RESTART_DELAY_S</code>	5.0	Delay tra tentativi di restart

Ciclo di vita servizio:



Servizi gestiti: Attualmente solo `hunter` (HLTV sync daemon: `hltv_sync_service.py`, processo separato). Da non confondere con i 4 daemon-thread del Session Engine (Scanner/Digester/Teacher/Pulse) — lo Scanner riporta il proprio stato sotto la chiave storica `hunter` in `CoachState`.

-DatabaseGovernor (`db_governor.py`)

Controller autorevole per Database Tier 1, 2 e 3:

Metodo	Scopo
<code>audit_storage()</code>	Calcola dimensione monolith + WAL + SHM, conta match Tier 3, rileva anomalie
<code>verify_integrity(full=False)</code>	Lightweight: <code>SELECT 1</code> ; Full: <code>PRAGMA quick_check</code> (lento su DB grandi)
<code>prune_match_data(match_id)</code>	Cancellazione privilegiata dati telemetria match
<code>rebuild_indexes()</code>	Manutenzione: <code>REINDEX</code> via ORM engine

Anomalie rilevate:

- `CRITICAL: Monolith database.db not found!`
- `CRITICAL: hltv_metadata.db missing and no backup found.`
- `WARNING: hltv_metadata.db missing, but .bak exists. Restore required.`

Tier Architecture:

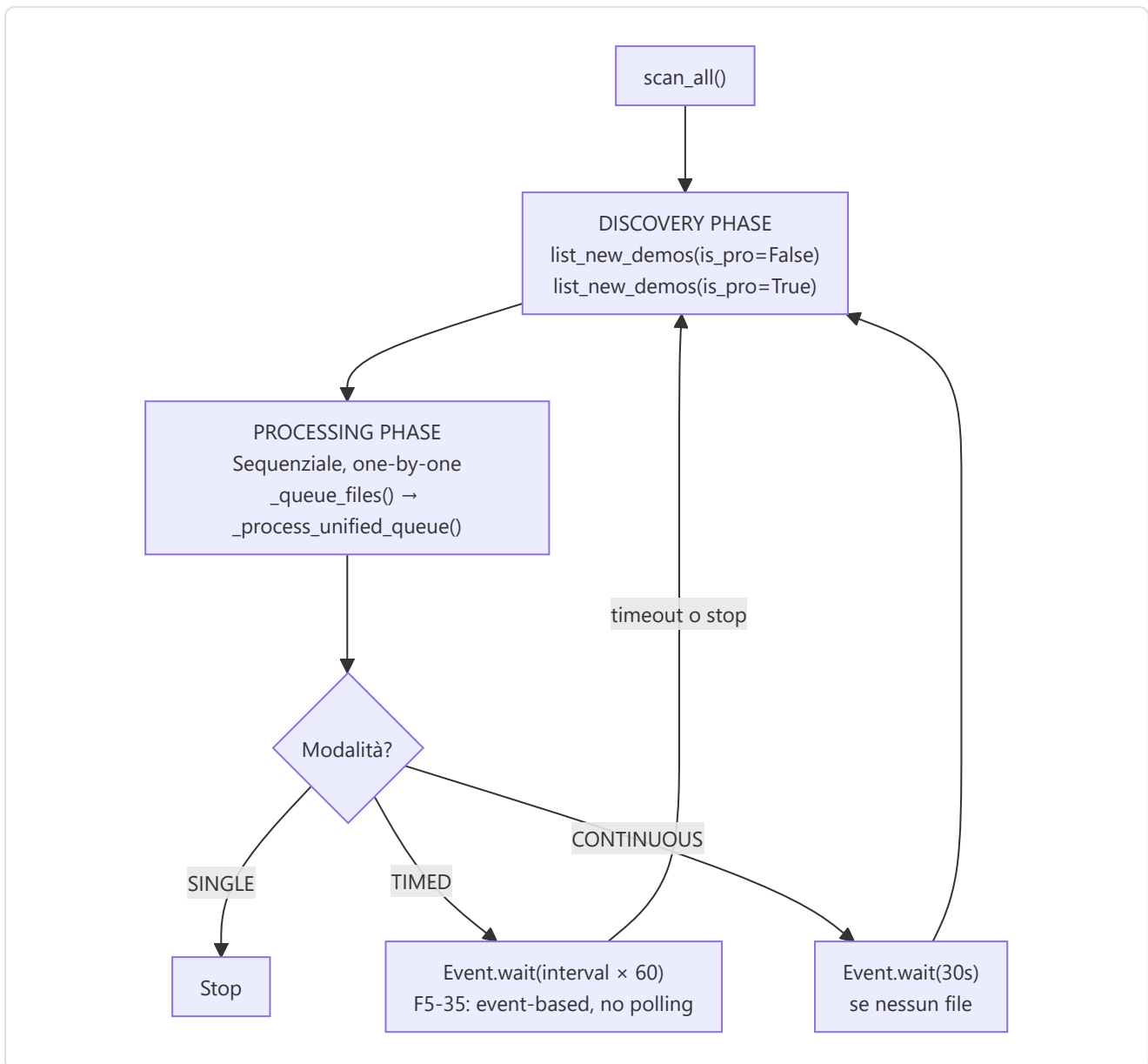
Tier	Database	Contenuto
Tier 1	<code>database.db</code> (monolith)	Giocatori, insight, profili, modelli, knowledge base
Tier 2	<code>hlvtv_metadata.db</code>	Metadati HLTv, ProPlayer, ProPlayerStatCard
Tier 3	<code>match_*.db</code> (per-match)	Dati telemetria tick-by-tick per partita

-IngestionManager (`ingest_manager.py`)

Controller di ingestione demo con 3 modalità operative:

Modalità	Comportamento
<code>SINGLE</code>	Processa una demo e si ferma
<code>CONTINUOUS</code>	Re-scan immediato, pausa 30s se nessun file trovato
<code>TIMED</code>	Re-scan ogni N minuti (default 30)

Ciclo unificato:



Segnale di stop (F5-35): `threading.Event()` per stop non-bloccante — elimina il polling a 1 secondo nelle wait loop.

Integrazione StateManager: Progresso parsing in tempo reale (0-100%) esposto via `state_manager.update_parsing_progress()`.

-MLController (`ml_controller.py`)

Supervisore del ciclo di vita ML con intervento in tempo reale:

Classe	Responsabilità
<code>MLControlContext</code>	Token di controllo passato ai loop ML
<code>MLController</code>	Supervisore thread-safe per training

MLControlContext — Comandi operatore:

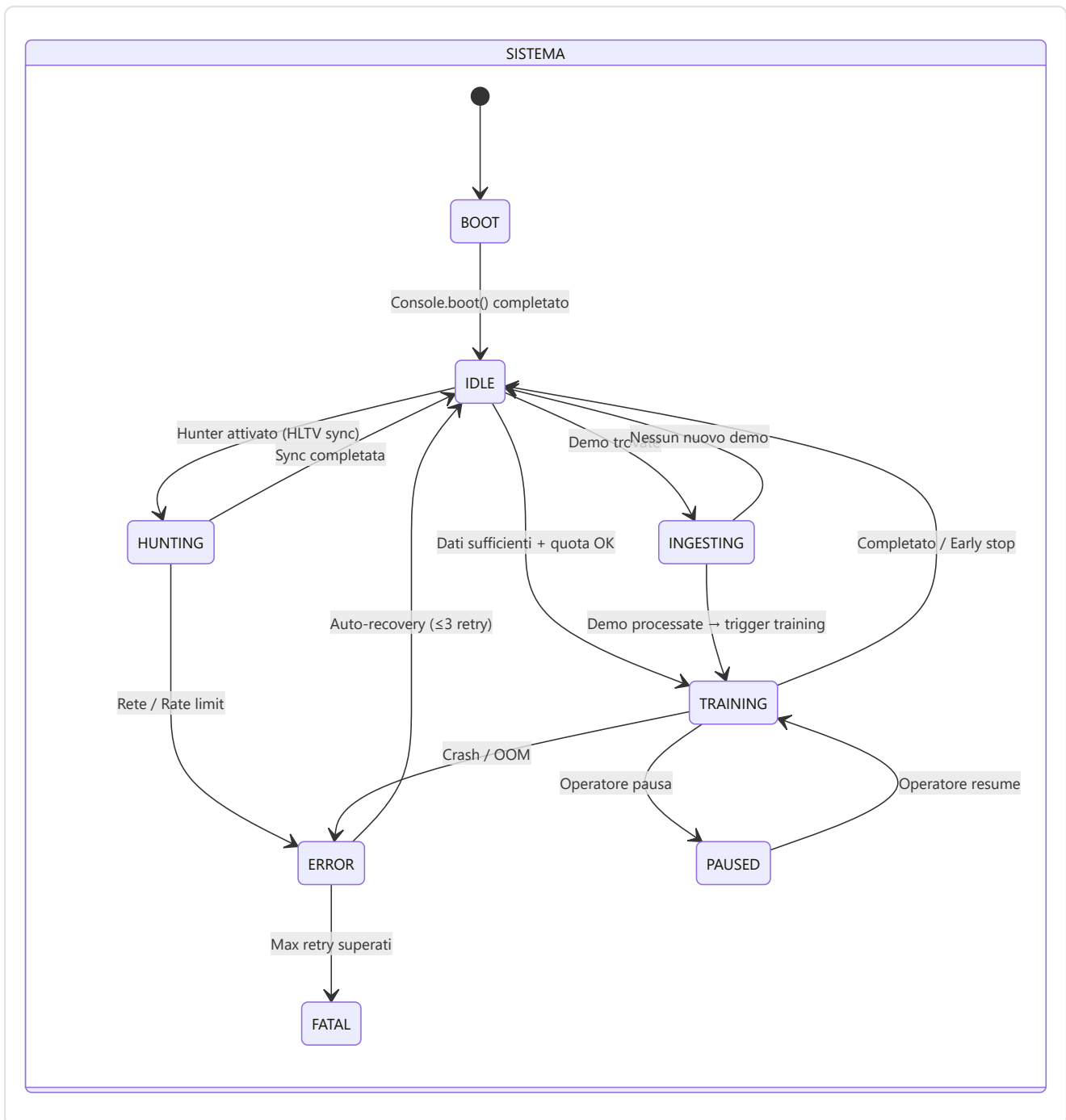
Metodo	Effetto
<code>check_state()</code>	Blocca se pause attivo (F5-15 event-based), lancia <code>TrainingStopRequested</code> se stop
<code>request_stop()</code>	Soft-stop al prossimo checkpoint sicuro
<code>request_pause()</code>	Blocca <code>check_state()</code> via <code>_resume_event.clear()</code>
<code>request_resume()</code>	Sblocca via <code>_resume_event.set()</code>
<code>set_throttle(value)</code>	0.0 = full speed, 1.0 = max delay

Pipeline: `start_training()` → thread daemon →

`CoachTrainingManager.run_full_cycle(context=self.context)` → `TrainingStopRequested` catturata per stop graceful → `set_error()` per crash.

Coordinamento Inter-Daemon

Il Modulo di Controllo orchestra i 4 daemon del sistema (Scanner, Digester, Teacher, Pulse) attraverso canali di comunicazione basati su stato condiviso (`CoachState`) e segnali event-based:



10. Sottosistema 8 — Progresso e Tendenze

Cartella nella repo: `backend/progress/` **File:** 2 moduli

Sottosistema leggero dedicato al tracciamento delle **traiettorie di performance** nel tempo.

-FeatureTrend (`longitudinal.py`)

Dataclass minimale che rappresenta il trend di una singola feature:

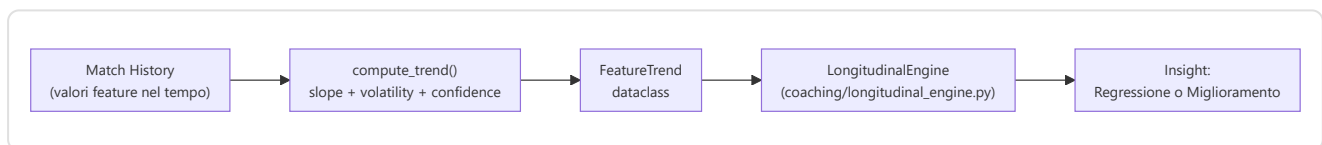
Campo	Tipo	Descrizione
<code>feature</code>	str	Nome della feature (es. "avg_adr")
<code>slope</code>	float	Pendenza della regressione lineare (positivo = miglioramento)
<code>volatility</code>	float	Deviazione standard dei valori (stabilità)
<code>confidence</code>	float	$\min(1.0, \text{num_samples} / 30)$ — richiede 30+ partite per piena fiducia

-compute_trend (trend_analysis.py)

La funzione `compute_trend(values)` — calcola slope, volatility e confidence da una serie di valori (non esiste una classe `TrendAnalysis`; il modulo espone questa funzione libera):

- **Slope:** Coefficiente lineare via `np.polyfit(x, y, 1)[0]`
- **Volatility:** `np.std(values)`
- **Confidence:** `min(1.0, n / TREND_CONFIDENCE_SAMPLE_SIZE)` con `TREND_CONFIDENCE_SAMPLE_SIZE = 30`; guardia per `n < 2`

Integrazione: I risultati di `compute_trend()` alimentano `FeatureTrend` → `LongitudinalEngine.generate_longitudinal_coaching()` → insight trend-aware nel coaching.



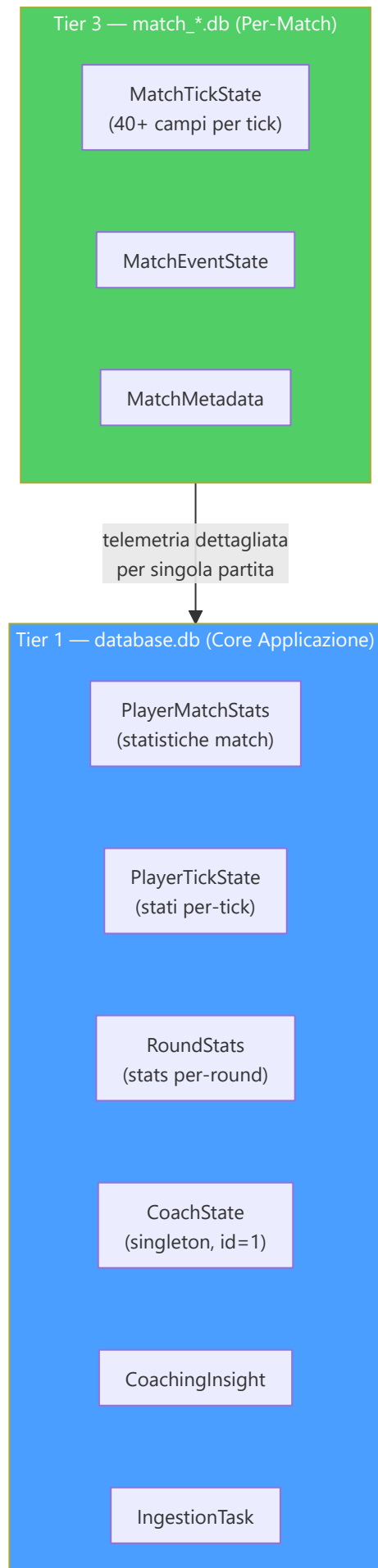
11. Sottosistema 9 — Database e Storage

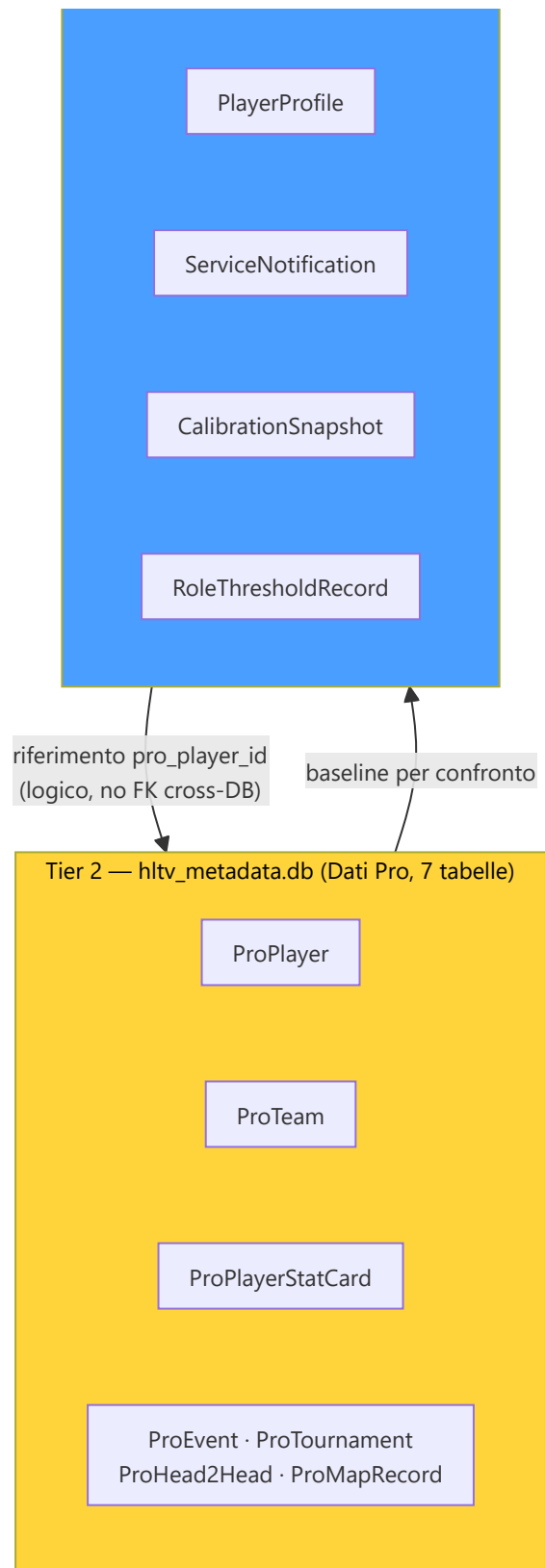
Cartella nella repo: `backend/storage/` **File:** 10 moduli

Questo sottosistema è il **sistema di archiviazione permanente** dell'intero progetto: ogni dato — dalle statistiche di una partita ai modelli addestrati, dalle esperienze di coaching ai profili dei giocatori professionisti — viene salvato, protetto, versionato e reso disponibile attraverso questa infrastruttura.

Architettura Tri-Database:







Spiegazione Diagramma: I tre database sono fisicamente separati: Tier 1 contiene i dati core dell'applicazione (18 tabelle in `_MONOLITH_TABLES`), Tier 2 i dati professionali scaricati da HLTV (7 tabelle in `_HLTV_TABLES`), e Tier 3 i dati di telemetria per-match in file SQLite individuali (3 tabelle). La separazione evita contesa WAL: le scritture di ingestione (Tier 1) non bloccano le letture HLTV (Tier 2) né la registrazione di telemetria (Tier 3). I riferimenti cross-database sono **logici** (non FK reali) poiché SQLite non supporta FK cross-file.

-Modelli di Dati (`db_models.py`)

Lo **schema canonico** di tutto il sistema: definisce ogni tabella, vincolo, indice e validatore tramite SQLAlchemy (Pydantic + SQLAlchemy). Due enum e **25 modelli** `table=True` in `db_models.py` (+ 3 per-match in `match_data_manager.py` = 28 totali), organizzati per tier.

Enum di integrità:

Enum	Valori	Uso
<code>DatasetSplit</code>	<code>TRAIN</code> , <code>VAL</code> , <code>TEST</code> , <code>UNASSIGNED</code>	Split ML nei <code>PlayerMatchStats</code>
<code>CoachStatus</code>	<code>Paused</code> , <code>Training</code> , <code>Idle</code> , <code>Error</code>	Stato pipeline ML nel <code>CoachState</code>

Costante di sicurezza: `MAX_GAME_STATE_JSON_BYTES = 16.384` (16 KB) — cap per `game_state_json` in `CoachingExperience`, validato da `@field_validator` Pydantic. Previene crescita illimitata del DB da snapshot di stato troppo grandi.

Catalogo dei modelli (Tier 1 — `database.db`):

Modello	Campi Chiave	Vincoli / Indici	Scopo
PlayerMatchStats	40+ campi: kills, ADR, rating, HLTV 2.0 components, trade metrics, utility breakdown	UNIQUE(demo_name, player_name), CHECK(avg_kills≥0), CHECK(0≤ratings≤5.0)	Statistiche aggregate per partita per giocatore
PlayerTickState	pos_x/y/z, view_x/y, health, armor, weapon, enemies_visible, round context	INDEX(demo_name, tick), INDEX(player_name, demo_name) (P2-05)	Stato giocatore per-tick per addestramento ML
PlayerProfile	player_name (unique), bio, role, profile_pic_path	UNIQUE(player_name)	Profilo utente con ruolo e bio
Ext_TeamRoundStats	equipment_value, damage, kills, deaths, accuracy	INDEX(match_id), INDEX(map_name)	Statistiche round da CSV esterni (tornei)
Ext_PlayerPlaystyle	6 probabilità ruolo, metriche aggregate, config utente (social, specs, cfg)	UNIQUE(steam_id)	Dati playstyle + profilo utente (tabella conflata, candidata a split futuro)
CoachingInsight	title, severity, message, focus_area, user_id	INDEX(player_name), INDEX(demo_name)	Feedback di coaching generati dal sistema
IngestionTask	demo_path (unique), status, retry_count, error_message	UNIQUE(demo_path), INDEX(status), R2-05: auto-refresh updated_at	Coda di ingestione demo con retry tracking
TacticalKnowledge	title, description, category, situation, embedding (384-dim JSON)	INDEX(title), INDEX(category), INDEX(map_name)	Base conoscenza RAG per coaching tattico
CoachState	status, daemon tracking (3 daemon), epoch/loss, heartbeat, resource limits	CHECK(id=1) singleton (P2-01)	Stato pipeline ML per la GUI — esattamente 1 riga
ServiceNotification	daemon, severity, message, is_read	INDEX(daemon), INDEX(is_read)	Coda notifiche errori/eventi per la UI
RoundStats	kills, deaths, assists, damage, utility, economy, round_won, round_rating	INDEX(demo_name, player_name), INDEX(demo_name, round_number) (P2-05)	Isolamento statistiche per-round per-giocatore
CalibrationSnapshot	calibration_type, parameters_json, sample_count, source	INDEX(calibration_type)	Storico calibrazione modello bayesiano
RoleThresholdRecord	stat_name (unique), value, sample_count, source	UNIQUE(stat_name)	Soglie ruolo apprese da dati pro

Modelli di match e pro (cross-tier):

Modello	Tier	Campi Chiave	Scopo
<code>MatchResult</code>	Tier 1	match_id (PK), team_a/b, winner, event_name	Metadati match
<code>MapVeto</code>	Tier 1	match_id (FK), map_name, action (pick/ban)	Veti mappe
<code>CoachingExperience</code>	Tier 1	context_hash, map/phase/side, game_state_json (16KB cap), action, outcome, embedding, effectiveness_score, feedback loop fields	Banca esperienze COPER con semantic search e feedback loop
<code>ProTeam</code>	Tier 2	hlvtv_id (unique), name, world_rank	Team professionistici
<code>ProPlayer</code>	Tier 2	hlvtv_id (unique), nickname, team_id (FK→ProTeam)	Giocatori professionisti
<code>ProPlayerStatCard</code>	Tier 2	player_id (FK), rating_2_0, kpr, adr, kast, detailed_stats_json	Schede statistiche HLTV con dati granulari
<code>ProEvent</code> / <code>ProTournament</code> / <code>ProHead2Head</code> / <code>ProMapRecord</code>	Tier 2	metadati eventi, tornei, scontri diretti e record per mappa	Tabelle HLTV estese
<code>DataLineage</code>	Tier 1	entity_type, entity_id, source_demo, pipeline_version, processing_step	Audit trail di provenienza append-only (DL-1); scritto via <code>DatabaseManager.record_lineage()</code>
<code>DataQualityMetric</code>	Tier 1	run_id, run_type, metric_name, metric_value, sample_count	Metriche qualità append-only per run (P5-E)

Nota architetturale: `PlayerMatchStats.pro_player_id` è un riferimento **logico** (non una FK reale) a `ProPlayer.hlvtv_id` perché risiedono in database separati. SQLite non supporta FK cross-file — il join viene effettuato a livello applicativo.

Nota (P2-01): Il vincolo `CHECK(id=1)` su `CoachState` garantisce che esista esattamente una riga nel database, trasformandolo in un singleton persistente. Qualsiasi tentativo di inserire una seconda riga fallisce a livello di database, non solo a livello applicativo.

-DatabaseManager (`database.py`)

Il **custode dell'archivio**: gestisce le connessioni SQLite con WAL mode obbligatorio, separando fisicamente il database monolite (Tier 1) dal database HLTV (Tier 2).

Due classi, due database:

Classe	Database	Tabelle	Engine
DatabaseManager	database.db	18 tabelle (<code>_MONOLITH_TABLES</code>)	<code>pool_size=1</code> , <code>max_overflow=4</code>
HLTVDatabaseManager	hltv_metadata.db	7 tabelle (<code>_HLTV_TABLES</code> : ProPlayer, ProTeam, ProPlayerStatCard, ProEvent, ProTournament, ProHead2Head, ProMapRecord)	<code>pool_size=1</code> , <code>max_overflow=4</code>

PRAGMA SQLite (applicati su ogni connessione via `@event.listens_for`):

PRAGMA	Valore	Scopo
<code>journal_mode</code>	<code>WAL</code>	Letture concorrenti + singola scrittura
<code>synchronous</code>	<code>NORMAL</code>	Bilanciamento performance/durabilità
<code>busy_timeout</code>	<code>30000</code> (30s)	Timeout contesa WAL lock
<code>foreign_keys</code>	<code>ON</code>	Integrità referenziale (DB-06)
<code>wal_autocheckpoint</code>	<code>512</code>	Checkpoint WAL automatico (DB-07)

API pubblica:

Metodo	Descrizione
<code>create_db_and_tables()</code>	Crea schema nel monolite (solo <code>_MONOLITH_TABLES</code> , non tutte le tabelle SQLAlchemy)
<code>get_session()</code>	Context manager transazionale: auto-commit su successo, auto-rollback su eccezione
<code>upsert(model_instance)</code>	Upsert atomico; gestione speciale per <code>PlayerMatchStats</code> (lookup per <code>demo_name</code> + <code>player_name</code>)
<code>get(model_class, pk)</code>	Lettura per chiave primaria

Singleton: `get_db_manager()` e `get_hltv_db_manager()` con double-checked locking (pattern identico). Evita la creazione di engine duplicati durante l'import da thread multipli.

Isolamento tabelle (critico): La lista esplicita `_MONOLITH_TABLES` impedisce che i modelli per-match (`MatchTickState` , `MatchEventState` , `MatchMetadata`) vengano creati nel database monolite se i loro moduli sono importati prima di `create_db_and_tables()` . Senza questa guardia, ogni modello SQLAlchemy globalmente registrato verrebbe creato in `database.db` .

Nota sulla riconciliazione HLTV: `HLTVDatabaseManager._reconcile_stale_schema()` rileva colonne mancanti in tabelle HLTV esistenti (schema aggiornato ma DB vecchio) e ricrea le tabelle interessate tramite `DROP TABLE` + `CREATE TABLE` , operando come una migrazione distruttiva di emergenza — accettabile perché i dati HLTV sono ri-scaricabili.

-MatchDataManager (`match_data_manager.py`)

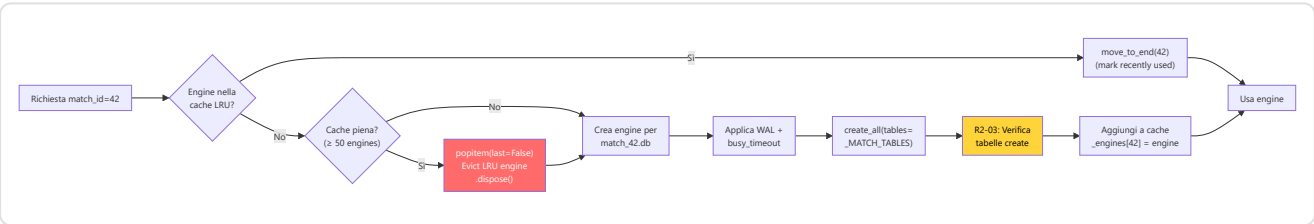
Il **sistema di telecamere di sicurezza** del progetto: ogni partita riceve il suo file SQLite dedicato (`match_{id}.db`) contenente la telemetria tick-by-tick — fino a **1,7 milioni di righe per partita**.

3 modelli per-match (Tier 3):

Modello	Campi	Scopo
<code>MatchTickState</code>	40+ campi: posizione, stato, equipaggiamento, visibility, round context, cumulativi match	Stato giocatore ad ogni tick (128 Hz)
<code>MatchEventState</code>	tick, event_type, player/victim info, position, weapon, damage, entity_id	Eventi di gioco per ricostruzione Player-POV
<code>MatchMetadata</code>	match_id, demo_name, map_name, tick/round/player count, team names/scores	Metadati per accesso senza DB monolite

Nota (C-08): `MatchEventState.entity_id` usa `-1` come sentinel (non `0`) per distinguere "ID non popolato dal parser" da "entity ID reale = 0", evitando accoppiamenti errati fumo/molotov.

Cache LRU (M-18):



Nota (R2-03): Il filtro `tables=_MATCH_TABLES` nel `create_all()` è **critico**. Senza di esso, `SQLModel` creerebbe tutte le ~20 tabelle globali in ogni file per-match. Un controllo difensivo post-creazione verifica che solo le 3 tabelle attese esistano nel DB, loggando un warning se compaiono tabelle inaspettate.

API pubblica:

Metodo	Descrizione
<code>get_match_session(match_id)</code>	Context manager transazionale per un match specifico
<code>store_tick_batch(match_id, ticks)</code>	Inserimento batch di <code>MatchTickState</code> — ritorna conteggio
<code>store_metadata(match_id, metadata)</code>	Upsert metadati match
<code>get_ticks_for_round(match_id, round)</code>	Query tick per round specifico
<code>get_player_ticks(match_id, player, start, end)</code>	Finestra tick per giocatore + intervallo
<code>get_all_players_at_tick(match_id, tick)</code>	Snapshot tutti i giocatori a un tick
<code>get_all_players_tick_window(match_id, start, end)</code>	Finestra tick per tutti i giocatori (addestramento RAP)

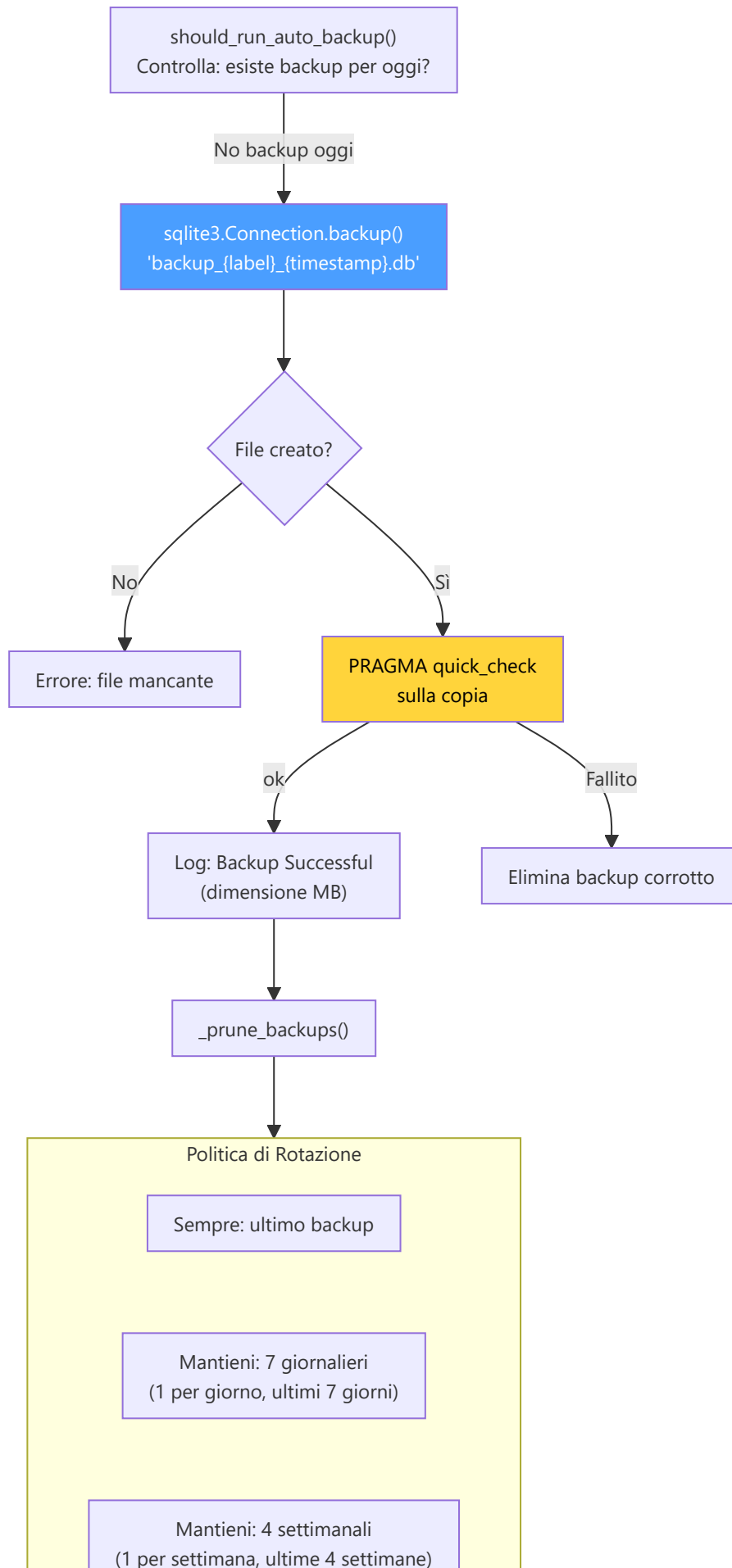
-BackupManager (`backup_manager.py`)

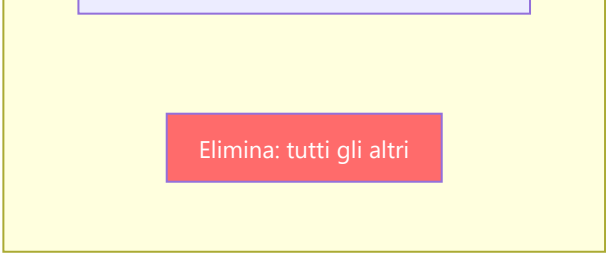
Il **responsabile della sicurezza dei dati**: crea copie di backup a caldo tramite la **SQLite Online Backup API** (`sqlite3.Connection.backup()` — scelta esplicitamente al posto di `VACUUM INTO` per evitare interpolazione del path in SQL; copia incrementale sicura in WAL-mode senza fermare i writer) e le gestisce con una politica di rotazione. Il modulo complementare `db_backup.py` usa la stessa API in modo atomico (WR-33, anti-TOCTOU) e fornisce `backup_monolith`, `backup_match_data`, `rotate_backups` (keep_count=5) e `restore_backup`.

Correzione H-02 — DB handle leak: `_verify_integrity()` utilizza un blocco `try/finally` per garantire la chiusura della connessione `sqlite3` anche in caso di errore. Il PRAGMA utilizzato è `quick_check` (non `integrity_check`) per ridurre il tempo di verifica sui database di grandi dimensioni — `quick_check` omette la validazione degli indici, sufficiente per verificare l'integrità strutturale delle pagine del backup.

Pipeline di backup:







Elimina: tutti gli altri

Sicurezza path: Il label del backup viene pre-validato con regex (BE-01, DB-03) e il path di destinazione viene verificato per garantire che risieda all'interno della directory di backup — difesa in profondità (BE-06) contro path traversal.

-StorageManager (`storage_manager.py`)

Gestore dello **storage locale** per demo CS2: cartelle di ingestione, archivio post-elaborazione, e quote disco.

Funzionalità	Dettaglio
Cartella ingestione	Path configurabile (<code>DEFAULT_DEMO_PATH</code>), fallback a <code>data/</code> in-project
Archivio	<code>datasets/user_archive/</code> e <code>datasets/pro_archive/</code> nel "Brain Root"
Quote	<code>MAX_DEMOS_PER_MONTH</code> e <code>MAX_TOTAL_DEMOS_PER_USER</code> da config
Quota disco	<code>LOCAL_QUOTA_GB</code> (default 10 GB), verificata prima dell'archiviazione
Discovery	<code>list_new_demos(is_pro)</code> — cerca <code>.dem</code> non ancora elaborati in <code>PRO_DEMO_PATH</code> o <code>DEFAULT_DEMO_PATH</code>

-StateManager (`state_manager.py`)

DAO centralizzato per il singleton `CoachState` : interfaccia thread-safe tra i daemon e la GUI.

Metodo	Descrizione
<code>get_state()</code>	Recupera il singleton (lo crea se non esiste, lock P0-04)
<code>update_status(daemon, status, detail)</code>	Aggiorna stato di un daemon specifico (hunter/digester/teacher/global)
<code>update_training_progress(epoch, total, train_loss, val_loss, eta)</code>	Progresso real-time addestramento per la GUI
<code>update_parsing_progress(progress)</code>	Barra di progresso parsing demo (0.0–100.0)
<code>push_notification(daemon, severity, message)</code>	Inserisce una <code>ServiceNotification</code> nella coda per la UI
<code>get_unread_notifications(limit)</code>	Recupera notifiche non lette per visualizzazione

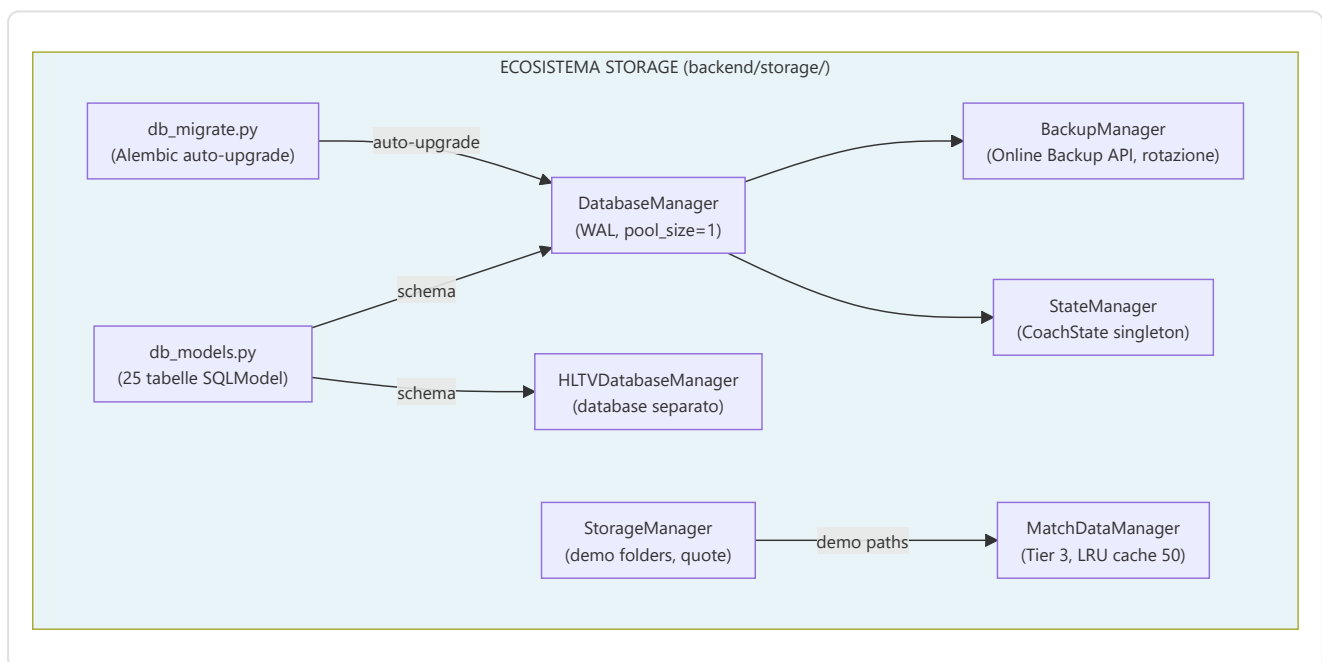
-Servizi Storage Aggiuntivi

stat_aggregator.py — Persistenza dati spider HLTV: `persist_player_card(card_data, player_hltv_id)` converte i dati estratti dallo spider in un `ProPlayerStatCard` e lo salva nel database HLTV. `persist_team(team_data)` salva/aggiorna un `ProTeam`.

db_migrate.py — Orchestrazione migrazioni Alembic: `ensure_database_current()` viene chiamata all'avvio dell'applicazione per auto-aggiornare lo schema. Confronta `current_rev` vs `head_rev` e se diversi esegue `alembic.command.upgrade(cfg, "head")`. Se `alembic.ini` non esiste (sviluppo senza Alembic), ritorna `True` silenziosamente.

maintenance.py — Manutenzione database: `prune_old_metadata(days=90)` elimina record vecchi in batch da 500 (`SQLITE_MAX_VARIABLE_NUMBER` safety), evitando query DELETE con migliaia di parametri che SQLite rifiuterebbe.

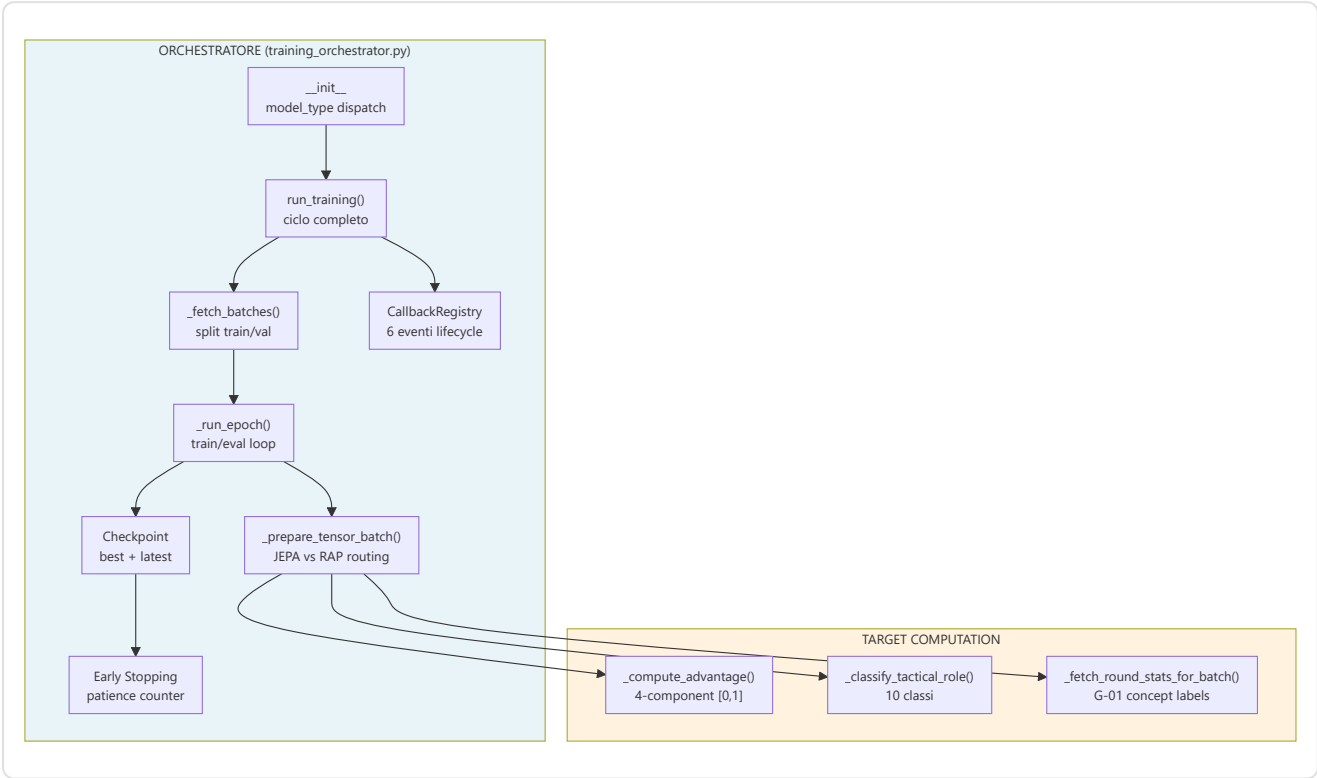
remote_file_server.py — Server di cloud storage personale costruito su FastAPI per accesso remoto ai dati dell'applicazione. Implementa autenticazione tramite API key con confronto HMAC-safe (`hmac.compare_digest`) per prevenire timing attacks, rate limiting a finestra scorrevole (10 req/min per IP) tramite middleware ASGI, e protezione path traversal su tutti gli endpoint di accesso file. **Sicurezza BE-07:** rifiuta il binding su indirizzi non-localhost senza TLS — se vengono forniti certificato e chiave, serve su HTTPS via uvicorn; altrimenti, si limita a `localhost`. Endpoint: `GET /list` (lista file con auth), `GET /download/{filename}` (download con auth e protezione path traversal), `GET /health` (health check senza auth).



12. Pipeline di Addestramento e Orchestrazione

Il sottosistema di addestramento ha tre responsabilità distinte:

- 1. **Orchestrazione** — Ciclo di vita completo: caricamento dati → epoche → checkpoint → arresto anticipato
- 2. **Preparazione Tensori** — Conversione tick grezzi dal database in batch tensoriali pronti per il modello
- 3. **Calcolo Target** — Generazione di etichette di addestramento: funzione vantaggio (continua) e ruolo tattico (10 classi)



-TrainingOrchestrator (training_orchestrator.py)

La classe centrale della pipeline di addestramento. Gestisce JEPA, VL-JEPA e RAP da un punto di ingresso unificato, con dispatch basato su `model_type`.

Costruttore e Configurazione:

Parametro	Default	Descrizione
<code>model_type</code>	<code>"jepa"</code>	Routing: <code>"jepa"</code> → JEPATrainer, <code>"vl-jepa"</code> → JEPATrainer (VL mode), <code>"rap"</code> → RAPTrainer
<code>max_epochs</code>	100	Limite superiore epoche
<code>patience</code>	10	Epoche senza miglioramento prima di early stopping
<code>batch_size</code>	32	Campioni per batch
<code>callbacks</code>	<code>CallbackRegistry()</code>	Sistema plugin per TensorBoard, logging, etc.

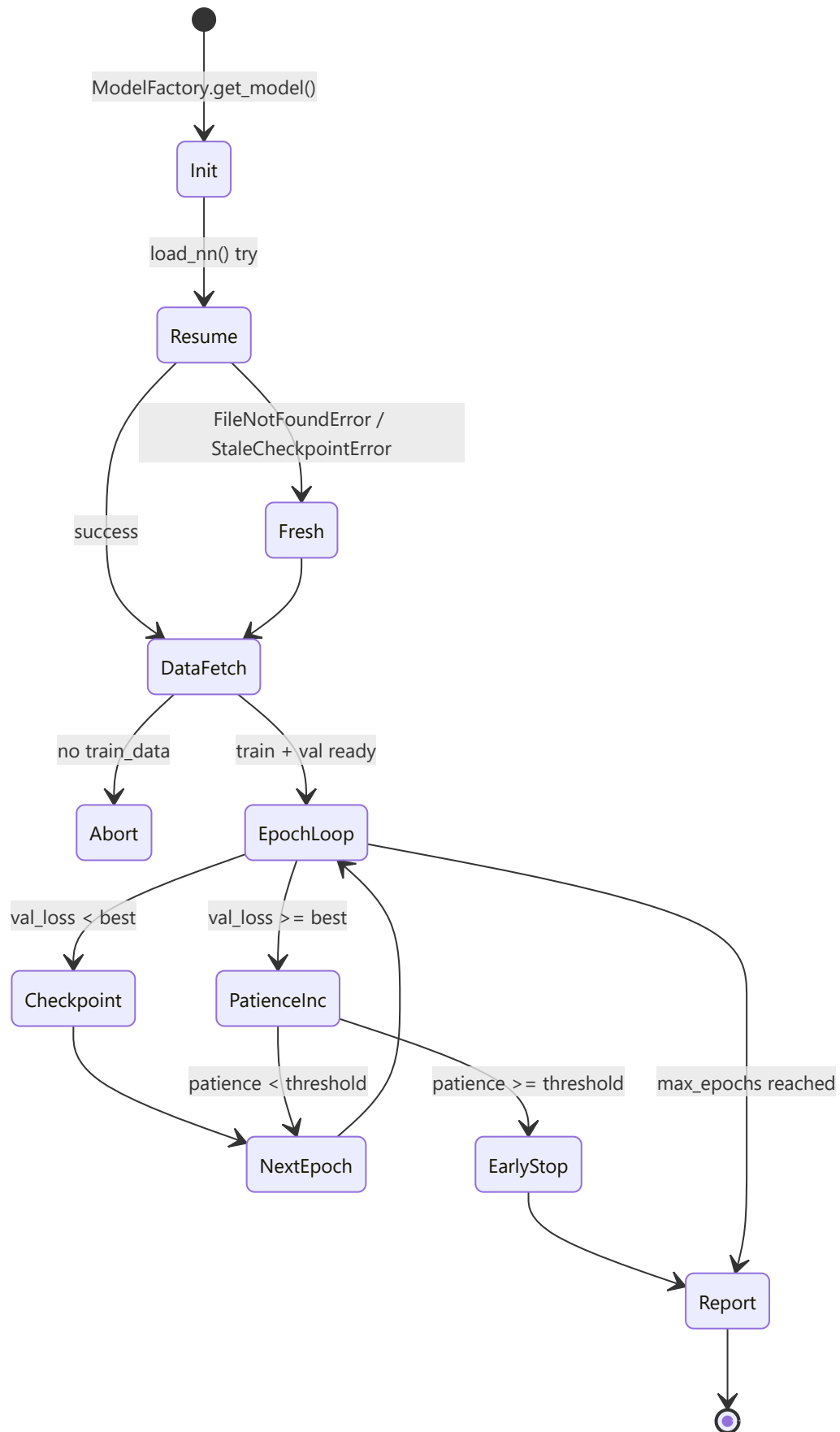
Dettagli notevoli del costruttore:

- **RNG deterministico** (F3-02): `np.random.default_rng(seed=42)` per il campionamento negativi JEPA — garantisce riproducibilità

- **Contatori fallback** (F3-11): `_total_samples` e `_total_fallbacks` tracciano il tasso aggregato di tensori-zero nell'intero ciclo
- **RAP gated**: Il modello RAP richiede `USE_RAP_MODEL=True` nelle impostazioni — se disabilitato, `ValueError` immediato
- **Learning rate**: JEPA 1e-4, RAP 5e-5 (più conservativo per il multi-task)

Ciclo di Vita: `run_training()`

Il metodo principale esegue 6 fasi sequenziali:



1. **Init:** `ModelFactory.get_model(type)` istanzia il modello, poi tenta `load_nn()` per riprendere da checkpoint esistente. Gestisce 3 eccezioni: `FileNotFoundError` (primo addestramento), `StaleCheckpointError` (architettura cambiata), e generico (corruzione)
2. **Data Fetch:** `_fetch_batches(is_train=True/False)` con split train/val
3. **Epoch Loop:** Per ogni epoca, `_run_epoch()` in modalità train e poi eval. Il `context.check_state()` permette interruzione cooperativa (TrainingStopRequested)
4. **Checkpoint:** Doppio salvataggio — `model_name` (best) e `model_name_latest` (sempre). Solo il best viene salvato quando `val_loss` migliora
5. **Early Stopping:** Contatore `patience_counter` incrementato ad ogni epoca senza miglioramento
6. **Report:** Callback `on_train_end` con metriche finali + log del tasso di fallback F3-11

Preparazione Dati: `_fetch_batches()`

Recupera tick dal database tramite il manager, rispettando lo split train/val:

- Chiama `manager._fetch_jepa_ticks(is_pro, split)` per tutti i tipi di modello (RAP incluso — pipeline dati RAP-specifica non ancora implementata, con warning esplicito)
- **Ordinamento temporale preservato** — nessuno shuffle, perché i modelli sono sequenziali
- Suddivisione in batch di dimensione `self.batch_size`

Routing Tensori: `_prepare_tensor_batch()`

Il metodo più critico: converte oggetti `PlayerTickState` dal database in dizionari di tensori PyTorch.

Flusso JEPA (context/target/negatives):

Tensore	Shape	Costruzione
<code>context</code>	<code>(1, 10, METADATA_DIM)</code>	Primi 10 tick del batch, padding se < 10
<code>target</code>	<code>(1, METADATA_DIM)</code>	Media dell'ultimo tick (next-item prediction)
<code>negatives</code>	<code>(1, 5, METADATA_DIM)</code>	5 campioni casuali dal batch (RNG deterministico)

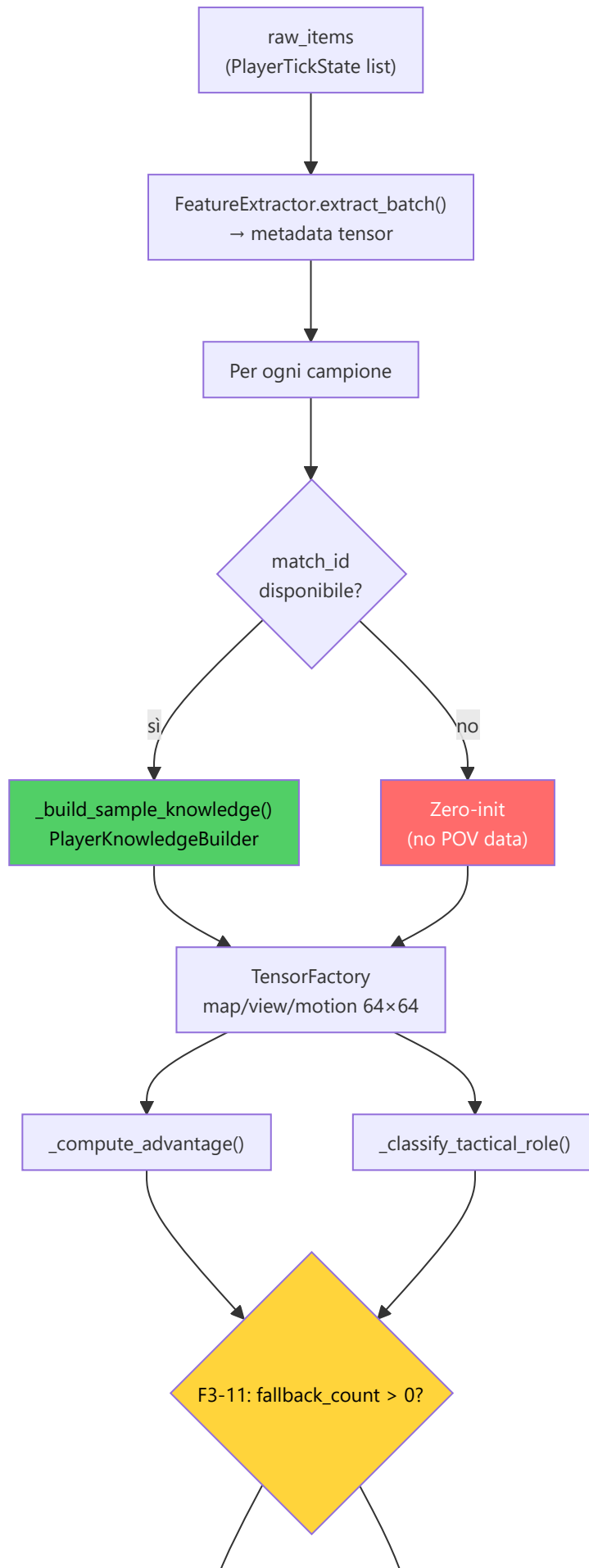
Richiede almeno 5 campioni reali per i negativi contrastivi — se il batch è troppo piccolo, ritorna `None` (skip).

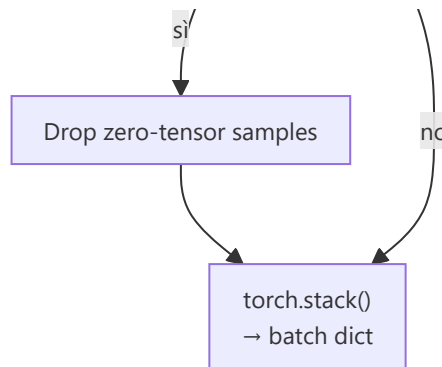
Correzione G-01: Per VL-JEPA, viene anche chiamato `_fetch_round_stats_for_batch()` per fornire etichette concettuali basate sugli esiti reali dei round (elimina label leakage da labeling euristico).

Flusso RAP (Player-POV completo):

Il percorso RAP è significativamente più complesso — delegato a `_prepare_rap_batch()`:







Per ogni campione nel batch:

1. **Risoluzione match_id** → query al `MatchDataManager` per tutti i giocatori al tick
2. **PlayerKnowledgeBuilder** → modello sensoriale NO-WALLHACK (visibilità, suono, utilità)
3. **TensorFactory** → tensori map/view/motion a 64×64 (risoluzione training)
4. **Target computation** → advantage [0,1] + ruolo tattico (10 classi)

Cache per-batch (4 dizionari) evitano query ridondanti sullo stesso match/tick:

Cache	Chiave	Valore
<code>_all_players_cache</code>	<code>(match_id, tick)</code>	Lista giocatori al tick
<code>_window_cache</code>	<code>(match_id, tick)</code>	Storico 320 tick per enemy memory
<code>_event_cache</code>	<code>(match_id, tick)</code>	Eventi nell'intervallo [-320, +64]
<code>_metadata_cache</code>	<code>match_id</code>	Metadati match (mappa, etc.)

Correzione F3-11: I campioni con fallback a zero-tensor vengono **scartati** (non usati per training). Il tasso aggregato viene tracciato lungo tutto il ciclo e, se al termine supera il **30%**, `_finalize_training()` **aborta il run** con errore esplicito. Se l'intero batch è fallback, viene saltato completamente.

Correzione H-03 — Rimozione batch swallowing silenzioso: I loop di training e validazione del RAP non contengono più il blocco `except (KeyError, TypeError): continue` che mascherava silenziosamente errori nei dati. Ora `train_step()` deve restituire un `dict` con chiave `'loss'` o viene sollevato un `ValueError` esplicito. Questo garantisce che errori strutturali nei batch vengano rilevati immediatamente anziché ignorati, migliorando la diagnosticabilità del pipeline di addestramento.

Funzione Vantaggio: `_compute_advantage()`

Formula continua [0, 1] che sostituisce il target binario vinto/perso (G-04):

```

advantage = 0.4 × alive_diff + 0.2 × hp_ratio + 0.2 × equip_ratio + 0.2 × bomb_factor

```

Componente	Peso	Calcolo	Range
<code>alive_diff</code>	0.4	<code>(team_alive - enemy_alive + 5) / 10</code>	[0, 1]
<code>hp_ratio</code>	0.2	<code>team_hp / (team_hp + enemy_hp)</code>	[0, 1]
<code>equip_ratio</code>	0.2	<code>team_equip / (team_equip + enemy_equip)</code>	[0, 1]
<code>bomb_factor</code>	0.2	Planted: T=0.7, CT=0.3 / Not planted: 0.5	{0.3, 0.5, 0.7}

L' `alive_diff` ha il peso maggiore (0.4) perché la superiorità numerica è il singolo fattore più predittivo nel CS2. La normalizzazione `(diff + 5) / 10` mappa il range [-5, +5] (5v0 → 0v5) in [0, 1].

Classificazione Ruolo Tattico: `_classify_tactical_role()`

Assegna uno di 10 ruoli tattici ad ogni tick basandosi su euristiche informate dal `PlayerKnowledge`:

Indice	Ruolo	Condizione
0	Site Take	T default (nessuna condizione speciale)
1	Rotation	— (riservato)
2	Entry Frag	Nemici visibili + distanza < 800u
3	Support	T + distanza media team < 500u + ≥2 compagni
4	Anchor	CT + accovacciato
5	Lurk	Distanza media dal team > 1500u + nessun nemico visibile
6	Retake	CT + bomba piazzata
7	Save	Equipaggiamento < \$1500 (priorità massima)
8	Aggressive Push	Nemici visibili + distanza ≥ 800u
9	Passive Hold	CT default (senza knowledge)

La catena di priorità è: Save → Retake → Lurk → Entry Frag → Aggressive Push → Anchor/Passive Hold → Support → Site Take. Senza `PlayerKnowledge`, si ricade su default basati sul team (CT → Passive Hold, T → Site Take).

Risoluzione Mappa: `_resolve_map_name()`

Catena di fallback a 3 livelli per determinare la mappa di ogni campione:

1. **Match metadata DB** → `MatchDataManager.get_metadata(match_id).map_name`
2. **Regex su demo_name** → cerca nomi mappa noti (mirage, inferno, dust2, etc.)
3. **Fallback statico** → `"de_mirage"` (mappa più giocata)

Validazione Contrastiva (Evaluation)

Durante la validazione, il calcolo della loss JEPA richiede un passaggio extra per i negativi:

1. Raw negatives `[batch, n_neg, feat_dim]` → flatten a `[batch*n_neg, feat_dim]`
2. Encode tramite `target_encoder` → `[batch*n_neg, latent_dim]`
3. Reshape a `[batch, n_neg, latent_dim]`
4. Calcolo `jepa_contrastive_loss(pred, target, neg_latent)`

Per RAP, la validazione usa `trainer.criterion_val` (MSE su `value_estimate` vs `target_val`).

-JEPATrainer (`jepa_trainer.py`)

Il trainer specializzato per l'architettura JEPA. Gestisce pre-training self-supervised, monitoraggio drift, e il protocollo VL-JEPA a tripla loss.

Costruttore:

Parametro	Default	Dettaglio
<code>lr</code>	1e-4	Learning rate per AdamW
<code>weight_decay</code>	1e-2	Regolarizzazione L2
<code>drift_threshold</code>	2.5	Soglia z-score per DriftMonitor
AMP + accumulo	GradScaler (CUDA), <code>_accumulation_steps=4</code> , grad clip 1.0	Mixed precision e batch effettivi più grandi

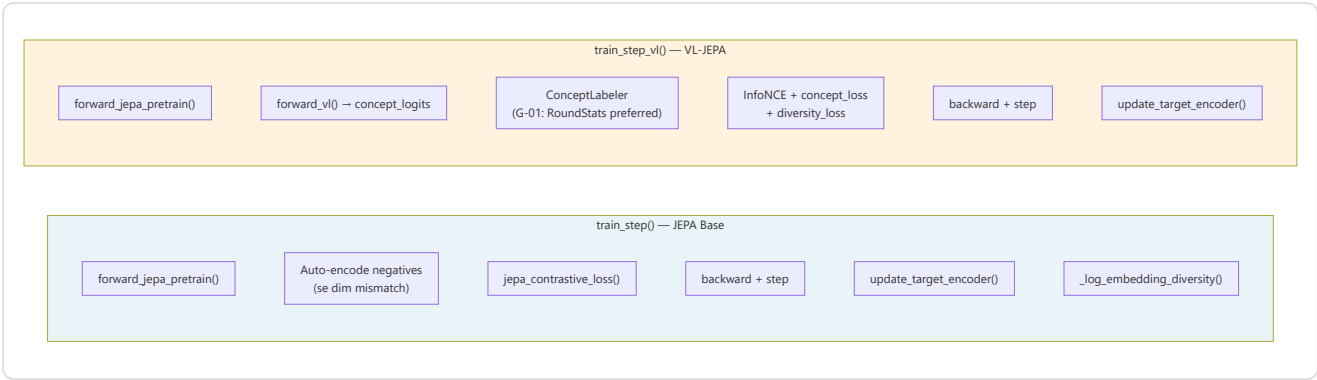
- **NN-36:** I parametri del `target_encoder` vengono **esclusi** dall'ottimizzatore — sono aggiornati solo via EMA (Exponential Moving Average), mai con gradienti diretti
- **KT-05:** I parametri dei layer `concept` ricevono un **moltiplicatore LR 0.05×** (5% del learning rate base) tramite gruppi di parametri separati nell'ottimizzatore. Questo previene il collapse delle embedding concettuali durante il training VL-JEPA (per VL-JEPA paper, Section 4.6, Assran et al.)
- **Scheduler:** `SequentialLR` — warmup lineare (5% di `T_max`, `start_factor` 0.01) seguito da `CosineAnnealingLR` con `T_max=100`, step una volta per epoca

Schedule EMA Momentum (J-6):

$$\tau(t) = 1 - (1 - \tau_{\text{base}}) \cdot (\cos(\pi t/T) + 1) / 2$$

Con `$\tau_{\text{base}} = 0.996$` , il momentum parte da 0.996 (tracking veloce del target encoder) e converge a 1.0 (target encoder congelato) durante il training. Questo schedule coseno, adattato da Assran et al. (CVPR 2023, Section 3.2), permette al target encoder di tracciare rapidamente l'online encoder nelle prime fasi, poi stabilizzarsi per fornire target consistenti nella fase avanzata dell'addestramento.

Due modalità di training step:



train_step() (JEPA base):

- 1. Forward → `pred_embedding` , `target_embedding`
- 2. Auto-detect negativi raw (dim mismatch con `pred_embedding`) → encode via `target_encoder`
- 3. `jepa_contrastive_loss(pred, target, negatives)` — InfoNCE
- 4. Backward + optimizer step
- 5. EMA update del target encoder
- 6. **P9-02**: Monitoraggio varianza embeddings — warning se < 0.01 (rischio collapse)

train_step_vl() (VL-JEPA):

Tre componenti di loss:

Loss	Peso	Funzione
InfoNCE	1.0	Contrastive self-supervised
Concept alignment	$\alpha = 0.5$	Allineamento concetti a etichette
Diversity regularization	$\beta = 0.1$	Previene concept collapse

Correzione G-01: Le etichette concettuali vengono preferibilmente generate da `ConceptLabeler.label_from_round_stats(rs)` usando i dati reali di esito round. Solo come fallback viene usato il labeling euristico (con warning logged una volta sola).

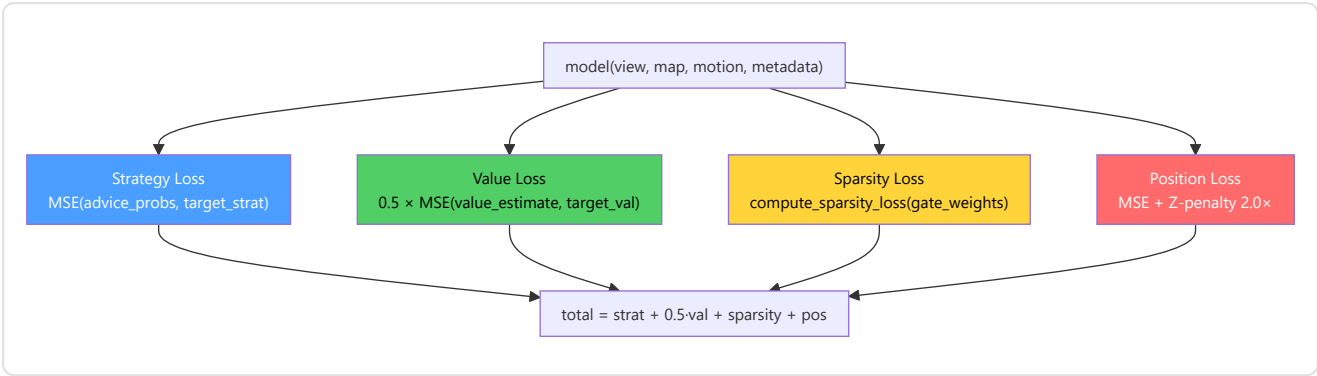
Sistema Drift (Task 2.19.3):

- `check_val_drift(val_df, reference_stats)` → `DriftMonitor.check_drift()` con z-threshold 2.5
- `should_retrain(history, window=5)` → True se drift persistente nelle ultime 5 epoche
- `retrain_if_needed()` → ciclo completo di riaddestramento con scheduler resettato

-RAPTrainer (rap_coach/trainer.py)

Il trainer multi-task per il modello RAP Coach. Gestisce 4 componenti di loss simultaneamente.

Loss Composition:



Loss	Peso	Criterio	Target
Strategy	1.0	MSE	<code>target_strat</code> (ruolo tattico one-hot, 10 classi)
Value	0.5	MSE	<code>target_val</code> (advantage [0,1])
Sparsity	1.0	Entropia del gate MoE $\times 1e-4$	Interpretabilità: routing deciso (bassa entropia)
Position	1.0	MSE + $Z \times 2.0$	<code>target_pos</code> delta (opzionale)

Ottimizzazione RAPTrainer: AdamW ($lr=1e-4$, $weight_decay=1e-2$), `CosineAnnealingLR(T_max=100, eta_min=1e-6)`, accumulo gradienti $\times 4$, AMP su CUDA, grad clip 1.0; dopo ogni optimizer step `_notify_memory_step()` attiva la memoria Hopfield.

Task 2.17.1 — Penalità Z-axis: La `compute_position_loss()` applica un peso $2\times$ sull'errore asse Z (verticalità), perché nel CS2 gli errori di elevazione (es. nemico su una scala vs piano terra) sono tatticamente critici.

F3-07: I `gate_weights` vengono passati esplicitamente a `compute_sparsity_loss()` anziché letti dall'istanza del modello — garantisce thread-safety.

-Punto di Ingresso Training (train.py)

Modulo entry-point che fornisce la funzione `train_nn()` con dispatch automatico e funzioni standalone.

`train_nn(X, y, X_val, y_val, model, config_name, context):`

config_name	Percorso	Pipeline
"jepa"	<code>_train_jepa_self_supervised()</code>	Self-supervised, InfoNCE, 5 epoche prototype
altro	<code>_execute_validated_loop()</code>	Supervised, MSE, DataLoader standard

Funzioni helper:

Funzione	Responsabilità
<code>_prepare_splits(X, y)</code>	Train/val split 80/20 con <code>random_state=42</code> . Rifiuta se < 20 campioni (P1-04)
<code>_train_jepa_self_supervised()</code>	Prototipo JEPA: <code>SelfSupervisedDataset</code> , context_len=10, prediction_len=5, clip_grad=1.0
<code>_execute_validated_loop()</code>	Loop supervisionato con <code>EarlyStopping(patience=10)</code> , throttling configurabile
<code>run_training()</code>	Standalone entry: usa <code>CoachTrainingManager</code> per fetch dati pro

Nota P1-02: Tutti i percorsi chiamano `set_global_seed()` prima dell'addestramento per garantire riproducibilità. Il seed globale è configurato nel modulo `config.py` .

-WinProbabilityTrainer (win_probability_trainer.py)

Modulo dedicato al modello di probabilità vittoria — la rete più semplice dell'ecosistema.

Architettura `WinProbabilityTrainerNN` :

```
Input(9) → Linear(32) → ReLU → Linear(16) → ReLU → Linear(1) → Sigmoid
```

9 Feature di Input:

#	Feature	Tipo
1	<code>ct_alive</code>	Giocatori CT vivi
2	<code>t_alive</code>	Giocatori T vivi
3	<code>ct_health</code>	HP totale CT
4	<code>t_health</code>	HP totale T
5	<code>ct_armor</code>	Armatura totale CT
6	<code>t_armor</code>	Armatura totale T
7	<code>ct_eqp</code>	Valore equipaggiamento CT
8	<code>t_eqp</code>	Valore equipaggiamento T
9	<code>bomb_planted</code>	Bomba piazzata (0/1)

Training:

- Loss: **BCELoss** (Binary Cross-Entropy — output è probabilità [0,1])
- Optimizer: Adam (lr=0.001) — non AdamW, più semplice per questa rete
- Early stopping: patience=10, min_delta=1e-4
- Target: `did_ct_win` (binario)
- Split: 80/20 con `random_state=42`

- Minimo campioni: 20 (AR-6)
- Salvataggio: `torch.save(state_dict, model_path)` — persistenza diretta, non tramite `save_nn()`

`predict_win_prob(model, state_dict)` — Inferenza singola: costruisce il tensore dalle 9 feature e ritorna la probabilità CT-win come float.

Nota A-12 — Architetture distinte: Questo `WinProbabilityTrainerNN` (9 feature, addestramento offline) è un modello *separato e incompatibile* rispetto al `WinProbabilityNN` a 12 feature descritto nella Sezione 7. Il primo viene addestrato sui dati aggregati delle demo; il secondo opera in tempo reale durante l'analisi live. I checkpoint non sono intercambiabili.

-Utilità di Addestramento

I seguenti moduli forniscono l'infrastruttura di supporto al ciclo di addestramento — configurazione, controllo, monitoraggio, dataset, e arresto anticipato.

`training_config.py` — Dataclass centralizzate per iperparametri:

Dataclass	Parametri Chiave	Uso
<code>TrainingConfig</code>	<code>base_lr=1e-4</code> , <code>max_epochs=100</code> , <code>patience=10</code> , <code>batch_size=1</code> , <code>gradient_clip=1.0</code> , <code>ema_decay=0.999</code>	Configurazione base per tutti i modelli
<code>JEPATrainingConfig</code>	Estende <code>TrainingConfig</code> + <code>latent_dim=256</code> , <code>contrastive_temperature=0.07</code> , <code>momentum_target=0.996</code> , <code>pretraining_epochs=50</code>	Specifica per JEPA self-supervised

`training_controller.py` — Gatekeeper che decide **se** un nuovo demo debba essere usato per addestramento:

- **Quota mensile:** `MAX_DEMOS_PER_MONTH = 10` — conta i `PlayerMatchStats` processati negli ultimi 30 giorni
- **Score di diversità:** Confronta il nuovo demo con gli ultimi 5 match via similarità coseno su 6 feature normalizzate (kills, deaths, ADR, HS%, utility, opening duels). Se `diversity < 0.3`, il demo viene rifiutato
- Ritorna un `TrainingDecision(should_train, reason, diversity_score)` — il chiamante decide se procedere

`training_monitor.py` — Persistenza metriche in JSON per monitoraggio real-time e analisi post-training. Traccia: epoche, train_loss, val_loss, learning_rate, best_val_loss, stato (in progress / early stopped / completed).

`early_stopping.py` — Implementazione callable dell'arresto anticipato:

```
stopper = EarlyStopping(patience=10, min_delta=1e-4)
if stopper(val_loss): # True = ferma l'addestramento
    break
```

Logica: se `val_loss < best_loss - min_delta`, reset del contatore. Altrimenti incrementa. Quando `counter ≥ patience`, ritorna `True`. Usato da tutti i trainer (JEPA, RAP, Legacy, WinProb).

dataset.py — Due Dataset PyTorch:

Dataset	Input	Output	Uso
<code>ProPerformanceDataset</code>	<code>(X, y)</code> tensori	<code>(x[i], y[i])</code> tuple	Training supervisionato Legacy
<code>SelfSupervisedDataset</code>	X sequenziale	<code>(context, target)</code> finestre scorrevoli	JEPA self-supervised

Il `SelfSupervisedDataset` usa una **finestra scorrevole**: `context_len=10` tick come input, `prediction_len=5` tick come target. Il numero totale di campioni è `len(X) - context_len - prediction_len`. Solleva `ValueError` se i dati sono insufficienti (F3-34).

-Sistema di Callback (`training_callbacks.py` + `tensorboard_callback.py`)

Architettura plugin a due livelli per l'osservabilità del training, senza modificare il loop principale.

Layer 1 — `training_callbacks.py`:

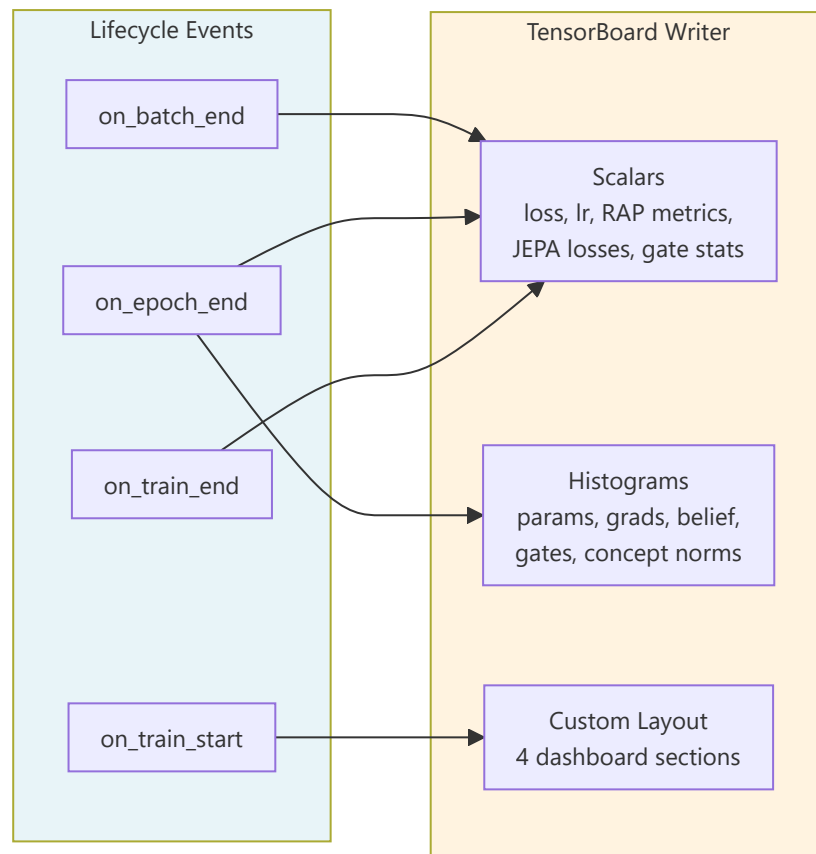
`TrainingCallback` (ABC) definisce 7 hook di lifecycle:

Hook	Quando	Parametri
<code>on_train_start</code>	Prima della prima epoca	model, config dict
<code>on_epoch_start</code>	Inizio di ogni epoca	epoch
<code>on_batch_end</code>	Dopo ogni batch di training	batch_idx, loss, outputs dict
<code>on_epoch_end</code>	Fine di ogni epoca	epoch, train_loss, val_loss, model, optimizer
<code>on_validation_end</code>	Dopo il passaggio di validazione	epoch, val_loss, model
<code>on_train_end</code>	Dopo il completamento	model, final_metrics dict
<code>close</code>	Rilascio risorse	—

Nota F3-31: Nessun metodo è `@abstractmethod` — pattern opt-in dove le sottoclassi implementano solo gli hook necessari. Tutti i metodi base sono no-op.

`CallbackRegistry` è il dispatcher: `fire(event, **kwargs)` chiama l'hook corrispondente su ogni callback registrato. Errori nei callback vengono catturati e loggati — **mai crashano il training**.

Layer 2 — `tensorboard_callback.py`:



Segnali registrati per tipo di modello:

Categoria	Metriche	Tipo
Core	<code>loss/train</code> , <code>loss/val</code> , <code>loss/gap</code> , <code>loss/batch</code>	Scalari per-epoch/batch
RAP	<code>rap/sparsity_ratio</code> , <code>rap/z_axis_error</code> , <code>rap/loss_position</code>	Scalari per-batch
JEPA	<code>jepa/inforce_loss</code> , <code>jepa/concept_loss</code> , <code>jepa/diversity_loss</code>	Scalari per-batch
Gates	<code>gates/mean_activation</code> , <code>gates/sparsity</code> , <code>gates/active_ratio</code>	Scalari per-batch
Diagnostica	<code>params/*</code> , <code>grads/*</code> , <code>belief/vector</code> , <code>concepts/embedding_norms</code> , <code>gates/activations</code>	Istogrammi per-epoch

4 dashboard custom: **Coach Vital Signs**, **RAP Coach Internals**, **JEPA Self-Supervised**, **Superposition Gates**.

Import graceful: se `tensorboard` non è installato, il callback diventa un no-op completo (`_TB_AVAILABLE = False`).

13. Funzioni di Perdita — Dettaglio Implementativo

Catalogo delle Loss Functions



-jepa_contrastive_loss() — InfoNCE

File: `jepa_model.py` | **Signature:** `(pred, target, negatives, temperature=0.07) → scalar` (in produzione la temperatura è il parametro appreso `log_temperature`, clamp [0.01, 1.0])

Formula InfoNCE (Noise Contrastive Estimation):

$$L = -\log\left(\frac{\exp(\text{sim}(\text{pred}, \text{target}))}{\tau} / \left(\frac{\exp(\text{sim}(\text{pred}, \text{target}))}{\tau} + \sum \exp(\text{sim}(\text{pred}, \text{neg}_i)) / \tau\right)\right)$$

Passaggi:

1. **Normalizzazione L2** di pred, target, negatives (cosine similarity space)
2. **Similitudine positiva:** `pos_sim = (pred · target) / τ` con `τ = 0.07`
3. **Similitudini negative:** `neg_sim = bmm(negatives, pred) / τ`
4. **Cross-entropy:** concat `[pos_sim, neg_sim] → F.cross_entropy` con labels=0 (il positivo è sempre l'indice 0)

La temperatura `τ = 0.07` è bassa, il che rende la loss **sensibile a piccole differenze** — appropriato per le embedding CS2 dove i tick consecutivi sono molto simili.

-vl_jepa_concept_loss() — Allineamento Concettuale

File: `jepa_model.py` | **Signature:** `(concept_logits, concept_labels, concept_embeddings, α=0.5, β=0.1) → (total, concept, diversity)`

Due componenti:

Componente	Formula	Peso	Scopo
Concept alignment	<code>BCE_with_logits(logits, labels)</code>	$\alpha = 0.5$	Multi-label: ogni concetto attivato indipendentemente
Diversity (VICReg)	<code>-mean(std(emb_norm, dim=0))</code>	$\beta = 0.1$	Previene collapse: penalizza bassa varianza delle embedding concettuali

I 16 concetti di coaching (`COACHING_CONCEPTS`) sono organizzati in 5 dimensioni: positioning (0-2), utility (3-4), decision (5-6, 11-12), engagement (7-10) e psychology (13-15) — la tassonomia completa è documentata nella Parte 1A.

-compute_sparsity_loss() — Sparsità Gate RAP

File: `experimental/rap_coach/model.py` | **Signature:** `(gate_weights) → scalar`

```
L_sparsity = context_gate_l1_weight × (-∑ p · log p)      # entropia del gate MoE, range [0, log 4]
```

Entropia sulla distribuzione softmax del gate MoE, scalata da `context_gate_l1_weight` (default 1e-4 da `HeuristicConfig`). L'obiettivo è un routing **deciso**: entropia bassa = pochi esperti dominanti — questo rende il modello **interpretabile** (quale esperto ha davvero deciso?).

Nota F3-07: I `gate_weights` vengono passati come parametro esplicito anziché letti da `self._last_gate_weights` — previene race condition in ambienti multi-thread.

-Position Loss RAP — Penalità Z-axis

File: `experimental/rap_coach/trainer.py` | **Signature:** `(pred_delta, target_delta) → (loss, z_error)`

```
L_pos = MSE(Δx) + MSE(Δy) + 2.0 × MSE(Δz)
```

Il peso 2× sull'asse Z riflette l'importanza critica della verticalità nel CS2: confondere il piano terra con il primo piano (es. Nuke, Vertigo) porta a decisioni tattiche completamente errate.

-Loss Legacy e Win Probability

- **TeacherRefinementNN:** `MSELoss` standard su predizioni supervised (train.py)
- **WinProbabilityNN:** `BCELoss` per output sigmoid [0,1] → probabilità CT-win